

PESU Center for
Information Security,
Forensics and
Cyber Resilience



Welcome to

PES University

Ring Road Campus, Bengaluru



PESU Center for
Information Security,
Forensics and
Cyber Resilience



Applied Cryptography

UE24CS342AA4

Dr. Shruti Jadon

LECTURE: From AES to Public-Key Cryptography

A Note on Security



Emergency Exit



Assembly Point



Washroom



No Chatting



Phones on silent



No Sleeping

A Note on Security

- In this course, you will be exposed to information about security problems and vulnerabilities with computing systems and networks.
- To be clear, you are not to use this or any other similar information to test the security of, break into, compromise, or otherwise attack, any system or network without the express consent of the owner.
- In particular, you will comply with all my instructions when doing the labs.
  - My instructions are in consonance with applicable laws of India and PES University policies.
  - If in any doubt, please consult your professor!
- Any violation is at **YOUR RISK!**
  And may result in severe consequences.

From AES to Public-Key Cryptography

Why AES is still essential — and why public-key cryptography is needed



A conceptual bridge from Unit 2 (AES) to Unit 3 (Public-Key Cryptography)

Lecture Roadmap

01 Setting the Scene
Course context and the two questions cryptography must answer

02 Key Distribution & Scalability
Why shared secrets become hard to manage — and what public keys change

03 Cost & Performance: AES vs. RSA
Why public-key operations are expensive and why AES is fast

04 Hybrid Cryptography
Protect the key, not the file — RSA example and key agreement

05 AES Security in Context
Algorithm vs. implementation, and what AES is (and is not) responsible for

06 Hybrid Systems in Practice
The hybrid model, TLS/HTTPS, and a classroom thought experiment

07 Review & Transition to Unit 3
Misconceptions, one-slide summary, and the bridge to public-key cryptography

Guiding idea
Public-key cryptography complements AES — it solves a different problem.

SECTION 01

Setting the Scene

Course context and the two questions cryptography must answer

Where We Are in the Course

1

Classical ciphers

→ limitations of substitution/transposition-based approaches

2

Symmetric-key cryptography

→ same secret key is used by communicating parties

3

DES

→ important historical symmetric standard, but key-size/security limitations made replacement necessary

4

AES

→ modern symmetric block cipher designed for strong security and efficient implementation

Next question: **If AES is so useful, why do we need public-key cryptography at all?**

The Two Questions Cryptography Must Answer

Question 1

— How do we encrypt a large amount of data efficiently?

- Symmetric cryptography, such as AES, is extremely well suited to this.

Question 2

— How do two parties establish cryptographic secrets when they do not already share one?

- This is the key-establishment / key-distribution problem.
- Public-key cryptography provides mechanisms that address this problem.

Important: **AES itself is not a key-distribution mechanism.**

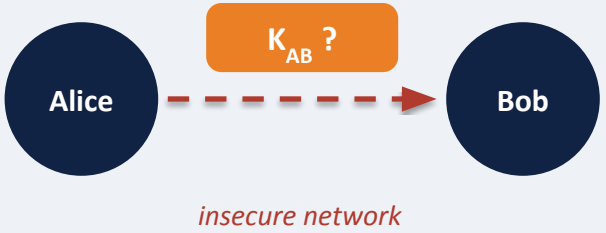
SECTION 02

Key Distribution & Scalability

Why shared secrets become hard to manage — and what public keys change

Why Key Distribution Becomes a Problem

1. Symmetric cryptography requires communicating parties to possess the same secret key.
2. For Alice and Bob, the basic requirement is a shared secret K_{AB}.
3. The difficult question is: how does Alice securely give K_{AB} to Bob if they do not already have a secure channel?
4. Sending K_{AB} over an ordinary insecure network defeats the purpose.
5. **Therefore, symmetric encryption has a key-establishment problem in addition to the encryption problem.**

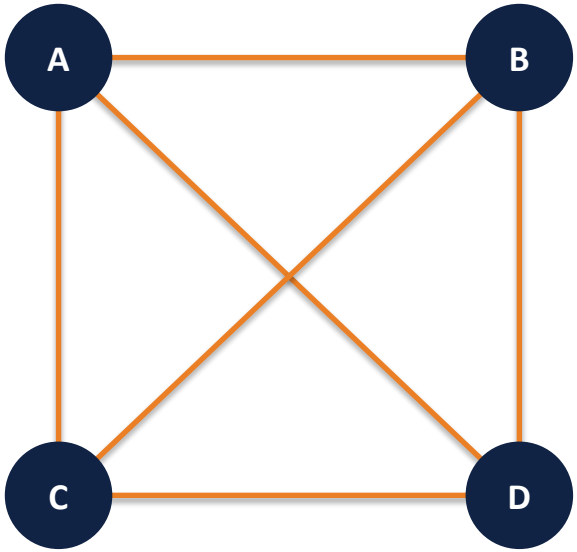


Encryption problem
solved by the symmetric cipher

Key-establishment problem
still open

Key Scalability: Four Users

Number of pairwise symmetric keys = C(n,2) = n(n−1)/2



6 edges = 6 pairwise keys

For 4 users: C(4,2) = 4×3/2 = 6 keys.

- A–B, A–C, A–D, B–C, B–D, C–D.
- Each pair needs a different secret if pairwise confidentiality is required.

Adding a fifth user

Adding another user E may require new shared secrets with A, B, C and D → 4 additional pairwise keys.

How Fast Does the Symmetric Key Count Grow?

10 users →	100 users →	1,000 users →	10,000 users →
45	**4,950**	**499,500**	**49,995,000**
pairwise keys	pairwise keys	pairwise keys	pairwise keys

The exact deployment architecture can use key servers, group keys, or other mechanisms, but the basic pairwise-key scaling problem is **$O(n^2)$**.

Important:
The problem is not that symmetric encryption is weak. The problem is that managing pairwise shared secrets does not scale naturally.

What Changes in a Public-Key System?

Each user generates a key pair:

Public key
→ intended to be shared.

Private key
→ kept secret by its owner.

one user = one key pair

1. If n users exist, the basic key-pair inventory grows linearly with n rather than requiring a separate secret for every pair.

2. At the simplest level: n users → n public/private key pairs (2n individual keys).

3. A new user therefore contributes one new key pair, not a new secret for every existing pair.

A Crucial Clarification About “Only Two Keys”

✔ **Say:**

“Each new user has one public/private key pair.”

✘ **Do not say:**

“The number of keys does not increase.” It does increase.

The important scaling difference is:

Symmetric pairwise model → approximately n(n−1)/2 shared keys.

Public-key model → approximately n key pairs.

This changes the basic key-management growth from quadratic to linear.

Symmetric: $O(n^2)$ vs. Public-key: $O(n)$

SECTION 03

Cost & Performance: AES vs. RSA

Why public-key operations are expensive and why AES is fast

Then Why Not Throw Away AES?

Because symmetric and public-key cryptography solve different problems.

AES

AES is designed for efficient encryption of bulk data.

Public-key

Public-key operations are useful for key establishment, authentication, and digital signatures.

- Using a public-key algorithm for every byte of a large file would be inefficient and unnecessary.
- Modern systems therefore combine the two approaches.

Important:
Public-key cryptography does not make symmetric cryptography obsolete. It complements it.

Why Is Public-Key Cryptography More Expensive?

1. **Algorithm length is not a measure of computational cost.**
2. RSA can look mathematically short: C = M^e mod n.
3. But RSA requires modular exponentiation involving very large integers.
4. With RSA-2048, the modulus is 2048 bits wide.
5. **Each large-integer multiplication/reduction is substantially more expensive than the small operations used repeatedly by AES.**

RSA-2048 modulus

2048

bits wide

vs. AES block

128

bits

RSA Looks Simple — What Is Actually Expensive?

$$C = M^e \bmod n$$

The compact equation hides the work performed by modular exponentiation.

1. Practical implementations use techniques such as square-and-multiply to compute large powers efficiently.
2. Even optimized modular exponentiation involves many arithmetic operations on large integers.
3. The apparent simplicity of the mathematical notation does not mean the computation is cheap.
4. Private-key operations can involve different exponent sizes and optimizations; performance depends on the exact operation and implementation.

Why AES Is Fast for Bulk Encryption

AES operates on fixed-size 128-bit blocks.



16 bytes = 128 bits

AES-128 **AES-192** **AES-256**

AES-128, AES-192 and AES-256 differ mainly in key length; the block size remains 128 bits.

Round operations

Substitution	**Row shifting**
Column mixing	**XOR round key**

Core operations include substitution, permutation/row shifting, column mixing, and XOR with round keys.

These operations are highly regular and efficient to implement.

Modern processors can provide dedicated AES acceleration (for example, AES-NI on many x86 systems).

Therefore AES can achieve very high throughput for large streams of data.

The Important Comparison

RSA

RSA: relatively fewer high-cost operations on very large integers.

Think: *“large arithmetic.”*

AES

AES: many low-cost operations on fixed-size blocks.

Think: *“small, regular, highly optimized operations.”*

- Therefore, algorithm size on a page is not a meaningful indicator of speed.
- Exact performance depends on implementation, hardware, key size, mode, and operation being measured.

A short algorithm can be computationally expensive; a long algorithm can be extremely fast when its operations are simple and hardware-optimized.

A Simple Bulk-Data Example

Suppose Alice must send a 1 MB file to Bob.

1 MB file → **÷ 16-byte blocks** → **≈ 62,500 AES blocks**

AES

AES uses 128-bit (16-byte) blocks, so the data can be processed as roughly 1,000,000 / 16 ≈ 62,500 blocks (ignoring padding/mode details).

AES is designed precisely for this repeated bulk-processing workload.

RSA

RSA is not a bulk-data encryption algorithm: its message size is constrained by the modulus and padding scheme, and each operation involves expensive big-integer arithmetic.

Therefore, using RSA repeatedly for a large file is the wrong design choice.

SECTION 04

Hybrid Cryptography

Protect the key, not the file — RSA example and key agreement

The Best Mental Model: “Protect the Key, Not the File”

1. **Generate a random symmetric session key Ks.**
2. **Use a public-key mechanism to establish or protect Ks.**
3. **Use Ks with AES to encrypt the actual large data.**

Thus the expensive public-key operation is applied to a small cryptographic object, while AES handles the bulk data.



Simplified RSA-Based Hybrid Example

1. Alice generates a random session key Ks.



2. She protects Ks using Bob's public key (conceptual RSA example).



3. Bob uses his private key to recover Ks.



4. Both now have Ks and can use AES to encrypt/decrypt the bulk data.



Important Real-World Refinement: Key Agreement

⚠ **Not True- "RSA transfers the AES key" as a universal description of modern systems.**

Simplified example

RSA can be used in a simplified hybrid-encryption example to protect a session key.

Modern protocols

In modern protocols, public-key key-agreement mechanisms such as Diffie–Hellman variants (including elliptic-curve forms) are commonly used to establish shared secrets.

The general idea remains the same:



SECTION 05

AES Security in Context

Algorithm vs. implementation, and what AES is (and is not) responsible for

Why Do We Say AES Is Still Secure?

✘ Avoid

"AES has never been attacked." That is too absolute.

✔ More accurate

A more accurate statement is: no practical attack is known that breaks properly implemented full-round AES at its intended security levels.

Cryptanalysis

There are published attacks on reduced-round variants and theoretical attacks with impractical requirements.

Implementation

Real systems can also be broken through implementation weaknesses even when the AES primitive itself remains secure.

AES Algorithm vs. AES Implementation

An algorithm can be mathematically strong while an implementation is insecure.

Examples of implementation/system problems:

- Timing leakage
- Power or electromagnetic side channels
- Cache-based side channels
- Weak random-number generation
- Poor key storage or key management
- Incorrect IV/nonce handling or insecure modes

Therefore: secure algorithm ≠ automatically secure system.

Security = algorithm + implementation + key management + protocol

A Student Question: “If AES Has a Key Problem, Is AES Defective?”

No. AES is an encryption primitive, not a complete key-management system.

What AES assumes

AES assumes the communicating parties have an appropriate secret key.

The separate problem

The separate system-design problem is how that secret becomes available to both parties securely.

Who addresses it

Public-key cryptography, key-agreement protocols, certificates, trusted infrastructure, and key-management systems address parts of that larger problem.

Analogy

a lock can be excellent at locking a door even though it does not deliver the key to the person who needs it.

SECTION 06

Hybrid Systems in Practice

The hybrid model, TLS/HTTPS, and a classroom thought experiment

The Hybrid Cryptographic Model

Public-key cryptography:

- Addresses key establishment and/or authentication-related tasks.
- More computationally expensive for bulk encryption.

Symmetric cryptography:

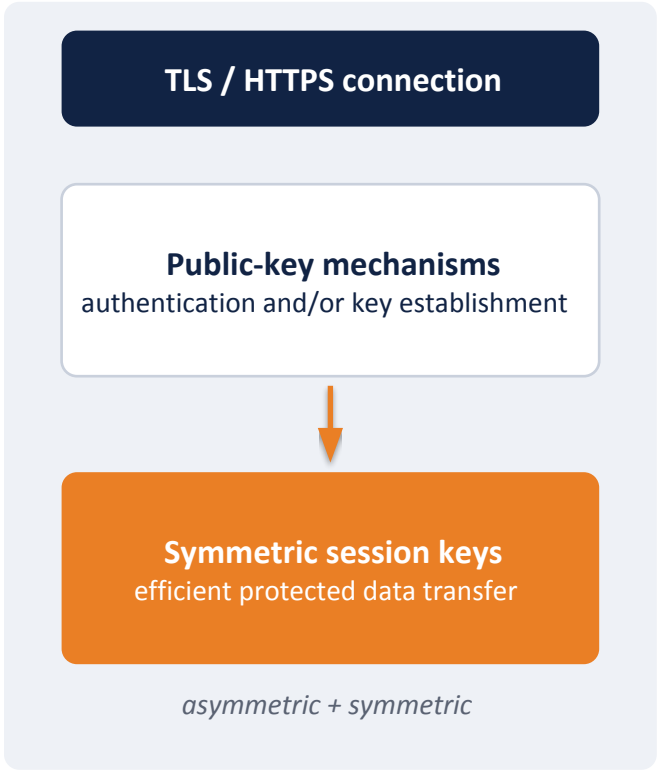
- Encrypts the actual application data efficiently.
- Excellent throughput for large data volumes.

Hybrid systems combine the strengths of both.



Where Students Encounter This in Practice

1. Secure web communication (TLS/HTTPS) is a useful motivating example.
2. The protocol does not simply choose “RSA instead of AES.”
3. It uses public-key mechanisms for authentication and/or key establishment, then uses symmetric session keys for efficient protected data transfer.
4. The exact algorithms depend on the protocol version and negotiated cipher suite.
5. **This is why a modern secure connection can involve both asymmetric and symmetric cryptography.**



An Experiment

Q “You have to send a 10 GB video securely. Would you RSA-encrypt the entire video?”

A **Expected answer:** No.

Q “Then why do we need RSA/public-key cryptography?”

A **Expected answer:** To help establish/protect the cryptographic secret and/or authenticate the communicating party.

Q “Which algorithm handles the 10 GB?”

A **Answer:** A symmetric encryption scheme such as AES.

Key point

The public-key system handles the difficult setup problem; the symmetric cipher handles the heavy lifting.

SECTION 07

Review & Transition to Unit 3

Misconceptions, one-slide summary, and the bridge to public-key cryptography

Common Misconceptions to Correct

Misconception	Correction
❌ “Public-key cryptography replaces symmetric cryptography.”	**Correct:** They are complementary.
❌ “AES is insecure because it has a key-distribution problem.”	**Correct:** key distribution is a system-level problem; AES is an encryption primitive.
❌ “RSA is faster because its formula looks shorter.”	**Correct:** RSA performs expensive large-integer modular arithmetic.
❌ “AES has never been attacked.”	**Correct:** distinguish full-round AES, reduced-round cryptanalysis, and implementation attacks.
❌ “A new public-key user adds only two keys to the entire system.”	**Correct:** the user adds one public/private key pair; the total key inventory grows linearly.

Summary

1. Symmetric cryptography is fast and efficient for bulk data.
2. Symmetric key management becomes difficult as the number of communicating parties grows.
3. Pairwise symmetric keys scale as $n(n-1)/2$.
4. Public-key systems give each user a public/private key pair and avoid requiring a unique pre-shared secret for every pair.
5. Public-key operations are computationally more expensive, especially because of large-integer arithmetic in schemes such as RSA.
6. AES remains essential because it is excellent for bulk encryption.
7. **Modern systems commonly combine the two: public-key/key agreement for setup + symmetric encryption for data.**

Suggested Transition to Unit 3

“We have solved the problem of efficient encryption with AES.”

“But we have not completely solved the problem of establishing the shared secret.”

“What if Alice and Bob have never met and have no pre-shared secret?”

“Can they still establish a secret over an insecure network?”

→ This is the question that leads us to public-key cryptography.

Next concepts:

public/private keys → **RSA** → **Diffie–Hellman** → **digital signatures** → **certificates** → **hybrid cryptography.**

Opening question:
How can two strangers agree on a secret without sending the secret itself over the network?

Thank you!

Follow us



isfcr.pesu



www.isfcr.pes.edu



ISFCR

PESU ISFCR

PESU Center for
Information Security, Forensics and Cyber Resilience

PES UNIVERSITY



PESU Center for
Information Security,
Forensics and
Cyber Resilience



Welcome to

PES University

Ring Road Campus, Bengaluru



PESU Center for
Information Security,
Forensics and
Cyber Resilience



Applied Cryptography

UE24CS342AA4

Dr. Shruti Jadon

LECTURE 20

A Note on Security



Emergency Exit



Assembly Point



Washroom



No Chatting



Phones on silent



No Sleeping

A Note on Security

- In this course, you will be exposed to information about security problems and vulnerabilities with computing systems and networks.
- To be clear, you are not to use this or any other similar information to test the security of, break into, compromise, or otherwise attack, any system or network without the express consent of the owner.
- In particular, you will comply with all my instructions when doing the labs.
  - My instructions are in consonance with applicable laws of India and PES University policies.
  - If in any doubt, please consult your professor!
- Any violation is at **YOUR RISK!**
  And may result in severe consequences.

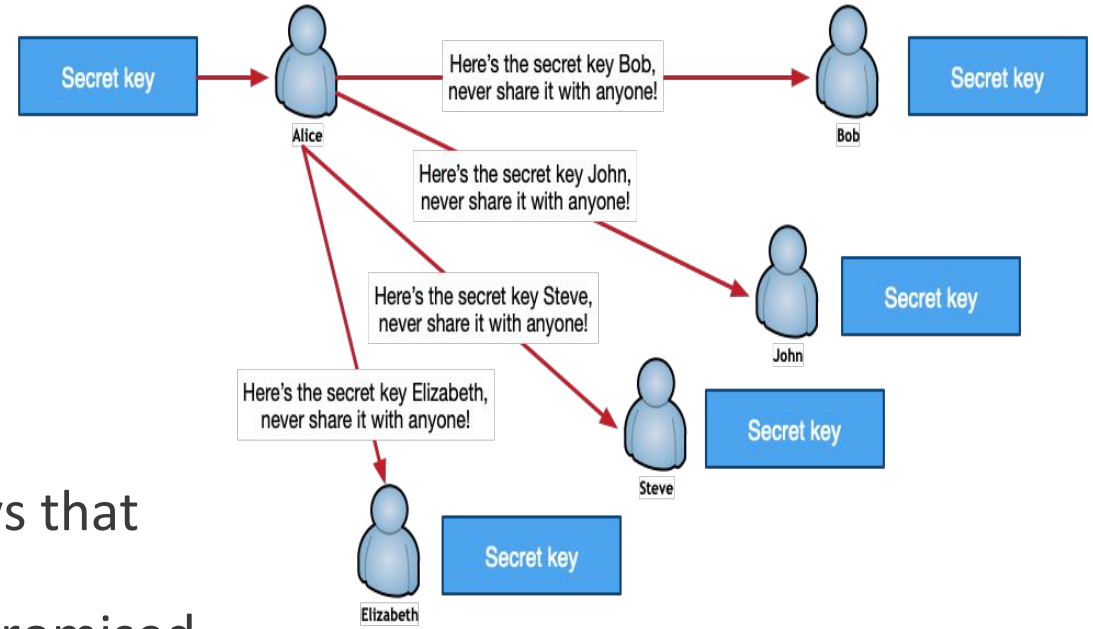
Public Key Cryptography

PKC

Advantages and Disadvantages of Symmetric Key Crypto Methods:

Advantages:

- Extremely Secure
- Relatively Fast



Disadvantages:

- Exchange of the key.
- The number of the keys that are required.
- More Damage if Compromised.

Key distribution

- Symmetric key cryptography: Alice and Bob share a common secret key.
- Some means of distributing a copy of the secret key physical distribution.

Network example

- Assume that n users are connected in a network and any two of them
  may want to communicate.
- This would require each user to securely store n − 1 different symmetric keys (one for each other user), resulting in a total of n(n − 1)/2 keys.
- If the network is connecting 2000 university students, then there will be roughly 2 million different keys.
- A huge key management problem with questions like:
  - **How do you add add a new user to the system?**
  - **What if a user's key is compromised?**
  - **How long should a key be considered valid and how should we refresh them?**

Key distribution

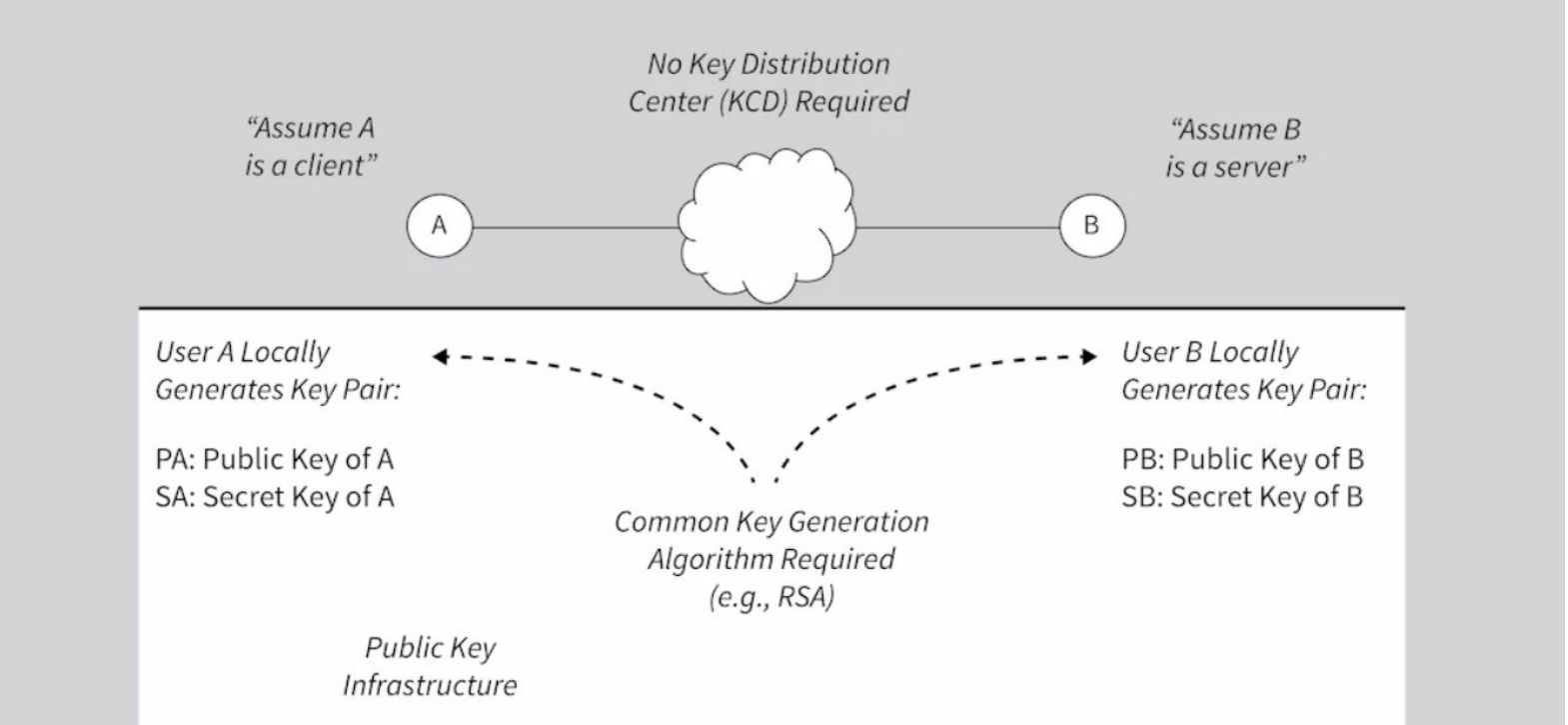
- Better solution to the key distribution problem is obtained if we use symmetric key distribution protocols.
- A trusted third party (TTP). Each user has a unique secret key shared with the TTP.
- When two users would like to communicate, they establish a shared secret key, usually called a session key, by interacting with the TTP.
- There are still drawbacks that can be serious problems in certain situations.
- For example, we need access and trust to a TTP and we still need to distribute one key shared with the TTP for each user.

Public Key Cryptography

☞ An extremely elegant solution...

- We assume two different keys, one for encryption of a plaintext message and another for decryption of a ciphertext message.
- Allow the encryption key to be public!
- Anyone with access to the public encryption key to send an encrypted plaintext to the receiver

Asymmetric/Public Key Encryption was born



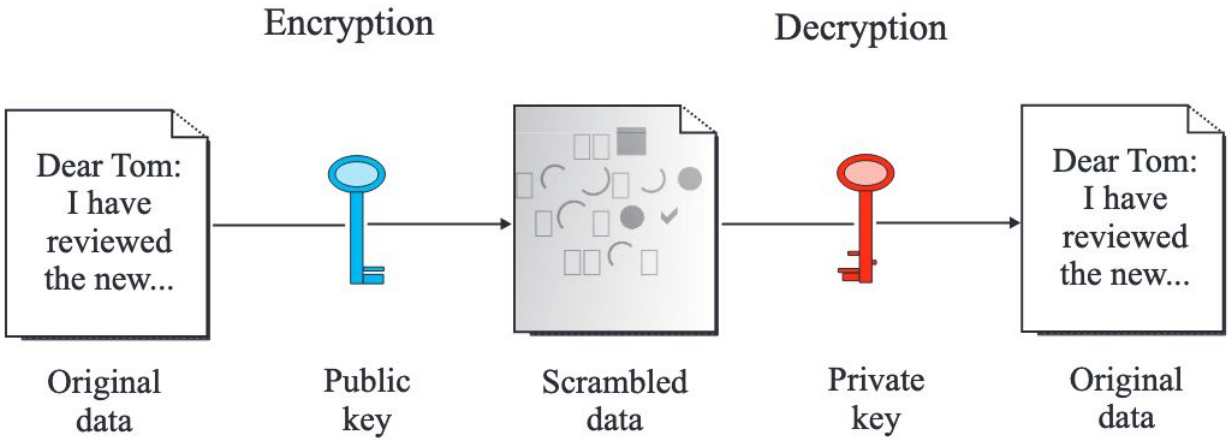
Public Key Cryptography Early History

- Proposed by Diffie and Hellman, documented in “New Directions in Cryptography” (1976)
  - Public-key encryption schemes
  - Key distribution systems
  - Diffie-Hellman key agreement protocol
  - Digital signature
- Public-key encryption was proposed in 1970 in a classified paper by James Ellis
- paper made public in 1997 by the British Governmental Communications Headquarters
- Concept of digital signature is still originally due to Diffie & Hellman

Public Key Cryptography

- In 1976, Martin Hellman, Whitfield Diffie, and Ralph Merkle, have introduced the concept of public-key cryptography.
- In 1977, Ronald L. Rivest, Adi Shamir and Leonard M. Adleman proposed the first public-key cipher which is still secure and used (it is known as RSA).
- Public key cryptography or asymmetric cryptography –
  - one of the prominent cryptographic techniques used in present times.

Idea of public-key cryptography

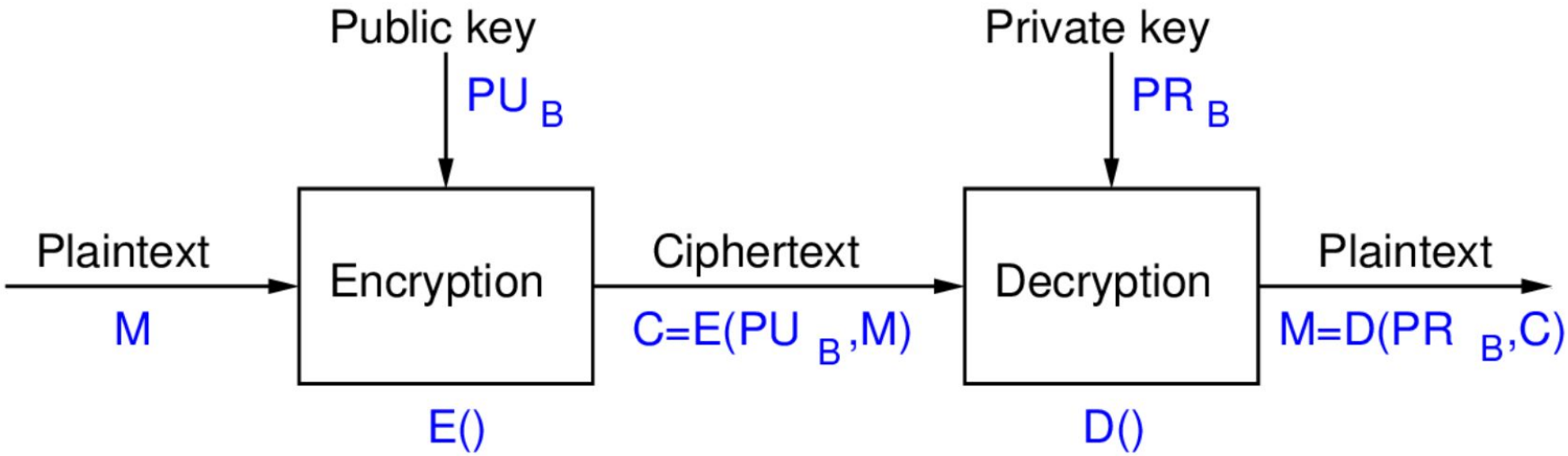




Idea of public-key cryptography

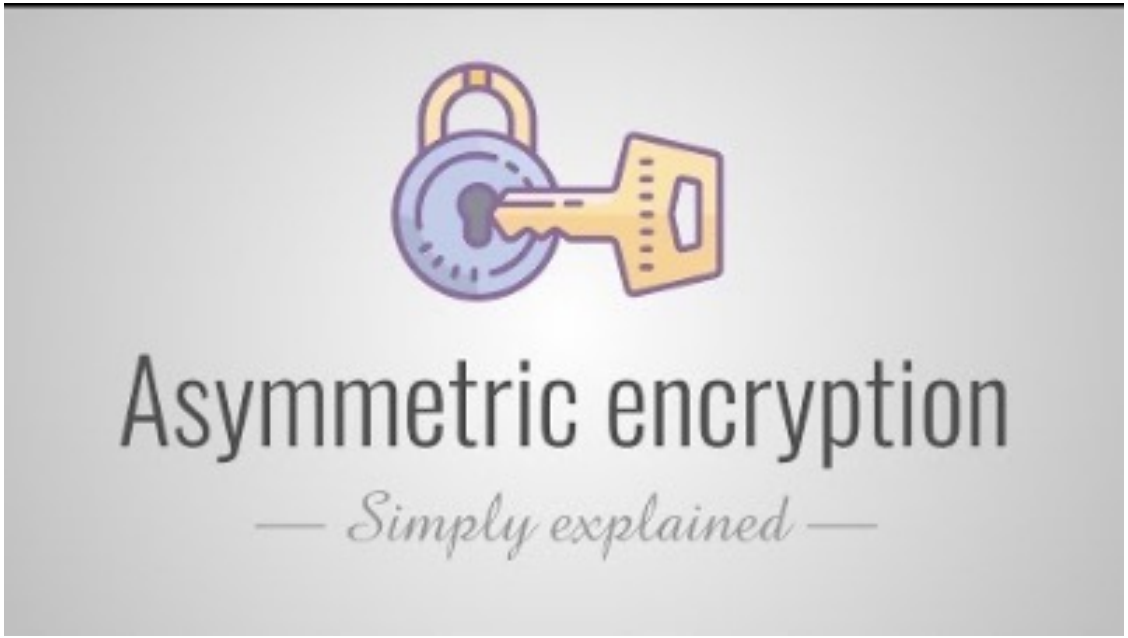
- User generates a related pair of keys – public and private
- Private key can't be computed from the public key efficiently.
- Public-key:
  - Used for encryption, verification of signatures, etc.
  - Can be distributed freely, i.e. anyone can encrypt data for the user or verify user's signatures
- Private-key:
  - Used for decryption (asymmetric encryption schemes), signing (digital signature schemes), etc.
  - Should be kept Private.

Overview



- User A is sender, user B is receiver
- Encrypt using receivers public key, PU_B
- Decrypt using receivers private key, PR_B
- Only B has PR_B, therefore only B can successfully decrypt → confidentiality

Overview



Security Mechanisms of Public-Key Cryptography

☞ Here are main mechanisms that can be realized with asymmetric cryptography:

- **Key Distribution** (e.g., Diffie-Hellman key exchange, RSA) without a pre-shared secret (key)
- **Nonrepudiation and Digital Signatures** (e.g., RSA, DSA or ECDSA) to provide message integrity.
- **Identification** using challenge-response protocols with digital signatures
- **Encryption** (e.g., RSA / ElGamal)
- **Disadvantage:** Computationally very intensive (1000 times slower than symmetric Algorithms)

Summary :

- Public key ciphers consist of:
  - Key generation algorithm
  - Encryption algorithm
  - Decryption algorithm
- Designed around computationally hard mathematical problems
- Very hard to solve without key, i.e. trapdoor functions
  - Finding prime factors of large integers
  - Solving logarithms in modulo arithmetic
  - Solving logarithms on elliptic curves

Security of Public Key Schemes

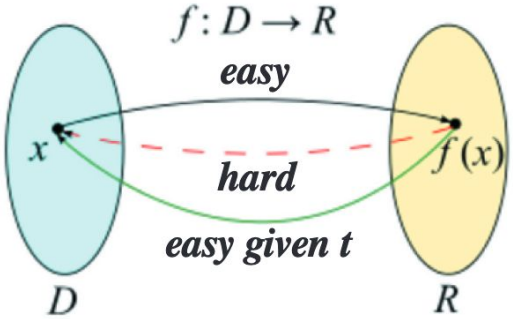
- Like private key schemes brute force exhaustive Search attack is always theoretically possible.
- But keys used are too large (>512bits)
- Security relies on a large enough difference in difficulty between easy (en/decrypt) and hard (cryptanalyse) problems.

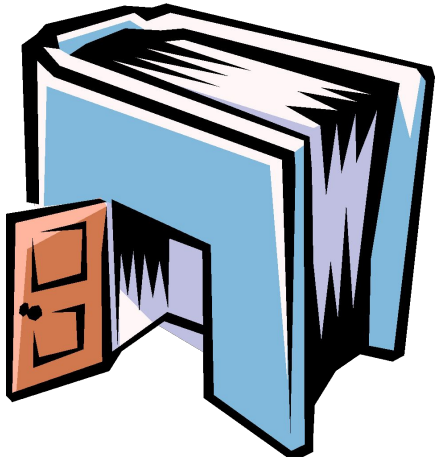
Public Key Requirements

- Need a **trapdoor one-way** function
- One-way function has
  - $Y = f(X)$ easy
  - $X = f^{-1}(Y)$ infeasible
- A trap-door one-way function has
  - $Y = f_k(X)$ easy, if k and X are known
  - $X = f_k^{-1}(Y)$ easy, if k and Y are known
  - $X = f_k^{-1}(Y)$ infeasible, if Y known but k not known
- A practical public-key scheme depends on a suitable trap-door one-way function

Trapdoor one-way function

- A trapdoor function is a special case of one-way function **(F : X → Y)** that contains a “trapdoor.”
- The term “trapdoor” refers to specific information usually obtained during the process of creating a TOWF that can provide an easily computable inverse of a said function.





One-way functions and trapdoor one-way functions

- A **one-way function** f (x) is a function from a set X to a set Y such that f (x) is easy to compute for all $x \in X$, but for “essentially all” elements $y \in Y$ it is “computationally infeasible” to find any $x \in X$ such that f (x) = y.
- A **trapdoor one-way function** f (x) is a one-way function $f : X \rightarrow Y$ such that if one knows some specific information T , called the **trapdoor information**, then f (x) is computationally easy to invert f , i.e., for any $y \in Y$ it is easy to find a $x \in X$ such that f (x) = y.
- For anyone without knowledge of the trapdoor information T , f (x) is a one-way function

Types of One Way Functions

- **Strong One-Way Function** : A Strong One-Way function is a function which is easy to compute and can be inverted only with a negligible probability on a random input or it is hard to invert on all but a negligible fraction of inputs.
- **A Weak One-Way function** is a function which is easy to compute and slightly hard to invert for random inputs or easy to invert on some non-negligible fraction of the inputs.

Relationship of One-Way Functions and Cryptography

- Secure encryption and MAC schemes imply/require the existence of one-way functions
- Given a one-way function, one can construct PRG, PRF, PRP
  - Thus, one can construct secure encryption and MAC schemes
- One-way functions are foundation of modern cryptography theory

EXAMPLES OF ONE - WAY FUNCTIONS

☞ Hash Function

- A hash function H accepts a variable-size message M as input and outputs a fixed-size representation H(M) of M.

☞ RSA Function

- 1977 by Ron **Rivest**, Adi **Shamir** and L. **Adleman**

Review : Euler Totient Function

- The Euler totient function $\phi(n)$ calculates the number of elements in $Z*n$with the help of the following rules –
  - $\emptyset(1) = 0$.
  - $\emptyset(P) = P - 1$ if P is a Prime.
  - $\emptyset(m \times n) = \phi(m) \times \phi(n)$ if m and n are relatively prime.
  - $\emptyset(P^e) = P^e - P^{e-1}$ (if P is a prime)
- It plays an essential role in cryptography.
- It can discover the number of integers that are both smaller than n and relatively prime to n.

Euler's Theorems

Euler's Theorem

Given integer n > 1, such that gcd(a, n) = 1 then

$$a^{\Phi(n)} \equiv 1 \pmod{n}$$

Corollary: Given integer n > 1, such that gcd(a, n) = 1 then $a^{\Phi(n)-1}$ mod n is a multiplicative inverse of a mod n.

Corollary: Given integer n > 1, x, y, and a positive integers with gcd(a, n) = 1. If $x \equiv y \pmod{\Phi(n)}$, then

$$a^x \equiv a^y \pmod{n}.$$

Review : Primality testing

☞ How to check whether a given number m is a prime or not?

- Naive approach: test by trial division, i.e., we check if $x|m$ for all integers x, where $2 \leq x \leq \sqrt{m}$.
- The problem is that if m is a 1024-bit number then $\sqrt{m}$ is of size 512 bits and 2^{512} tests is not possible.

Probabilistic algorithms for testing primality

- **Fermat's Little Theorem**
  - if $a^{m-1} = 1 \bmod m$,
    m is a prime and a such that $1 \leq a \leq m - 1$.
  - if $a^{m-1} \neq 1 \bmod m$ we know for certain that m is not a prime number

Applications of Fermat's little theorem

☞ Three **applications** of Fermat's little theorem and Euler's generalization:

- Primality testing
- RSA public key encryption

Summary

- Public-key algorithms have capabilities that symmetric ciphers don't have, in particular digital signature and key establishment functions.
- Public-key algorithms are computationally intensive (a nice way of saying that they are slow), and hence are poorly suited for bulk data encryption
- Only three families of public-key schemes are widely used. This is considerably fewer than in the case of symmetric algorithms.
- The extended Euclidean algorithm allows us to compute modular inverses quickly, which is important for almost all public-key schemes.
- Euler's phi function gives us the number of elements smaller than an integer n that are relatively prime to n. This is important for the RSA

Thank you!

Follow us



isfcr.pesu



www.isfcr.pes.edu



ISFCR

PESU ISFCR

PESU Center for
Information Security, Forensics and Cyber Resilience

PES UNIVERSITY



PESU Center for
Information Security,
Forensics and
Cyber Resilience



Welcome to

PES University

Ring Road Campus, Bengaluru



PESU Center for
Information Security,
Forensics and
Cyber Resilience



Applied Cryptography

UE24CS342AA4

Dr. Shruti Jadon

A Note on Security



Emergency Exit



Assembly Point



Washroom



No Chatting



Phones on silent



No Sleeping

A Note on Security

- In this course, you will be exposed to information about security problems and vulnerabilities with computing systems and networks.
- To be clear, you are not to use this or any other similar information to test the security of, break into, compromise, or otherwise attack, any system or network without the express consent of the owner.
- In particular, you will comply with all my instructions when doing the labs.
  - My instructions are in consonance with applicable laws of India and PES University policies.
  - If in any doubt, please consult your professor!
- Any violation is at **YOUR RISK!**
  And may result in severe consequences.

Introduction

- Miller–Rabin is a probabilistic primality testing algorithm.
- It determines whether a number is prime or composite.
- It is widely used in cryptography.
- It is useful for generating large prime numbers.
- Applications include RSA, Diffie–Hellman, and digital signatures.

Introduction

- Miller–Rabin is a probabilistic primality testing algorithm.
- It determines whether a number is prime or composite.
- It is widely used in cryptography.
- It is useful for generating large prime numbers.
- Applications include RSA, Diffie–Hellman, and digital signatures.

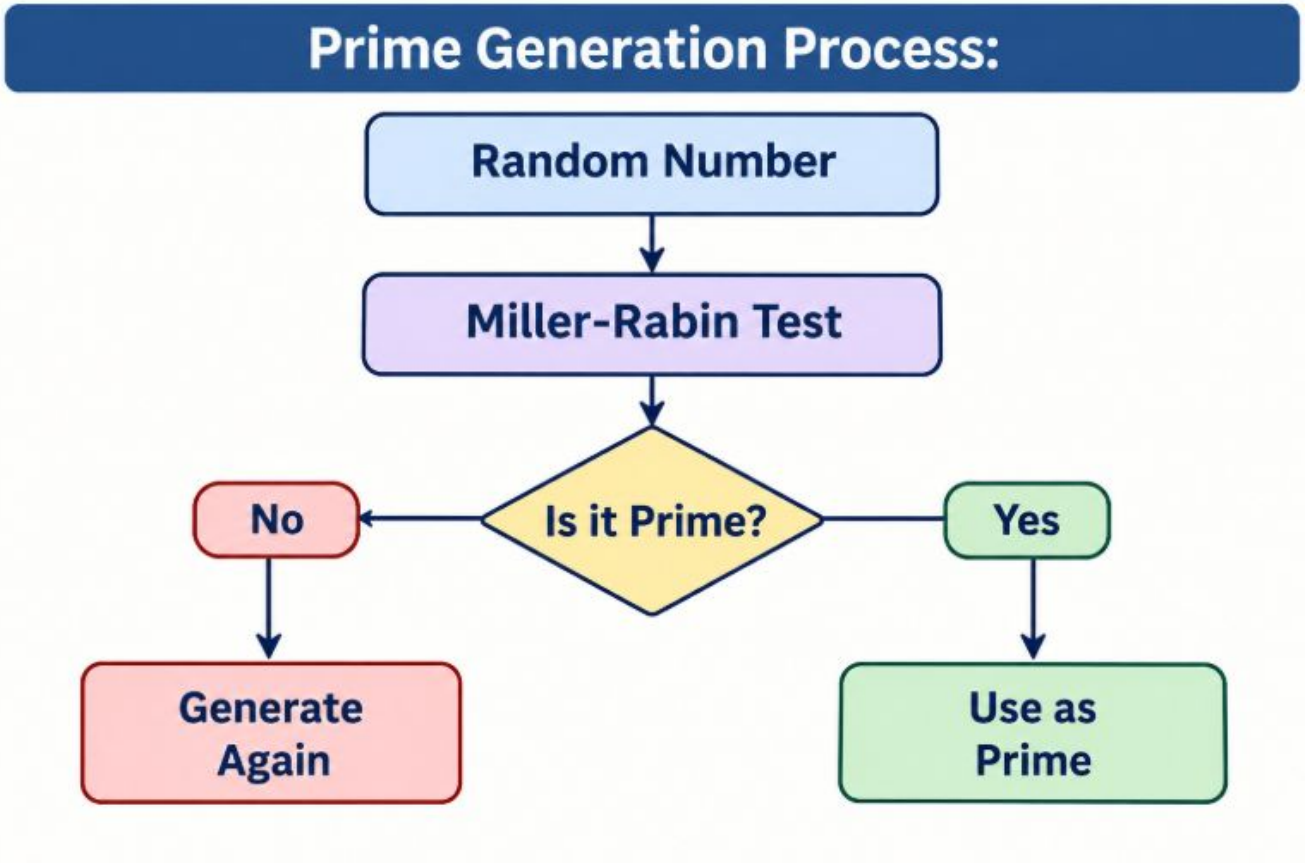
Prime Number

- A prime number is an integer greater than 1.
- It has exactly two factors: 1 and The number itself
- Examples: 2, 3, 5, 7, 11, 13, 17, 19
- Composite numbers have more than two factors.
- Large prime numbers are important in public-key cryptography.

Why do we need large prime numbers?

- Cryptographic algorithms require very large prime numbers.
- Checking large numbers using trial division is inefficient.
- Miller–Rabin provides a fast method for primality testing.
- Random numbers can be generated and tested until a probable prime is found.

Prime Generation Process



Prime Generation Process

For cryptographic applications, a large random prime is usually generated as follows:

Step 1: Generate a random number with the required number of bits.

Step 2: Set the highest bit to ensure the required size.

Step 3: Make the number odd.

Step 4: Perform preliminary small-prime checks.

Step 5: Apply Miller–Rabin with appropriate bases or rounds.

Step 6: If the number is composite: Generate another random candidate.

Step 7: If it passes the required tests: Accept it as a probable prime.

Miller–Rabin Primality Test

Algorithm

Step 1: Find $n-1 = 2^k \times m$

Step 2: Choose 'a' such that $1<a<n-1$

Step 3: Compute $b_0 = a^m \pmod n$, ... , $b_i = b^2_{i-1} \pmod n$

+1 ⟶ Composite

-1 ⟶ Probably Prime

Prime Generation Process

Goal: Generate a random 8-bit prime number (128–255).
ATTEMPT 1

Step 1: Generate random bits
01100110 = 102

Step 2: Set highest bit
11100110 = 230

Step 3: Make it odd
11100111 = 231

Step 4: Small-prime check
231 is divisible by 3 because:
2 + 3 + 1 = 6
Therefore, 231 is composite.
Discard 231 and try again.

Prime Generation Process

ATTEMPT 2

Step 1 & 2: Generate and set highest bit
11011110 = 222

Step 3: Make it odd
11011111 = 223

Step 4: Small-prime checks
223 is not divisible by 3, 5, 7, or 11.

Step 5: Miller-Rabin Test
223 - 1 = 222
222 = 2^1 × 111
So:
k = 1, m = 111

Choose base:
a = 2
Calculate:
2^111 mod 223 = 222 = -1 mod 223
Therefore, 223 passes the Miller-Rabin test.
Step 7: Accept
223 is accepted as a probable prime.

FINAL ANSWER: 223

Example

Question: Is 561 prime?

Solution:

Given n = 561.

Step 1:

$n-1 = 2^k \times m$

$560 = 2^4 \times 35$

So k = 4, and m = 35

$\frac{560}{2^1} = 280$ | $\frac{560}{2^2} = 140$ | $\frac{560}{2^3} = 70$ | $\frac{560}{2^4} = 35$ | $\frac{560}{2^5} = 17.5$

Example

Question: Is 561 prime?

Solution:

Given n = 561.

Step 3:

Compute $b_0 = a^m \pmod n$

$b_0 = a^m \pmod n$

$b_0 = 2^{35} \pmod{561} = 263$

Is $b_0 = \pm 1 \pmod{561}$? NO

So calculate b_1

$b_1 = b_0^2 \pmod n$

$b_1 = 263^2 \pmod{561}$

$b_1 = 166$

Is $b_1 = \pm 1 \pmod{561}$? NO

$b_2 = b_1^2 \pmod n$

$b_2 = 166^2 \pmod{561}$

$b_2 = 67$

Is $b_2 = \pm 1 \pmod{561}$? NO

$b_3 = b_2^2 \pmod n$

$b_3 = 67^2 \pmod{561}$

$b_3 = 1 \longrightarrow$ Composite

$\therefore 561$

nesoacademy.org

MILLER–RABIN

- Miller–Rabin is based on modular arithmetic.
- For an odd number n, write: n − 1 = 2^s × d
- d must be an odd number.
- s is the number of times n − 1 can be divided by 2.

Example: 29 − 1 = 28 ; 28 = $2^2 \times 7$

Therefore: s = 2 d = 7

MILLER–RABIN TESTING STEPS

1. Choose a random number a such that: $2 \leq a \leq n - 2$
2. Calculate: $x = a^d \bmod n$
3. If $x = 1$ or $x = n - 1$: → The number passes this round.
4. Otherwise, repeatedly calculate: $x = x^2 \bmod n$
5. If x becomes $n - 1$: → The number passes.
6. If x never becomes $n - 1$: → The number is composite.

IS MILLER–RABIN ALWAYS TRUE?

- Miller–Rabin is a probabilistic primality test.
- It does NOT always guarantee that a number is prime.
- There are two possible results:

1. Composite → The number is definitely NOT prime.
2. Probably Prime → The number has passed the selected test rounds.

- A composite number can sometimes pass a Miller–Rabin test for a particular base.
- Such a number is called a Miller–Rabin pseudoprime for that base.
- Testing multiple random bases greatly reduces the probability of a false result.
- Therefore, increasing the number of rounds increases confidence.

If Miller–Rabin says "Composite": → It is definitely composite. If Miller–Rabin says "Probably Prime": → It is highly likely to be prime, but the basic probabilistic test does not give an absolute guarantee.

HOW ACCURATE IS MILLER RABIN

- The reliability depends on the number of test rounds.
- Each round uses a different randomly selected base.
- More independent rounds make a false positive extremely unlikely. For a general Miller–Rabin test:
- 1 round → lower confidence
- 5 rounds → very high confidence for many practical applications
- 10+ rounds → extremely high confidence
- More rounds → higher confidence but slightly more computation

HOW ACCURATE IS MILLER RABIN

For a composite number:

The probability that it passes a random Miller–Rabin round is at most: 1/4

After k independent rounds: Probability of false positive ≤ (1/4)^k

Example: k = 5: (1/4)^5 = 1/1024

k = 10: (1/4)^10 = 1/1,048,576

Therefore, multiple rounds make Miller–Rabin very reliable in practice.

Note: For cryptographic applications, the number of rounds and the choice of bases are selected according to the required security level.

HOW HUGE A PRIME NUMBER CAN MILLER–RABIN GENERATE?

- Miller–Rabin itself does not have a fixed maximum prime size.
- It can test numbers with hundreds, thousands, or even more bits.
- The practical limit depends on:

- Computer memory
- Processing power
- Modular exponentiation speed
- Required testing time

HOW HUGE A PRIME NUMBER CAN MILLER–RABIN GENERATE?

Examples of prime sizes:

- 32-bit → relatively small
- 64-bit → commonly handled very quickly
- 128-bit → much larger
- 256-bit → used in some cryptographic applications
- 512-bit → large cryptographic prime
- 1024-bit → large prime used in some cryptographic systems
- 2048-bit and above → can also be tested using Miller–Rabin

Example: Testing for 29

Let’s check if 29 is a prime number using the Miller–Rabin test.

1. DECOMPOSE n − 1

For n = 29,

n − 1 = 29 − 1 = 28

Write 28 in the form 2^s × d:

$28 = 2^2 \times 7$

So, s = 2 and d = 7.

2. CHOOSE A RANDOM BASE

Pick a value of a such that:

$2 \leq a \leq n - 2$

Let’s choose a = 2.

Example: Testing for 29

3. COMPUTE x = a^d mod n

$x = 2^7 \bmod 29$

$x = 128 \bmod 29$

$x = 12$

Since x is neither 1 nor 28, we continue to the next step.

4. REPEAT THE SQUARING STEP

Compute $x = x^2 \bmod n$ (up to $s - 1$ times).

$x = 12^2 \bmod 29$

$x = 144 \bmod 29$

$x = 28$

Since $x = 28 = n - 1$, the number passes this Miller–Rabin round.

5. CONCLUSION

We obtained $n - 1$, so 29 passes this round.

Therefore, 29 is probably prime.

Example: Testing for 53

USING THE MILLER–RABIN TEST (BASE a = 2)

1. DECOMPOSE n − 1
For n = 53:
n − 1 = 53 − 1 = 52
Write 52 in the form 2^s × d:
$52 = 2^2 \times 13$
So,
s = 2
d = 13

2. CHOOSE A RANDOM BASE
Choose a base a such that:
$2 \leq a \leq n - 2$
Let:a = 2

Example: Testing for 53

3. COMPUTE x = a^d mod n
x = 2^13 mod 53
x = 8192 mod 53
x = 30
Since x is neither 1 nor 52, continue to the next step.

4. REPEAT THE SQUARING STEP
Square x modulo 53, up to s − 1 times.
Since s = 2, square once:
$x = 30^2 \bmod 53$
x = 900 mod 53
x = 900 − (53 × 16)
x = 900 − 848
x = 52
Therefore:
x = 52 = 53 − 1 = −1 mod 53
Since we obtained n − 1, 53 passes this Miller–Rabin round.

5. CONCLUSION
53 passes the Miller–Rabin test with base a = 2.
Therefore, 53 is probably prime.
In fact, 53 is prime because it has no divisors other than 1 and 53.

Example: Testing for 561

Using the Miller–Rabin Test (base a = 2)

1. Decompose n - 1
For n = 561:
n - 1 = 561 - 1 = 560
Write 560 in the form 2^s × d, where d is odd:
560 = 2^4 × 35
So,
s = 4
d = 35

2. Choose a Random Base
Choose a base a such that:
2 ≤ a ≤ n - 2
Let:a = 2

Example: Testing for 561

3. Compute x = a^d mod n
x = 2^35 mod 561
x = 263
Since x is neither 1 nor 560 = n - 1,
continue to the squaring step.

4. Repeat the Squaring Step
Since s = 4, square x up to s - 1 = 3 times.
First squaring:
x = 263^2 mod 561
x = 166
Second squaring:
x = 166^2 mod 561
x = 67
Third squaring:
x = 67^2 mod 561
x = 1

We reached:
x = 1
but we never obtained:
x = 560 = -1 mod 561
Therefore, 561 fails the Miller–Rabin test with base a = 2.

5. Conclusion
561 is composite.
In fact,
561 = 3 × 11 × 17

OTHER METHODS FOR GENERATING PRIMES

1. Trial Division

- Divide the number by smaller possible factors.
- If no factor is found up to √n, the number is prime.
- Simple but very slow for large numbers.

2. Sieve of Eratosthenes

- Generates all primes up to a fixed limit.
- Starts with 2 and removes multiples of each prime.
- Very efficient for generating smaller primes.
- Not suitable for extremely large individual primes.

OTHER METHODS FOR GENERATING PRIMES

3. Fermat Primality Test

- Based on Fermat's Little Theorem.
- Uses modular exponentiation.
- Faster than trial division.
- Can be fooled by certain composite numbers called Carmichael numbers.
- Less reliable than Miller–Rabin.

4. Solovay–Strassen Test

- Another probabilistic primality test.
- Uses properties of modular arithmetic and the Jacobi symbol.
- More reliable than simple Fermat testing.
- Miller–Rabin is generally preferred in modern applications.

COMPARISON OF PRIME GENERATION METHODS

Method: Trial Division

- Type: Deterministic
- Speed for large numbers: Slow
- Large cryptographic primes: Not practical
- Main use: Small numbers / basic learning

Method: Sieve of Eratosthenes

- Type: Deterministic
- Speed: Very efficient for a fixed range
- Large individual primes: Not practical
- Main use: Generate many primes up to a limit

COMPARISON OF PRIME GENERATION METHODS

Method: Fermat Test

- Type: Probabilistic
- Speed: Fast
- Reliability: Can fail for some composite numbers
- Main use: Basic primality testing

Method: Solovay–Strassen

- Type: Probabilistic
- Speed: Fast
- Reliability: Better than basic Fermat testing
- Main use: Historical / educational applications

Advantages and Limitations

Advantages:

- Fast for testing large numbers.
- Suitable for cryptographic applications.
- Uses efficient modular exponentiation.
- Works well with numbers having hundreds or thousands of bits.
- Multiple rounds provide very high confidence.

Limitations:

- Basic Miller–Rabin is probabilistic.
- Passing the test means the number is a probable prime.
- There is a small probability of a false positive.
- Increasing the number of rounds reduces this probability.

Applications

- RSA key generation
- Diffie–Hellman key exchange
- Digital signatures
- Cryptographic key generation
- Large prime number generation

Thank you!

Follow us



isfcr.pesu



www.isfcr.pes.edu



ISFCR

PESU ISFCR

PESU Center for
Information Security, Forensics and Cyber Resilience

PES UNIVERSITY



PESU Center for
Information Security,
Forensics and
Cyber Resilience



Welcome to

PES University

Ring Road Campus, Bengaluru



PESU Center for
Information Security,
Forensics and
Cyber Resilience



Applied Cryptography

UE24CS342AA4

Dr. Shruti Jadon

LECTURE 22

A Note on Security



Emergency Exit



Assembly Point



Washroom



No Chatting



Phones on silent



No Sleeping

A Note on Security

- In this course, you will be exposed to information about security problems and vulnerabilities with computing systems and networks.
- To be clear, you are not to use this or any other similar information to test the security of, break into, compromise, or otherwise attack, any system or network without the express consent of the owner.
- In particular, you will comply with all my instructions when doing the labs.
  - My instructions are in consonance with applicable laws of India and PES University policies.
  - If in any doubt, please consult your professor!
- Any violation is at **YOUR RISK!**
  And may result in severe consequences.

RSA

Rivest–Shamir–Adleman algorithm

History

- The RSA algorithm was first described in 1977 by Ron Rivest, Adi Shamir, and Leonard Adleman of MIT
- The RSA cryptosystem is the most widely-used public key cryptography algorithm in the world. You can use it to encrypt a message without the need to exchange a secret key separately.
- There are many applications for RSA, but in practice it is most often used for:
  - Most Public Key Infrastructure (PKI) products.
  - SSL/TLS: Certificates and key-exchange.
  - Secure e-mail: PGP, Outlook, …Digital signatures

Facts about RSA

- The RSA algorithm's security is based on the difficulty of solving certain mathematical problems, such as factoring large integers
- The algorithm uses logarithmic functions to make it complex enough to withstand brute force attacks, but streamlined enough to be fast after deployment
- RSA is one of the first practical public-key cryptosystems and is widely used for secure data transmission. In such a cryptosystem, the encryption key is public and differs from the decryption key which is kept secret.

Key Generation

Unlike symmetric algorithms (AES,3DES) public key algorithms require the computation of the pair (K_{pub}, K_{pr})

Steps involved in computing the public and private-key for an RSA cryptosystem are:

1. Choose two large prime numbers. E.g., p and q
2. Compute n=p*q and z=(p-1)*(q-1).
3. Choose number e such that e is less than n and e has no common factor (other than 1) with z.

4. Compute the modular multiplicative inverse of e modulo z to find the private exponent d. This can be done using the extended Euclidean algorithm. In other words, find number d: such that (ed - 1) is exactly divisible by z.

5. Keys are generated using n, d and e:
   - Public key is (n, e)
   - Private key is (n, d)

Key Generation

- Generate two different primes including p and q.
- Compute the modulus n = p × q
- Compute the totient $\phi(n) = (p - 1) \times (q - 1)$
- Select for public exponent an integer e such that $1 < e < \phi(n)$ and $\gcd(\phi(n), e) = 1$.
- Compute for the private exponent a value for d such that $d = e^{-1} \bmod \phi(n)$
- Public Key = [e, n]
- Private Key = [d, n]

Encryption and Decryption

- RSA encryption and decryption is done in the integer ring $Z_{n.}$
- RSA encrypts plaintexts x, where we consider the bit string representing x to be an element in Z_n = {0,1,...,n−1}.
- As a consequence, the binary value of the plaintext x must be less than n. The same holds for the ciphertext.

RSA Encryption Given the public key $(n, e) = k_{pub}$ and the plaintext x, the encryption function is:

$$y = e_{k_{pub}}(x) \equiv x^e \bmod n \qquad (7.1)$$

where $x, y \in \mathbb{Z}_n$.

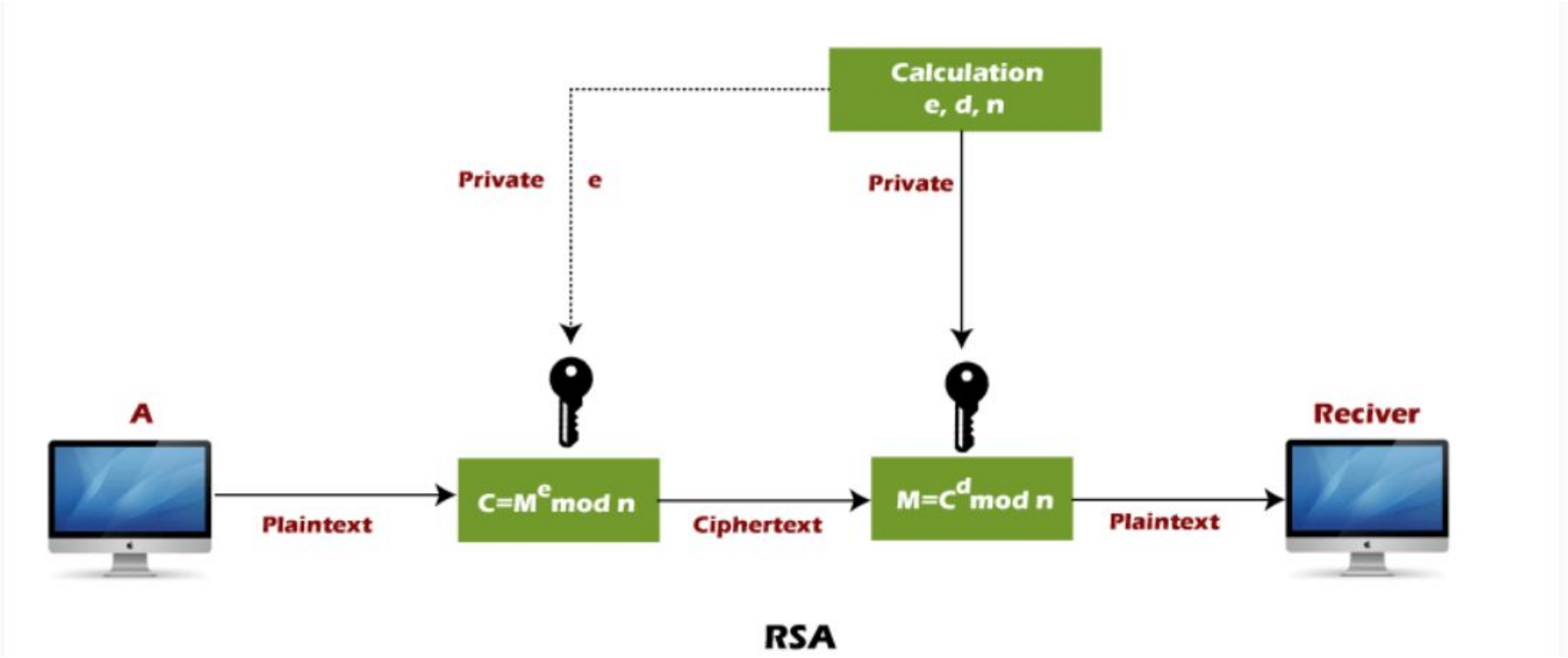
Encryption and Decryption

RSA Decryption Given the private key $d = k_{pr}$ and the ciphertext y, the decryption function is:

$$x = d_{k_{pr}}(y) \equiv y^d \bmod n \tag{7.2}$$

where $x, y \in \mathbb{Z}_n$.

In practice, x, y, n and d are very long numbers, usually 1024 bit long or more. The value e is sometimes referred to as **encryption exponent** or **public exponent**, and the private key d is sometimes called **decryption exponent** or **private exponent**. If Alice wants to send an encrypted message to Bob, Alice needs to have his public key (n,e), and Bob decrypts with his private key d.


Calculation
e, d, n
Private
e
Private
A
Reciver
Plaintext
C=M^e mod n
Ciphertext
M=C^d mod n
Plaintext
RSA

Example 1

How we can encrypt plaintext 9 using the RSA public-key encryption algorithm?

Step 1: Select two large prime numbers, p, and **q**.

- p = 7
- q = 11

Step 2: Multiply these numbers to find **n = p x q,** where **n** is called the modulus for encryption and decryption.

- First, we calculate
- **n = p x q**
- n = 7 x 11
- n = 77

Example 1

Step 3: Choose a number **e** less that **n**, such that n is relatively prime to **(p - 1) x (q -1).** It means that **e** and **(p - 1) x (q - 1)** have no common factor except 1. Choose "e" such that 1<e < φ (n), e is prime to φ (n), gcd (e, d (n)) =1.

- Second, we calculate
- **φ (n) = (p - 1) x (q-1)**
- φ (n) = (7 - 1) x (11 - 1)
- φ (n) = 6 x 10
- φ (n) = 60
- Let us now choose relative prime e of 60 as 7.
- Thus the public key is <e, n> = (7, 77)

Example 1

- **Step 4:** A plaintext message **m** is encrypted using public key <e, n>. To find ciphertext from the plain text following formula is used to get ciphertext C.
- To find ciphertext from the plain text following formula is used to get ciphertext C.
- $\mathbf{C = m^e \bmod n}$
- $C = 9^7 \bmod 77$
- $C = 37$

Example 1

Step 5: The private key is <d, n>. To determine the private key, we use the following formula d such that:

- **D_e mod {(p - 1) x (q - 1)} = 1**
- 7d mod 60 = 1, which gives d = 43
- The private key is <d, n> = (43, 77)

Step 6: A ciphertext message **c** is decrypted using private key <d, n>. To calculate plain text **m** from the ciphertext c following formula is used to get plain text m.

- **$m = c^d \bmod n$**
- $m = 37^{43} \bmod 77$
- m = 9
- In this example, Plain text = 9 and the ciphertext = 37

Example 2

- In an RSA cryptosystem, a particular A uses two prime numbers, 13 and 17, to generate the public and private keys. If the public of A is 35. Then the private key of A is ……?.

d=11

- A RSA cryptosystem uses two prime numbers 3 and 13 to generate the public key= 3 and the private key = 7. What is the value of cipher text for a plain text?

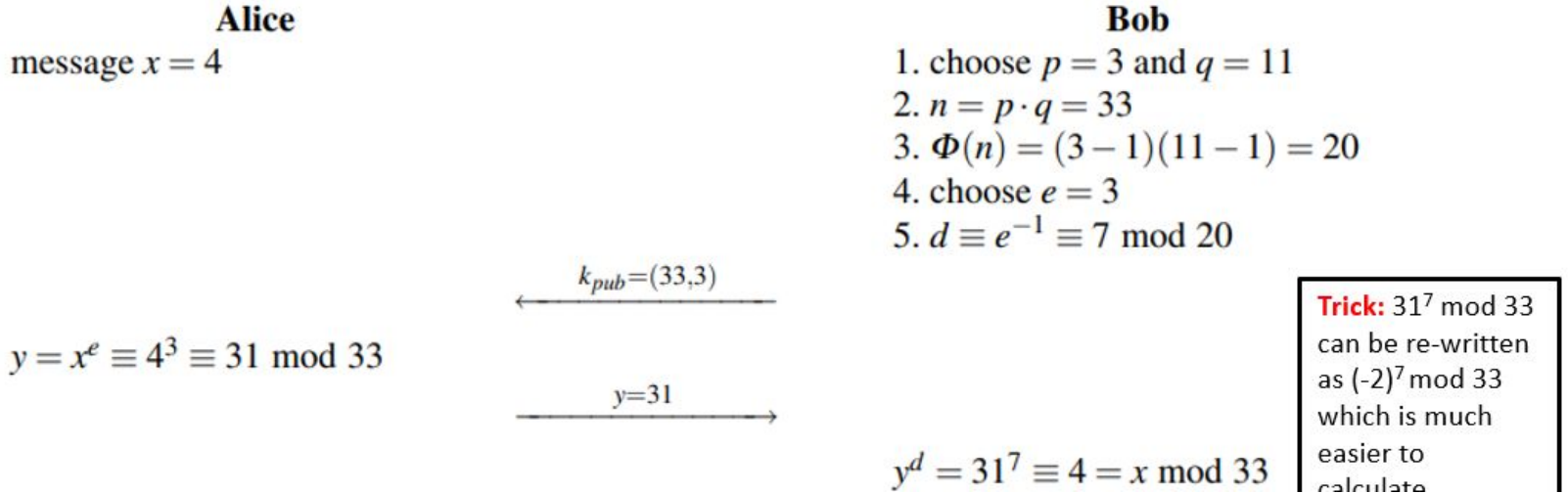
C=8

- A RSA cryptosystem uses two prime numbers, 3 and 11, to generate private key = 7. What is the value of ciphertext for a plain text 5 using the RSA public-key encryption algorithm?

E=3

Example

- Alice wants to send an encrypted message to Bob.
- Bob first computes his RSA parameters. He then sends Alice his public key.
- Alice encrypts the message (x = 4) and sends the ciphertext y to Bob.
- Bob decrypts y using his private key.



Remarks

Practical RSA parameters are much, much larger. The RSA modulus n should be at least 1024 bit long, which results in a bit length for p and q of 512.

- Here is an example of RSA parameters for this bit length:

- **p =** E0DFD2C2A288ACEBC705EFAB30E4447541A8C5A47A37185C5A9 CB98389CE4DE19199AA3069B404FD98C801568CB9170EB712BF 10B4955CE9C9DC8CE6855C6123 $_h$

- **q =** EBE0FCF21866FD9A9F0D72F7994875A8D92E67AEE4B515136B2 A778A8048B149828AEA30BD0BA34B977982A3D42168F594CA99 F3981DDABFAB2369F229640115 $_h$

- **n =** CF33188211FDF6052BDBB1A37235E0ABB5978A45C71FD381A91 AD12FC76DA0544C47568AC83D855D47CA8D8A779579AB72E635 D0B0AAAC22D28341E998E90F82122A2C06090F43A37E0203C2B 72E401FD06890EC8EAD4F07E686E906F01B2468AE7B30CBD670 255C1FEDE1A2762CF4392C0759499CC0ABECFF008728D9A11ADF $_h$

- **e =** 40B028E1E4CCF07537643101FF72444A0BE1D7682F1EDB553E3 AB4F6DD8293CA1945DB12D796AE9244D60565C2EB692A89B888 1D58D278562ED60066DD8211E67315CF8985716720612040 5B0 8B54D10D4EC4ED4253C75FA74098FE3F7FB751FF5121353C554 391E114C85B56A9725E9BD5685D6C9C7EED8EE442366353DC39 $_h$

- **d =** C21A93EE751A8D4FBFD77285D79D6768C58EBF283743D2889A3 95F266C78F4A28E86F545960C2CE01EB8AD5246905163B28D0B 8BAABB959CC03F4EC499186168AE9ED6D88058898907E61C7CC CC584D65D801CFE32DFC983707F87F5AA6AE4B9E77B9CE630E2 C0DF05841B5E4984D059A35D7270D500514891F7B77B804BED81 $_h$

RSA challenge

- One of the first description of RSA was in the paper.
- Martin Gardner: Mathematical games, Scientific American, 1977
- and in this paper RSA inventors presented the following challenge.

- Decrypt the cryptotext:
- 9686 9613 7546 2206 1477 1409 2225 4355 8829 0575 9991 1245 7431 9874 6951 2093 0816 2982 2514 5708 3569 3147 6622 8839 8962 8013 3919 9055 1829 9451 5781 5154

RSA challenge

- Encrypted using the RSA cryptosystem with
- *n*: 114 381 625 757 888 867 669 235 779 976 146 612 010 218 296 721 242 362 562 561 842 935 706 935 245 733 897 830 597 123 513 958 705 058 989 075 147 599 290 026 879 543 541.
- and with *e* = 9007
- The problem was solved in 1994 by first factorizing *n* into one 64-bit prime and one 65-bit prime, and then computing the plaintext
  - THE MAGIC WORDS ARE SQUEMISH OSSIFRAGE

How to design a good RSA cryptosystem

1. How to choose large primes *p,q*?

☞Choose randomly a large integer *p*, and verify, using a randomized algorithm, whether *p* is prime. If not, check *p* + 2, *p* + 4,…

From the Prime Number Theorem if follows that there are approximately

$$\frac{2^d}{\log 2^d} - \frac{2^{d-1}}{\log 2^{d-1}}$$

d bit primes. (A probability that a 512-bit number is prime is 0.00562.)

2. What kind of relations should be between *p* and *q*?

- Difference |*p*-*q*| should be neither too small not too large.
- gcd(*p*-1, *q*-1) should not be large.
- Both *p*-1 and *q*-1 should contain large prime factors.
- Quite ideal case: *q, p* should be safe primes - such that also (*p*–1)/2 and (*q*-1)/2 are primes (83,107,10^{100} – 166517 are examples of safe primes).

3. How to choose *e* and *d*?

- Neither *d* nor *e* should be small.
- *d* should not be smaller than $n^{1/4}$. (For $d < n^{1/4}$ a polynomial time algorithm is known to determine *d*.

How important is factorization for breaking RSA?

1. If integer factorization is feasible, then RSA is breakable.

2. There is no proof that factorization is needed to break RSA.

3. If a method of breaking RSA would provide an effective way to get a trapdoor information, then factorization could be done effectively.

Theorem Any algorithm to compute $\phi(n)$ can be used to factor integers with the same complexity.

Theorem Any algorithm for computing d can be converted into a break randomized algorithm for factoring integers with the same complexity.

4. There are setups in which RSA can be broken without factoring modulus n.

Example An agency chooses p, q and computes a modulus $n = pq$ that is publicized and common to all users U_1, U_2 and also encryption exponents e_1, e_2,... are publicized. Each user U_i gets his decryption exponent d_i.

In such a setting any user is able to find in deterministic quadratic time another user's decryption exponent.

Encryption and Decryption: Fast Exponentiation

Problem in practice:

- Unlike symmetric algorithms such as AES, DES or stream ciphers, public-key algorithms are based on arithmetic with very long numbers.
- Without careful attention to how these computations are carried out, it's possible to create schemes that are too slow for practical use.
- If we look at RSA encryption and decryption, we see that both are based on **modular exponentiation.**

$$y = e_{k_{pub}}(x) \equiv x^e \bmod n \quad \text{(encryption)}$$
$$x = d_{k_{pr}}(y) \equiv y^d \bmod n \quad \text{(decryption)}$$

These modular exponentiation operations involve raising numbers to large exponents and then taking the result modulo 'n'. The efficiency and speed of these operations are critical to the practical use of RSA, as large numbers can lead to significant computational overhead.

Encryption and Decryption: Fast Exponentiation

A straightforward way of exponentiation looks like this:

$$x \xrightarrow{SQ} x^2 \xrightarrow{MUL} x^3 \xrightarrow{MUL} x^4 \xrightarrow{MUL} x^5 \ldots$$

where SQ denotes squaring and MUL multiplication.

Unfortunately, the exponents ‘e’ and ‘d’ are in general very large numbers. The exponents are typically chosen in the range of 1024–3072 bit or even larger. Straightforward exponentiation as shown above would thus require around 2^{1024} or more multiplications. Thus, we require a faster method for exponentiation. One such method is the square-and-multiply algorithm.

Encryption and Decryption: Fast Exponentiation

Example 7.2. Let's look at how many multiplications are required to compute the simple exponentiation x^8. With the straightforward method:

$$x \xrightarrow{SQ} x^2 \xrightarrow{MUL} x^3 \xrightarrow{MUL} x^4 \xrightarrow{MUL} x^5 \xrightarrow{MUL} x^6 \xrightarrow{MUL} x^7 \xrightarrow{MUL} x^8$$

we need seven multiplications and squarings. Alternatively, we can do something faster:

$$x \xrightarrow{SQ} x^2 \xrightarrow{SQ} x^4 \xrightarrow{SQ} x^8$$

which requires only three squarings that are roughly as complex as a multiplication.

- This fast method works fine but is restricted to exponents that are powers of 2, i.e., values e and d of the form 2^i.

Encryption and Decryption: Fast Exponentiation

Example 7.3. This time we have the more general exponent 26, i.e., we want to compute x^{26}. Again, the naïve method would require 25 multiplications. A faster way is as follows:

$$x \xrightarrow{SQ} x^2 \xrightarrow{MUL} x^3 \xrightarrow{SQ} x^6 \xrightarrow{SQ} x^{12} \xrightarrow{MUL} x^{13} \xrightarrow{SQ} x^{26}.$$

This approach takes a total of six operations, two multiplications and four squarings.

- In the example above we computed the sequence SQ, MUL, SQ, SQ, MUL, SQ. However, we do not know the sequence in which the squaring and multiplications must be performed for other exponents.
- One solution is the square-and-multiply algorithm. It provides a systematic way for finding the sequence in which we must perform squaring and multiplications by x for computing x^H.

Encryption and Decryption: Fast Exponentiation

Square-and-multiply Algorithm:

The algorithm is based on scanning the bit of the exponent from the left (the most significant bit) to the right (the least significant bit). In every iteration, i.e., for every exponent bit, the current result is squared. If and only if the currently scanned exponent bit has the value 1, a multiplication of the current result by x is executed following the squaring.

Example 7.4. We again consider the exponentiation x^{26}. For the square-and-multiply algorithm, the binary representation of the exponent is crucial:

$$x^{26} = x^{11010_2} = x^{(h_4 h_3 h_2 h_1 h_0)_2}.$$

Encryption and Decryption: Fast Exponentiation

$$x^{26} = x^{11010_2} = x^{(h_4 h_3 h_2 h_1 h_0)_2}.$$

The algorithm scans the exponent bits, starting on the left with h_4 and ending with the rightmost bit h_0.

Step		
#0	$x = x^{\mathbf{1}_2}$	inital setting, bit processed: $h_4 = 1$
#1*a*	$(x^1)^2 = x^2 = x^{\mathbf{10}_2}$	SQ, bit processed: h_3
#1*b*	$x^2 \cdot x = x^3 = x^{10_2} x^{1_2} = x^{\mathbf{11}_2}$	MUL, since $h_3 = 1$
#2*a*	$(x^3)^2 = x^6 = (x^{11_2})^2 = x^{\mathbf{110}_2}$	SQ, bit processed: h_2
#2*b*		no MUL, since $h_2 = 0$
#3*a*	$(x^6)^2 = x^{12} = (x^{110_2})^2 = x^{\mathbf{1100}_2}$	SQ, bit processed: h_1
#3*b*	$x^{12} \cdot x = x^{13} = x^{1100_2} x^{1_2} = x^{\mathbf{1101}_2}$	MUL, since $h_1 = 1$
#4*a*	$(x^{13})^2 = x^{26} = (x^{1101_2})^2 = x^{\mathbf{11010}_2}$	SQ, bit processed: h_0
#4*b*		no MUL, since $h_0 = 0$

Encryption and Decryption: Fast Exponentiation

Pseudo code:

Square-and-Multiply for Modular Exponentiation
Input:
base element x
exponent $H = \sum_{i=0}^{t} h_i 2^i$ with $h_i \in 0, 1$ and $h_t = 1$
and modulus n
Output: $x^H \bmod n$
Initialization: $r = x$
Algorithm:

```
1    FOR i = t − 1 DOWNTO 0
1.1      r = r^2 mod n
         IF h_i = 1
1.2          r = r · x mod n
2    RETURN (r)
```

Encryption and Decryption: Fast Exponentiation

Complexity of the square-and-multiply algorithm for an exponent H with a bit length of t +1:

$$\log_2 H = t + 1$$

The number of squaring is independent of the actual value of H, but the number of multiplications is equal to the Hamming weight, i.e., the number of ones in its binary representation.

Average number of multiplication, denoted by MUL:

$$\#SQ = t$$
$$\#\overline{MUL} = 0.5t$$

Encryption and Decryption: Fast Exponentiation

Question: How many operations are required on average for an exponentiation with a 1024-bit exponent?

Ans:

Straightforward exponentiation takes $2^{1024} \approx 10300$ multiplications. That is completely impossible, no matter what computer resources we might have at hand. However, the square-and-multiply algorithm requires only

$$\mathbf{1.5 \cdot 1024 = 1536}$$

squaring and multiplications on average. This is an impressive example for the difference of an algorithm with linear complexity (straightforward exponentiation) and logarithmic complexity (square-and-multiply algorithm).

Chinese Remainder Theorem (Recap)

States that if $M = m_1, m_2, \ldots, m_r$ is a set of pairwise relatively prime non-zero integers such that the greatest common divisor of each pair is 1 and $a_1, a_2, \ldots, a_r \in \mathbb{Z}$, then

The system of congruences

$$x \equiv a_1 \pmod{m_1},$$
$$x \equiv a_2 \pmod{m_2},$$
$$\ldots,$$
$$x \equiv a_r \pmod{m_r}$$

has a unique solution in modulo M for all $1 \leq i \leq r$

RSA Decryption and CRT

- RSA uses n = pq, where p and q are distinct secret primes. Therefore gcd(p, q) = 1 and CRT applies.
- Decryption computes $P = C^d \bmod n$. CRT evaluates this private-key operation modulo p and modulo q, then combines the results.
- The same optimization applies to RSA signature generation, which also uses the private exponent.

RSA and the Existing CRT Notation

CRT symbol	RSA value
m_1, m_2	p, q
$M = m_1m_2$	pq = n
$M_1 = M/m_1$	q
$M_2 = M/m_2$	p
a_1, a_2	Plaintext residues modulo p and q

$$\mathbf{P = (a_1M_1M_1^{-1} + a_2M_2M_2^{-1}) \bmod M}$$

RSA Decryption Using CRT

Precompute the reduced exponents:

$$d_1 = d \bmod (p - 1) \qquad d_2 = d \bmod (q - 1)$$

For each ciphertext C, compute the two residues:

$$a_1 = C^{d1} \bmod p \qquad a_2 = C^{d2} \bmod q$$

Use $M = pq$, $M_1 = q$ and $M_2 = p$. Compute M_1^{-1} modulo p and M_2^{-1} modulo q once for the key.

$$P = (a_1 M_1 M_1^{-1} + a_2 M_2 M_2^{-1}) \bmod M$$

RSA Example: Key and Ciphertext

Choose p = 11, q = 13 and public exponent e = 7.

$$n = pq = 143$$
$$\phi(n) = (11 - 1)(13 - 1) = 120$$
$$d = 103 \text{ [since } 7 \times 103 = 721 \equiv 1 \pmod{120}]$$

Let P = 42:

$$C = 42^7 \bmod 143 = 81$$

Example: Decryption Without CRT

$P = 81^{103} \bmod 143$, where $103 = 64 + 32 + 4 + 2 + 1$.

Repeated squaring modulo 143:

Exponent	**1**	**2**	**4**	**8**	**16**	**32**	**64**
$81^e \bmod 143$	81	126	3	9	81	126	3

$$\mathbf{P = (3 \times 126 \times 3 \times 126 \times 81) \bmod 143}$$

Reduce the running products: 3, 92, 133, 27, 42.

$$\mathbf{P = 42}$$

Example: CRT Residues

$$d_1 = 103 \bmod 10 = 3 \qquad d_2 = 103 \bmod 12 = 7$$

$$a_1 = 81^3 \bmod 11 = 4^3 \bmod 11 = 9$$

$$a_2 = 81^7 \bmod 13 = 3^7 \bmod 13 = 3$$

Now solve the same type of CRT system as before:

$$P \equiv 9 \pmod{11} \qquad P \equiv 3 \pmod{13}$$

Example: CRT Reconstruction

$$M = m_1 m_2 = 11 \times 13 = 143$$

i	a_i	m_i	$M_i = M/m_i$	$M_i^{-1} \bmod m_i$
1	9	11	13	6
2	3	13	11	6

$$P = (a_1 M_1 M_1^{-1} + a_2 M_2 M_2^{-1}) \bmod M$$

$$= (9 \times 13 \times 6 + 3 \times 11 \times 6) \bmod 143$$

$$= 900 \bmod 143 = 42$$

Comparison of the Two Methods

Property	Without CRT	With CRT
Exponentiation	$81^{103} \bmod 143$	$81^{3} \bmod 11$ and $81^{7} \bmod 13$
Modulus size	143: 8 bits	11 and 13: 4 bits each
Exponent size	103: 7 bits	3: 2 bits and 7: 3 bits
Recombination	Not required	900 mod 143
Recovered result	42	42

Advantage of using CRT is smaller operands.

Optimization for Large RSA Operands

- Let n be an L-bit modulus with primes of similar size. Then p and q each have approximately L/2 bits.

RSA modulus n	Direct modulus size	CRT modulus size
2048 bits	2048 bits	About 1024 bits per branch
4096 bits	4096 bits	About 2048 bits per branch

- The reduced exponents d_1 and d_2 also fit within about L/2 bits. The two branches can run independently.
- CRT reduces arithmetic operand sizes. The ciphertext stays the same size.

RSA-CRT Computational Speedup

Assume ordinary multiplication costs $O(L^2)$ and modular exponentiation uses $O(L)$ multiplications.

$$\textbf{Direct RSA: } T(L) \approx aL^3$$
$$\textbf{RSA-CRT: } 2T(L/2) \approx 2a(L/2)^3 = aL^3/4$$

$$\textbf{Estimated speedup} \approx 4\times$$

This estimate assumes similar prime sizes. Actual speed depends on the implementation. CRT also accelerates private exponentiation for RSA signatures.

RSA Security

- possible approaches to attacking RSA are:
- brute force key search - infeasible given size of numbers
- mathematical attacks - based on difficulty of computing ø(n), by factoring modulus n
- timing attacks - on running of decryption
- chosen ciphertext attacks - given properties of RSA

Factoring Problem

- mathematical approach takes 3 forms:

factor n=p.q, hence compute ø(n) and then d

determine ø(n) directly and compute d

find d directly

- currently believe all equivalent to factoring

have seen slow improvements over the years

as of May-05 best is 200 decimal digits (663) bit with LS

biggest improvement comes from improved algorithm cf QS to GHFS to LS

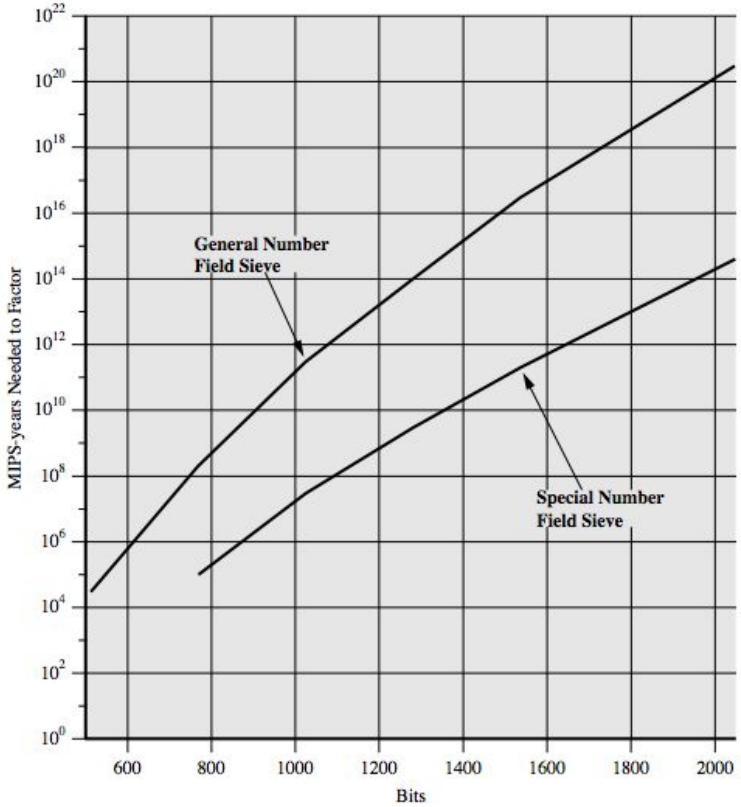
currently assume 1024-2048 bit RSA is secure

- ensure p, q of similar size and matching other constraints

Progress in Factoring

Number of Decimal Digits	Approximate Number of Bits	Date Achieved	MIPS-years	Algorithm
100	332	April 1991	7	quadratic sieve
110	365	April 1992	75	quadratic sieve
120	398	June 1993	830	quadratic sieve
129	428	April 1994	5000	quadratic sieve
130	431	April 1996	1000	generalized number field sieve
140	465	February 1999	2000	generalized number field sieve
155	512	August 1999	8000	generalized number field sieve
160	530	April 2003	—	Lattice sieve
174	576	December 2003	—	Lattice sieve
200	663	May 2005	—	Lattice sieve

Progress in Factoring



Timing Attacks

- developed by Paul Kocher in mid-1990's

exploit timing variations in operations

eg. multiplying by small vs large number

or IF's varying which instructions executed

- infer operand size based on time taken
- RSA exploits time taken in exponentiation
- countermeasures

use constant exponentiation time

add random delays

blind values used in calculations

Chosen Ciphertext Attacks

- RSA is vulnerable to a Chosen Ciphertext Attack (CCA)
- attackers chooses ciphertexts & gets decrypted plaintext back
- choose ciphertext to exploit properties of RSA to provide info to help cryptanalysis
- can counter with random pad of plaintext
- or use Optimal Asymmetric Encryption Padding (OAEP)

How Is RSA-OAEP Different From RSA?

- Bob's public key is (n, e). His private key is (n, d).
- RSA uses integers P and C, where $0 \leq P, C < n$.

$$C = P^e \bmod n$$

$$P = C^d \bmod n$$

- With OAEP, P is the encoded message supplied to RSA. The original message first undergoes padding and masking.

Short Message Attack

- If Eve knows the set of possible plaintexts, she can encrypt each candidate using Bob's public key.
- She compares each result with the intercepted ciphertext. A match identifies the plaintext.
- Example: for a four-digit number, Eve tries 0000 through 9999, a set of only 10,000 possibilities.
- Direct RSA encryption is deterministic. The same plaintext and public key always give the same ciphertext.

Need for OAEP

- Randomized padding before RSA encryption can be used to resist short message attacks.
- Simply appending extra data may make guessing harder, but does not by itself guarantee secure encryption.
- Optimal Asymmetric Encryption Padding (OAEP) combines a padded message with a fresh random value through public functions and XOR operations.

Message encoding precedes RSA exponentiation.

OAEP Notation

M : padded message of m bits

r : one-time random value of k bits, with k < m

P_1 : masked message of m bits

P_2 : recovery information of k bits

P : RSA plaintext, $P_1 \mathbin{//} P_2$

C : RSA ciphertext

$\oplus$ denotes bitwise XOR. // denotes concatenation.

Public Functions G and H

- G expands the k-bit random value into an m-bit mask => a public one-way function.

$$G : \{0,1\}^k \rightarrow \{0,1\}^m$$

- H maps an m-bit input to a k-bit output => a cryptographic hash function.

$$H : \{0,1\}^m \rightarrow \{0,1\}^k$$

Both functions are public. Their inputs and outputs must have the stated lengths.

OAEP Encryption: Initial Steps

Step 1: Pad the original message

☞ Alice pads a message shorter than m bits to obtain the m-bit padded message M.

Step 2: Choose a random value

☞ Alice generates a fresh k-bit random number r for this encryption.

☞ r is used only once and then discarded.

OAEP Encryption: Masked Message

- Alice computes G(r), an m-bit mask derived from the random value r.

- She XORs this mask with the padded message M. Both operands have m bits.

$$P_1 = M \oplus G(r)$$

P_1 is the masked message, the first part of P.

OAEP Encryption: Recovery Information

- Alice computes $H(P_1)$, a k-bit output from the masked message.
- She XORs $H(P_1)$ with the k-bit random value r.

$$P_2 = H(P_1) \oplus r$$

- P_2 allows Bob to recover r after RSA decryption and reconstruct the mask G(r).

OAEP Encryption: RSA Operation

☞ Alice concatenates the two parts to form the encoded RSA plaintext.

$$P = P_1 \mathbin{//} P_2$$

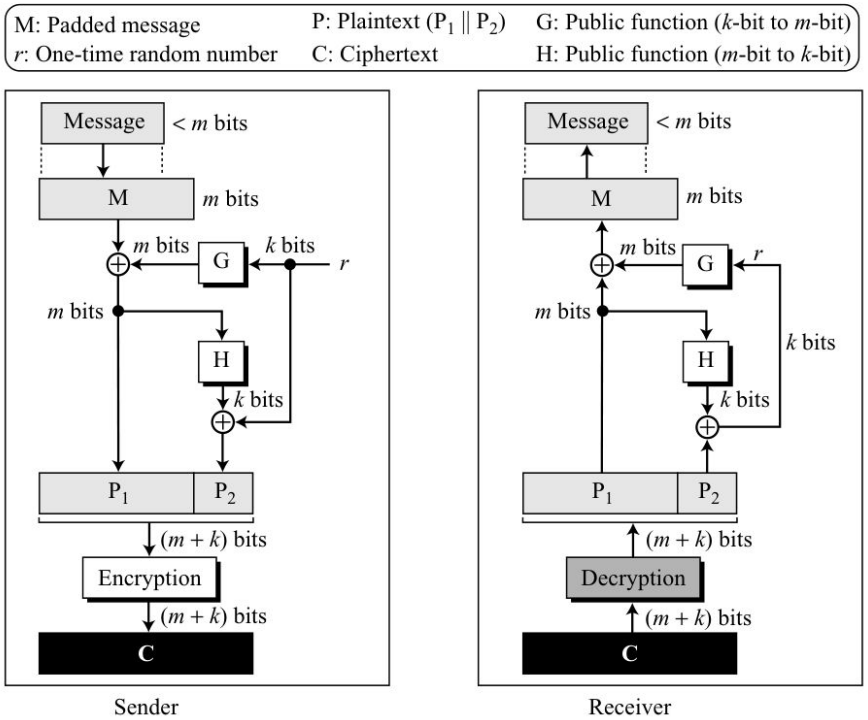
P_1 has m bits. P_2 has k bits. P has m + k bits.

☞ The encoding parameters must ensure that the integer represented by P is less than n.

$$C = P^e \bmod n$$

OAEP Diagram

Figure 10.9 *Optimal asymmetric encryption padding (OAEP)*



RSA-OAEP Encryption Algorithm

Input: original message and Bob's public key (n, e).

1. Pad the message to obtain M of m bits.
2. Generate a fresh random value r of k bits.
3. Compute $P_1 = M \oplus G(r)$.
4. Compute $P_2 = H(P_1) \oplus r$.
5. Form $P = P_1 \mathbin{//} P_2$.
6. Compute $C = P^e \bmod n$ and send C to Bob.

P must represent an integer in $0 \leq P < n$.

OAEP Decryption: RSA Operation

- Bob receives C and applies the RSA private exponent d.

$$P = C^d \bmod n$$

- He recovers the encoded bit string with its agreed fixed length, preserving any leading zeros.

- He splits P into P_1 of m bits and P_2 of k bits.

OAEP Decryption: Recovering r

During encryption: $P_2 = H(P_1) \oplus r$

☞ Bob knows P_1, so he can compute $H(P_1)$.

$$H(P_1) \oplus P_2$$
$$= H(P_1) \oplus [H(P_1) \oplus r]$$
$$= r$$

$$r = H(P_1) \oplus P_2$$

OAEP Decryption: Recovering M

During encryption: $P_1 = M \oplus G(r)$

☞ Bob uses the recovered r to compute the mask G(r).

$$P_1 \oplus G(r)$$
$$= [M \oplus G(r)] \oplus G(r)$$
$$= M$$

☞ He removes the padding from M to recover the original message.

RSA-OAEP Decryption Algorithm

Input: ciphertext C and Bob's private key (n, d).

1. Compute $P = C^d \bmod n$.
2. Recover the fixed-length encoding $P_1 \mathbin{//} P_2$.
3. Compute $r = H(P_1) \oplus P_2$.
4. Compute $M = P_1 \oplus G(r)$.
5. Remove the padding to recover the message.

XOR reverses itself: $(A \oplus B) \oplus B = A$

Textbook RSA and RSA with OAEP

Property	Direct RSA	RSA with OAEP
Input to RSA	Original plaintext	$P_1 \mathbin{/\!/} P_2$
Fresh randomness	None	k-bit value r
Message masking	None	$M \oplus G(r)$
Repeated message	Same ciphertext	Fresh r changes encoding
Guessing	Encrypt and compare	Includes random encoding

A fresh random value is essential to OAEP's purpose.

OAEP Summary

Encryption	Decryption
$P_1 = M \oplus G(r)$	$P = C^d \bmod n$
$P_2 = H(P_1) \oplus r$	$r = H(P_1) \oplus P_2$
$P = P_1 \ // \ P_2$	$M = P_1 \oplus G(r)$
$C = P^e \bmod n$	Remove the padding.

☞ OAEP masks the padded message using a one-time random value before RSA encryption.

Thank you!

Follow us



isfcr.pesu



www.isfcr.pes.edu



ISFCR

PESU ISFCR

PESU Center for
Information Security, Forensics and Cyber Resilience

PES UNIVERSITY



PESU Center for
Information Security,
Forensics and
Cyber Resilience



Welcome to

PES University

Ring Road Campus, Bengaluru



PESU Center for
Information Security,
Forensics and
Cyber Resilience



Applied Cryptography

UE24CS342AA4

Dr. Shruti Jadon

RSA Beyond Textbook RSA

Square-and-Multiply → CRT → RSA-CRT → Textbook RSA Weaknesses → OAEP

A Note on Security



Emergency Exit



Assembly Point



Washroom



No Chatting



Phones on silent



No Sleeping

A Note on Security

- In this course, you will be exposed to information about security problems and vulnerabilities with computing systems and networks.
- To be clear, you are not to use this or any other similar information to test the security of, break into, compromise, or otherwise attack, any system or network without the express consent of the owner.
- In particular, you will comply with all my instructions when doing the labs.
  - My instructions are in consonance with applicable laws of India and PES University policies.
  - If in any doubt, please consult your professor!
- Any violation is at **YOUR RISK!**
  And may result in severe consequences.

Learning goals

By the end, students should be able to connect efficiency and security improvements to RSA.

Reconstruct RSA key generation, encryption, and decryption.

Use Square-and-Multiply to evaluate modular exponentiation efficiently.

State and apply the Chinese Remainder Theorem (CRT).

Derive and execute RSA-CRT, including dP, dQ and CRT recombination.

Explain why CRT is an optimization even though it creates two exponentiations.

Compare the computational cost of direct RSA and RSA-CRT under a simple multiplication model.

Identify why textbook RSA is insecure: determinism, low-exponent/direct-message attacks, and malleability.

Explain why randomized padding is needed and how RSA-OAEP provides it.

Trace the OAEP encoding/decoding flow at the conceptual and formula level.

The story of the lecture

Each new idea solves a specific problem created by the previous one.

RSA

Public-key encryption framework.
Question: how do we encrypt/decrypt?

Square-and-Multiply

Makes modular exponentiation practical.
Question: can RSA be even faster?

CRT / RSA-CRT

Splits one large computation into two smaller ones.
Question: is textbook RSA secure?

OAEP

Randomized encoding before RSA encryption.
Question: how do we avoid deterministic textbook RSA?

Core distinction: CRT is primarily about PERFORMANCE; OAEP is about SECURITY.

1. RSA: the starting point

Choose p, q → n = pq → ϕ(n) = (p−1)(q−1) → choose e → find d

Public key: (e, n)

Private key: (d, n)

Encryption: C = M^e mod n

Decryption: M = C^d mod n

Correctness follows from modular arithmetic and Euler/Carmichael structure when RSA parameters are chosen correctly.

RSA worked example #1

Use small numbers so every step can be checked by hand.

Choose p = 5, q = 11 → n = 55

$\phi(n) = 4 \times 10 = 40$

Choose e = 3; find d such that $3d \equiv 1 \pmod{40}$ → d = 27

Public key = (3,55); private key = (27,55)

Let M = 7. Encrypt: C = 7^3 mod 55 = 343 mod 55 = 13

C = 13

2. Square-and-Multiply: why we need it

Directly computing C^d is impossible for realistic RSA exponents.

Binary expansion of the exponent tells us which powers are needed.

Repeated squaring computes C, C^2, C^4, C^8, C^{16}, … using one modular squaring per bit after the first.

For exponent $d = 27 = 16 + 8 + 2 + 1$:

- compute C^2, C^4, C^8, C^{16}; then multiply the selected powers $C^{16} \cdot C^8 \cdot C^2 \cdot C$.

Approximate operation count: O(log d) modular multiplications instead of O(d) repeated multiplications.

Square-and-Multiply on our RSA example

Decrypt C = 13 with d = 27 modulo 55.

$13^2 \bmod 55 = 4$

$13^4 \bmod 55 = 4^2 \bmod 55 = 16$

$13^8 \bmod 55 = 16^2 \bmod 55 = 36$

$13^{16} \bmod 55 = 36^2 \bmod 55 = 31$

$27 = 16 + 8 + 2 + 1$

Therefore $13^{27} \equiv 31 \times 36 \times 4 \times 13 \pmod{55} = 7$

$M = 13^{27} \bmod 55 = 7$

3. Chinese Remainder Theorem (CRT)

Formal statement and the intuition RSA will exploit.

Formal statement

If $\gcd(m_1,m_2)=1$, then the system $x \equiv a \pmod{m_1}$, $x \equiv b \pmod{m_2}$ has a unique solution modulo m_1m_2.

RSA-relevant form

Because $n = pq$ and $\gcd(p,q)=1$, a value modulo n can be represented by its residues modulo p and q — and reconstructed from them.

$$x \bmod (pq) \rightleftarrows \{ x \bmod p , x \bmod q \}$$

Chinese Remainder Theorem

What is CRT? What is it used for in cryptography? Solved examples

Teaching Assistant - Aditi S Kulkarni

Background

- Cryptography often works with very large numbers and modular arithmetic
- Many cryptographic algorithms (like RSA) rely on modular exponentiation
- **CRT helps make these computations faster and easier**
- It’s a way to combine several modular equations into one consistent result

What is the Chinese Remainder Theorem?

The **Chinese Remainder Theorem (CRT)** is a concept from number theory

It says that if you know the remainder of a number when divided by several **coprime** moduli (numbers that share no common factors),
then there is **one and only one** number that fits all those remainders

Mathematical Representation

States that if $M = m_1, m_2, \ldots, m_r$ is a set of pairwise relatively prime non-zero integers such that the greatest common divisor of each pair is 1 and $a_1, a_2, \ldots, a_r \in \mathbb{Z}$, then

The system of congruences

$$x \equiv a_1 \pmod{m_1},$$
$$x \equiv a_2 \pmod{m_2},$$
$$\ldots,$$
$$x \equiv a_r \pmod{m_r}$$

has a unique solution in modulo M for all $1 \leq i \leq r$

Applications in Cryptography

Many crypto systems (like **RSA**) use very large numbers:
$n = p \times q$ where p and q are large primes.

Instead of working with huge n, CRT lets us:

1. Do calculations separately mod p and mod q
2. Combine results at the end using CRT

Result: operations are **3–4× faster**, especially during RSA decryption.

Applications in Cryptography

- **RSA decryption optimization**
- **Secret sharing** schemes (reconstructing secrets)
- **Residue Number Systems** for fast arithmetic
- **Homomorphic encryption** arithmetic circuits

The Chinese Remainder Theorem

The Chinese Remainder Theorem (CRT) is used to solve a set of different congruent equations with one variable but different moduli which are relatively prime as shown below:

$X \equiv a_1 \pmod{m_1}$

$X \equiv a_2 \pmod{m_2}$

. . .

$X \equiv a_n \pmod{m_n}$

CRT states that the above equations have a unique solution of the moduli are relatively prime.

$X = (a_1 M_1 M_1^{-1} + a_2 M_2 M_2^{-1} + \ldots + a_n M_n M_n^{-1}) \bmod M$

CRT - Example 1

The Chinese Remainder Theorem

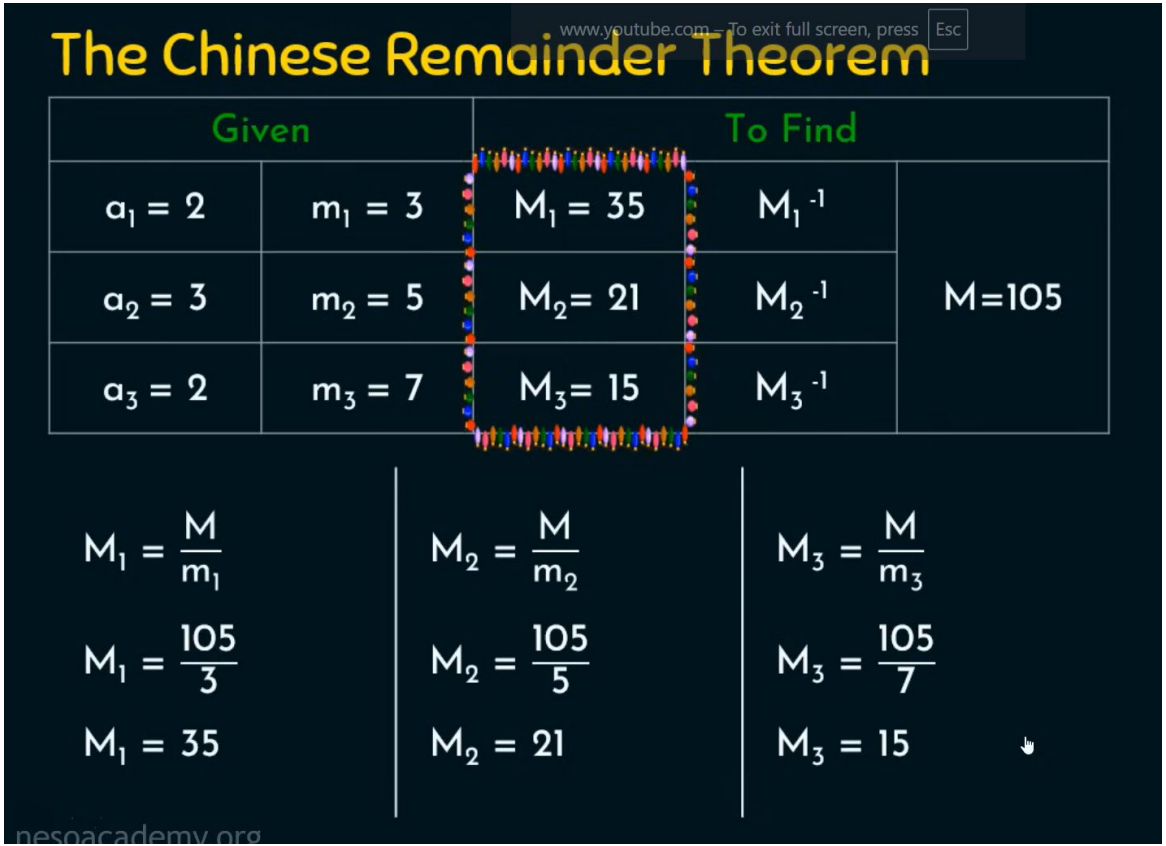
$X \equiv a_1 \pmod{m_1}$	$X \equiv 2 \pmod{3}$
$X \equiv a_2 \pmod{m_2}$	$X \equiv 3 \pmod{5}$
$X \equiv a_3 \pmod{m_3}$	$X \equiv 2 \pmod{7}$

Solution:

$$X = (a_1 M_1 M_1^{-1} + a_2 M_2 M_2^{-1} + a_3 M_3 M_3^{-1}) \bmod M$$

Given		To Find		
$a_1 = 2$	$m_1 = 3$	M_1	M_1^{-1}	M
$a_2 = 3$	$m_2 = 5$	M_2	M_2^{-1}	
$a_3 = 2$	$m_3 = 7$	M_3	M_3^{-1}	

CRT - Example 1



M = m1 * m2 * m3
= 3 * 5 * 7
= 105

CRT - Example 1

The Chinese Remainder Theorem

Given		To Find		
$a_1 = 2$	$m_1 = 3$	$M_1 = 35$	$M_1^{-1} = 2$	
$a_2 = 3$	$m_2 = 5$	$M_2 = 21$	$M_2^{-1} = 1$	$M = 105$
$a_3 = 2$	$m_3 = 7$	$M_3 = 15$	$M_3^{-1} = 1$	

$M_1 \times M_1^{-1} = 1 \bmod m_1$

$35 \times M_1^{-1} = 1 \bmod 3$

$35 \times 2 = 1 \bmod 3$

$M_1^{-1} = 2$

$M_2 \times M_2^{-1} = 1 \bmod m_2$

$21 \times M_2^{-1} = 1 \bmod 5$

$21 \times 1 = 1 \bmod 5$

$M_2^{-1} = 1$

$M_3 \times M_3^{-1} = 1 \bmod m_3$

$15 \times M_3^{-1} = 1 \bmod 7$

$15 \times 1 = 1 \bmod 7$

$M_3^{-1} = 1$

Inverse calculation - can use extended Euclidean algorithm

CRT - Example 1

The Chinese Remainder Theorem

Example 1: Solve the following equations using CRT

$X \equiv 2 \pmod 3$

$X \equiv 3 \pmod 5$

$X \equiv 2 \pmod 7$

$a_1 = 2$	$m_1 = 3$	$M_1 = 35$	$M_1^{-1} = 2$	$M=105$
$a_2 = 3$	$m_2 = 5$	$M_2 = 21$	$M_2^{-1} = 1$	
$a_3 = 2$	$m_3 = 7$	$M_3 = 15$	$M_3^{-1} = 1$	

Solution:

$$X = (a_1M_1 M_1^{-1} + a_2M_2M_2^{-1} + a_3M_3M_3^{-1}) \bmod M$$

$$= (2x35x2 + 3x21x1 + 2x15x1) \bmod 105$$

$$= 233 \bmod 105$$

$$X = 23$$

CRT - Example 2

The Chinese Remainder Theorem

Example 1: Solve the following equations using CRT:

$4X \equiv 5 \pmod 9$

$2X \equiv 6 \pmod{20}$

Rewrite the question as follows:

$4X \equiv 5 \pmod 9$	$2X \equiv 6 \pmod{20}$
Multiply by 4^{-1} on both sides	$2X \equiv 2x3 \pmod{20}$
$4^{-1} \times 4X \equiv 4^{-1} \times 5 \pmod 9$	$X \equiv 3 \pmod{20}$
$X \equiv 4^{-1} \pmod 9 \times 5 \pmod 9$	
$X \equiv 7 \times 5 \pmod 9$	
$X \equiv 35 \pmod 9$	
$X \equiv 8 \pmod 9$	

pesuacademy.com

CRT - Example 2

The Chinese Remainder Theorem

Example 1: Solve the following equations using CRT:

$X \equiv 8 \pmod 9$

$X \equiv 3 \pmod{20}$

$X \equiv a_1 \pmod{m_1}$	$X \equiv 8 \pmod 9$
$X \equiv a_2 \pmod{m_2}$	$X \equiv 3 \pmod{20}$

Solution:

$X = (a_1 M_1 M_1^{-1} + a_2 M_2 M_2^{-1}) \bmod M$

Given		To Find		
$a_1 = 8$	$m_1 = 9$	M_1	M_1^{-1}	M
$a_2 = 3$	$m_2 = 20$	M_2	M_2^{-1}	

CRT - Example 2

The Chinese Remainder Theorem

Given		To Find		
$a_1 = 8$	$m_1 = 9$	M_1	M_1^{-1}	M=180
$a_2 = 3$	$m_2 = 20$	M_2	M_2^{-1}	

$$M_1 = \frac{M}{m_1}$$

$$M_1 = \frac{180}{9}$$

$$M_1 = 20$$

$$M_2 = \frac{M}{m_2}$$

$$M_2 = \frac{180}{20}$$

$$M_2 = 9$$

M = m1 x m2
= 9 * 20
= 180

CRT - Example 2

The Chinese Remainder Theorem

Given		To Find		
$a_1 = 8$	$m_1 = 9$	$M_1 = 20$	M_1^{-1}	M=180
$a_2 = 3$	$m_2 = 20$	$M_2 = 9$	M_2^{-1}	

$M_1 \times M_1^{-1} = 1 \bmod m_1$

$20 \times M_1^{-1} = 1 \bmod 9$

$20 \times 5 = 1 \bmod 9$

$M_1^{-1} = 5$

$M_2 \times M_2^{-1} = 1 \bmod m_2$

$9 \times M_2^{-1} = 1 \bmod 20$

$9 \times 9 = 1 \bmod 20$

$M_2^{-1} = 9$

CRT - Example 2

The Chinese Remainder Theorem

Example 1: Solve the following equations using CRT:

$X \equiv 8 \pmod 9$

$X \equiv 3 \pmod{20}$

Given		To Find		
$a_1 = 8$	$m_1 = 9$	$M_1 = 20$	$M_1^{-1} = 5$	M=180
$a_2 = 3$	$m_2 = 20$	$M_2 = 9$	$M_2^{-1} = 9$	

Solution:

$X = (a_1 M_1 M_1^{-1} + a_2 M_2 M_2^{-1}) \bmod M$

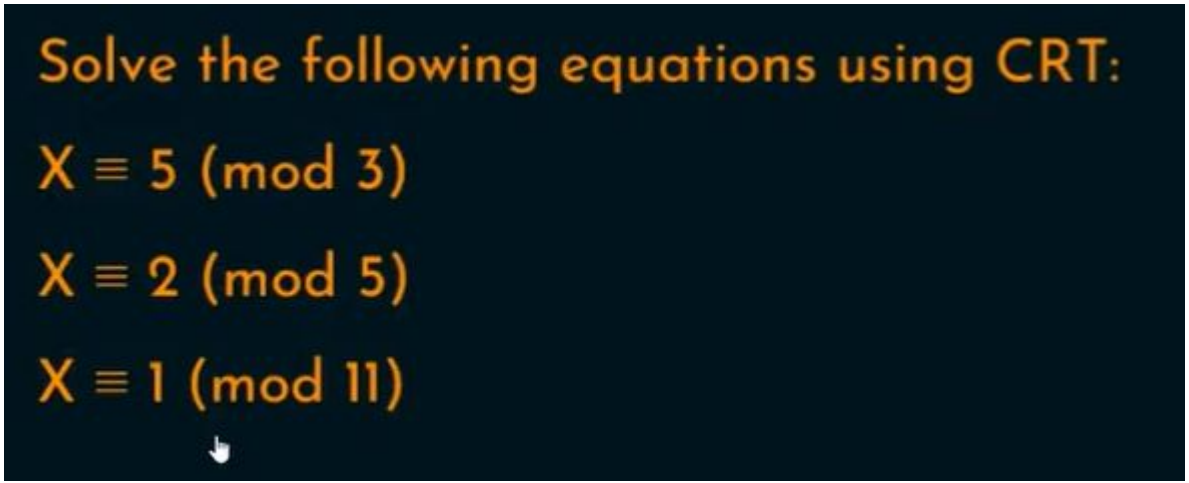
$= (8 \times 20 \times 5 + 3 \times 9 \times 9) \bmod 180$

$= (800 + 243) \bmod 180$

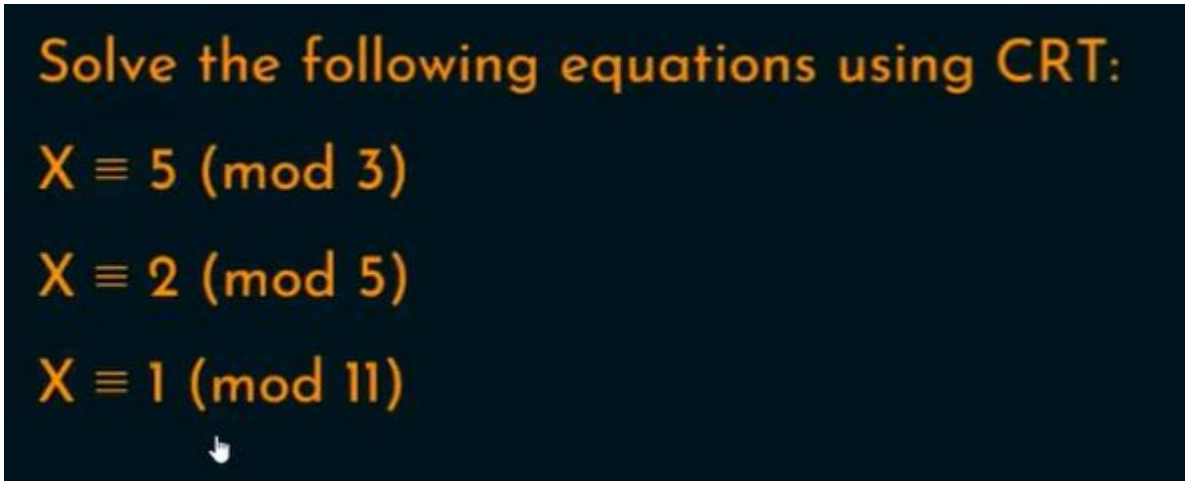
$= 1043 \bmod 180$

$X = 143$

CRT - Try it Yourself



CRT - Try it Yourself



Solution = 122

CRT worked example #1

Reconstruct a number from two smaller congruences.

Given $x \equiv 2 \pmod 5$ and $x \equiv 7 \pmod{11}$.

Numbers congruent to 2 mod 5 are 2, 7, 12, 17, …

7 mod 11 = 7, so x = 7 satisfies both.

Because 5 and 11 are coprime, the solution is unique modulo 55.

$$x \equiv 7 \pmod{55}$$

CRT recombination formula

One common two-modulus form.

$$x = a + m_1 \cdot [(b-a) \cdot m_1^{-1} \bmod m_2]$$

For $x \equiv a \pmod{m_1}$, $x \equiv b \pmod{m_2}$.

Need the modular inverse $m_1^{-1} \bmod m_2$.

In the RSA setting: $m_1 = p$, $m_2 = q$.

Equivalent implementation formulas may use $q^{-1} \bmod p$ and a recombination variable h; the mathematics is the same.

4. Put CRT inside RSA

Now the optimization becomes obvious: n is already factored as p×q.

$$M = C^d \bmod n$$

becomes

$$M_p = C^{d_p} \bmod p \quad \text{and} \quad M_q = C^{d_q} \bmod q$$

$$\text{CRT}(M_p, M_q) \rightarrow M \bmod n$$

where $d_p = d \bmod (p-1)$, $d_q = d \bmod (q-1)$

RSA-CRT worked example #1: setup

Same RSA system as before — now decrypt in two smaller domains.

p = 5, q = 11, n = 55, e = 3, d = 27

Ciphertext C = 13

Direct decryption: $13^{27} \bmod 55$

CRT parameters:

$d_p = d \bmod (p-1)$

$d_p = 27 \bmod 4 = 3;$

$d_q = d \bmod (q-1)$

$d_q = 27 \bmod 10 = 7$

RSA-CRT example #1: two branches

Modulo p = 5

$M_p = C^d \bmod p$
$= 13^3 \bmod 5$
$= 3^3 \bmod 5$
$= 27 \bmod 5$
$= 2$

Modulo q = 11

$M_q = C^d \bmod q$
$= 13^7 \bmod 11$
$= 2^7 \bmod 11$
$= 128 \bmod 11$
$= 7$

$$M \equiv 2 \pmod 5, \quad M \equiv 7 \pmod{11}$$

RSA-CRT example #1: recombine

We need $M \equiv 2 \pmod 5$ and $M \equiv 7 \pmod{11}$.

Among residues modulo 55, 7 satisfies both.

Therefore $M = 7 \bmod 55$.

Direct RSA and RSA-CRT produce exactly the same plaintext.

Direct RSA: $13^{27} \bmod 55 = 7$

RSA-CRT: CRT(2 mod 5, 7 mod 11) = 7

RSA-CRT worked example #2

A second example where students can verify both routes.

Choose p = 7, q = 13 → n = 91; $\phi(n)=72$.

Choose e = 5 → d = 29 because $5\times29 = 145 \equiv 1 \pmod{72}$.

Let M = 12. Encryption: $C = 12^5 \bmod 91 = 38$.

Direct decryption: $38^{29} \bmod 91 = 12$.

CRT route: ?????

RSA-CRT worked example #3

A slightly larger hand-checkable example.

Choose p = 11, q = 17 → n = 187; ϕ(n)=160.

Choose e = 7 → d = 23 because 7×23 = 161 ≡ 1 (mod 160).

Let M = 20. Encrypt: $C = 20^7 \bmod 187 = 98$.

Direct decryption: $98^{23} \bmod 187 = 20$.

CRT: $d_p = 23 \bmod 10 = 3$; d_q = 23 mod 16 = 7.

Compute $M_p = 98^3 \bmod 11 = 9$; $M_q = 98^7 \bmod 17 = 3$.

CRT reconstructs M = 20 mod 187.

5. Why is RSA-CRT an improvement?

The answer is not “fewer exponentiations.” It is “smaller arithmetic.”

Direct RSA

One exponentiation:
C^d mod n

n is the full RSA modulus.

RSA-CRT

Two exponentiations:
C^dP mod p
C^dQ mod q

p and q are about half the bit length of n.

Why this wins

Large-integer multiplication is expensive.
Halving operand size can reduce multiplication cost dramatically.

CRT trades ONE large problem for TWO smaller problems.

Complexity model: why halving the modulus matters

Use this as an intuition model, not as an exact benchmark.

schoolbook multiplication: $M(k) \approx O(k^2)$

Suppose n is 2048 bits; p and q are approximately 1024 bits.

One 2048-bit multiplication costs roughly $2048^2 = 4{,}194{,}304$ basic units.

One 1024-bit multiplication costs roughly $1024^2 = 1{,}048{,}576$ units.

Therefore one half-size multiplication is about 4× cheaper under this model.

But CRT performs TWO exponentiations — does that cancel the gain?

Direct: 1×2048^2

CRT: 2×1024^2

$$2048^2 / (2 \times 1024^2) = 2$$

Under the simple $O(k^2)$ model, the arithmetic work can be about 2× lower.

CRT + Square-and-Multiply: compare the actual exponentiation work

Square-and-Multiply reduces exponentiation from O(d) multiplications to O(log d) modular operations.

CRT then reduces the SIZE of each modular operation by splitting the 2048-bit modulus into two ~1024-bit moduli.

Thus the optimizations operate at two different levels:

Level 1 — algorithmic exponentiation: use the binary representation of d.

Level 2 — arithmetic size: use p and q instead of pq.

Teaching message: Square-and-Multiply and CRT are complementary, not competing techniques.

Comparison across different complexity scenarios

A useful classroom table.

Scenario	Modulus size	Exponent bits	Approx. S&M ops	Main benefit
Direct RSA	2048	2048	≈ 1.5×2048	Baseline
RSA-CRT	1024 + 1024	≈1024 each	≈ 2×(1.5×1024)	Smaller operands
Direct RSA, small d	2048	small	Fewer ops	Exponent effect
CRT + small d	1024 + 1024	small each	Fewer + smaller	Best of both

CRT: improvement, but not magic

The optimization has a tradeoff profile.

Benefit

Much faster RSA private-key operations; especially important for decryption/signing at large key sizes.

Extra work

Two exponentiations plus CRT recombination; implementation is more complex than direct RSA.

Security consideration

CRT implementations need fault-attack countermeasures; a faulty branch result can leak information in vulnerable implementations.

Bottom line: CRT is a practical optimization used in real RSA implementations.

6. We made RSA faster. Is textbook RSA secure?

Textbook RSA: $C = M^e \bmod n$

RSA's mathematical trapdoor does not automatically give a secure encryption scheme.

Textbook RSA is deterministic: the same plaintext and public key produce the same ciphertext.

It is also algebraically malleable and does not provide semantic security on its own.

Therefore RSA encryption needs an appropriate randomized encoding/padding scheme.

Textbook RSA weakness #1: deterministic encryption

Suppose an employee's status is one of a few known messages: M_1 = "YES", M_2 = "NO".

With textbook RSA, each message maps to a fixed ciphertext under the same public key.

An attacker can encrypt the candidate messages themselves and compare ciphertexts.

No private key is required to test the guess.

Same M + same public key → same C

Textbook RSA weakness #2: small-message / low-exponent attack

A classic example when e is small and padding is absent.

Suppose e = 3 and a small message M satisfies $M^3 < n$.

Then encryption gives $C = M^3 \bmod n = M^3$ because no reduction occurs.

An attacker can simply compute the integer cube root of C and recover M.

Example: n = 55, e = 3, M = 2 → $C = 2^3 \bmod 55 = 8$. The attacker sees 8 and immediately gets $\sqrt[3]{8} = 2$.

Small M + small e + no padding ⇒ C may literally reveal M^e

Textbook RSA weakness #3: multiplicative structure

$$\mathrm{Enc}(M_1) \cdot \mathrm{Enc}(M_2) \bmod n = \mathrm{Enc}(M_1 M_2 \bmod n)$$

Because $(M_1^e)(M_2^e) = (M_1 M_2)^e \bmod n$, textbook RSA is multiplicatively homomorphic.

An attacker can transform a ciphertext into the encryption of a related plaintext without knowing the private key.

Example idea: if $C = \mathrm{Enc}(M)$, then $C \cdot 2^e \bmod n = \mathrm{Enc}(2M \bmod n)$.

This does not mean every textbook-RSA system is immediately broken, but it shows why raw RSA is not a secure modern encryption scheme.

7. Why do we need OAEP?

Problem

Raw RSA directly exponentiates the message integer. It is deterministic and exposes algebraic structure.

Design goal

Before RSA, transform the message into a randomized, structured encoding so repeated encryptions differ.

Solution

RSA-OAEP = RSA encryption applied to an OAEP-encoded message.

M → OAEPEncode(M; random seed) → EM → RSA → C

OAEP at a high level

Focus first on what each component is doing.

1. Start with the message M and a random seed.
2. Hash/encode label information and combine it with M plus zero padding to form DB (data block).
3. Use MGF1 to derive a mask from the seed; XOR it with DB → maskedDB.
4. Use MGF1 again, now derived from maskedDB, to mask the seed → maskedSeed.
5. Concatenate maskedSeed || maskedDB to form the encoded message EM.
6. RSA encrypts EM, not the raw message M.

OAEP structure: the pieces

DB

lHash || PS || 0x01 || M
Contains the label hash, padding string, delimiter, and message.

seed

Fresh random seed for each encryption.
This is what makes encryption probabilistic.

MGF1

Mask Generation Function based on a hash function.
Expands a short input into a mask of the required length.

$$\text{maskedDB} = \text{DB} \oplus \text{MGF}(\text{seed})$$

$$\text{maskedSeed} = \text{seed} \oplus \text{MGF}(\text{maskedDB})$$

OAEP encoding flow

M + label → DB → mask with seed → maskedDB → mask seed → maskedSeed

Input

Message M
Optional label L

Randomness

Fresh seed

Masking

MGF1 + XOR

Encoded EM

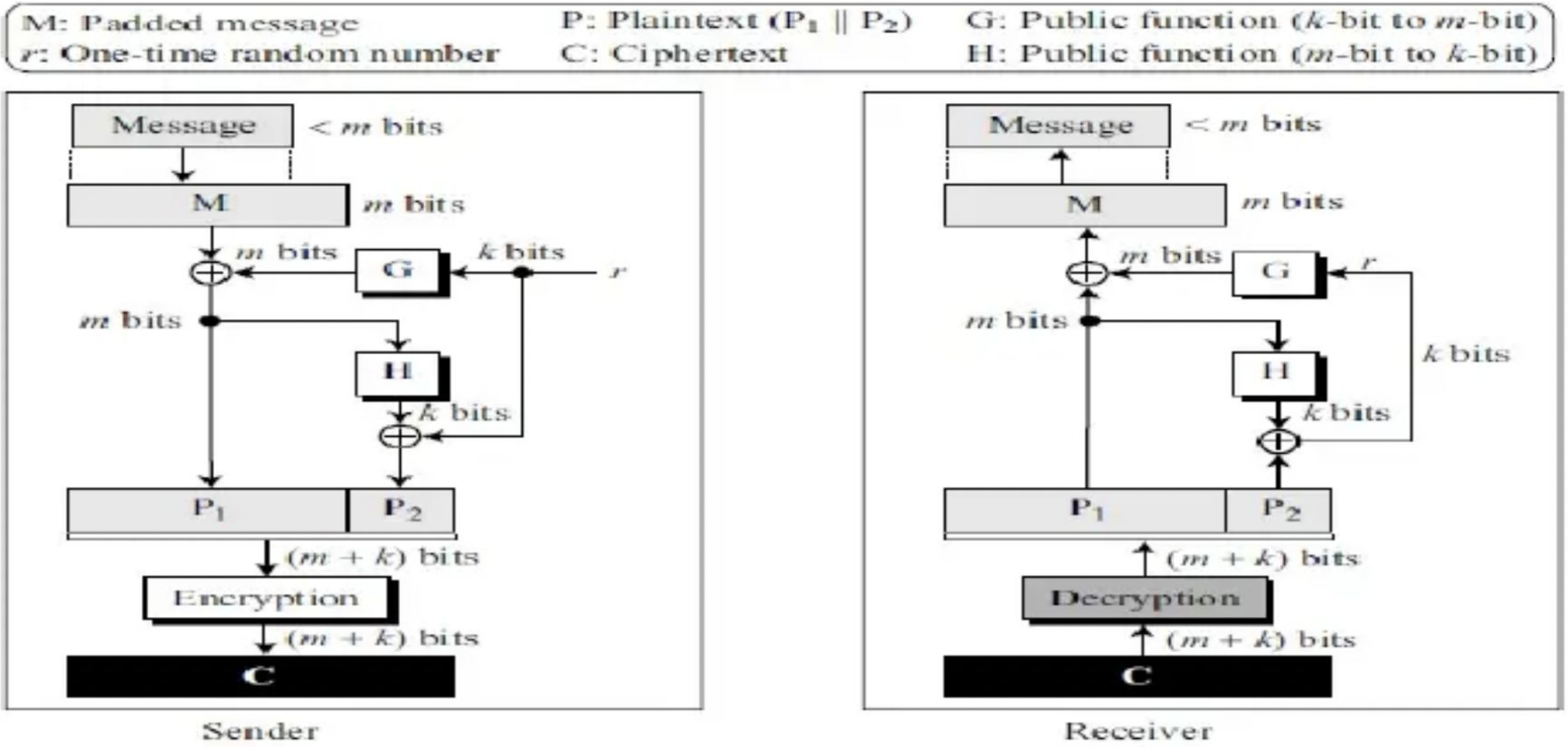
maskedSeed || maskedDB

RSA

C = EM^e mod n

OAEP's job ends before the RSA exponentiation begins.

OAEP encoding flow



How OAEP secures repeated encryption

Encryption #1

M = HELLO
seed = r_1
OAEP(M; r_1) = EM_1
RSA(EM_1) = C_1

Encryption #2

M = HELLO
seed = $r_2 \neq r_1$
OAEP(M; r_2) = EM_2
RSA(EM_2) = C_2

Same plaintext, same public key, but $C_1 \neq C_2$ (with overwhelming probability)

OAEP toy example: understand the mechanism, not the real parameters

Use tiny bit strings only to visualize XOR/masking; real OAEP uses a cryptographic hash and MGF1 with specified lengths.

Suppose DB = 10110010 and a random seed produces mask = 01100101.

maskedDB = DB XOR mask = 11010111.

Then a second MGF step derives a seed mask from maskedDB.

The important property: changing the random seed changes the mask, which changes the encoded message EM, which changes the RSA ciphertext.

XOR is reversible: DB = maskedDB ⊕ mask

OAEP decryption: reverse the encoding

1. RSA private-key operation recovers EM.
2. Split EM into maskedSeed and maskedDB.
3. Use MGF1(maskedDB) to recover the seed.
4. Use MGF1(seed) to recover DB.
5. Parse DB = lHash || PS || 0x01 || M and verify the expected structure.
6. If validation fails, reject the ciphertext rather than returning arbitrary plaintext.

C → RSA decrypt → EM → unmask → validate → M

Put everything together: modern RSA encryption

RSA mathematics

Public/private key pair
C = EM^e mod n
M = EM^d mod n

Square-and-Multiply

Efficiently computes modular exponentiation.
Used inside practical RSA implementations.

CRT

Optimizes private-key exponentiation by working mod p and mod q, then recombining.

Plaintext M → OAEP → EM → RSA public operation → Ciphertext C

On decryption: RSA private operation (often CRT-optimized) → OAEP decoding → M

Final comparison: what problem does each technique solve?

Square-and-Multiply

Problem: exponentiation is expensive.
Solution: reduce number of modular operations using binary exponentiation.

CRT / RSA-CRT

Problem: RSA private operation uses a large modulus.
Solution: two half-size computations + recombination.

OAEP

Problem: textbook RSA is deterministic and structurally unsafe.
Solution: randomized, structured encoding before RSA.

These are complementary layers, not alternatives.

In-class exercises / discussion prompts

Exercise 1 — RSA: p=5, q=11, e=3, M=7. Find C and decrypt directly.

Exercise 2 — RSA-CRT: same parameters. Compute dP, dQ, M□, M_q and recombine. Verify the same M.

Exercise 3 — RSA-CRT: p=7, q=13, e=5, M=12. Compare direct decryption and CRT.

Exercise 4 — Complexity: compare one 2048-bit multiplication with two 1024-bit multiplications under $O(k^2)$.

Exercise 5 — Security: give one reason textbook RSA is deterministic and one example of an attack enabled by missing padding.

Exercise 6 — OAEP: explain why encrypting the same message twice can produce different ciphertexts.

Common misconceptions to address

“CRT reduces one exponentiation to one.” → No. It creates two smaller exponentiations.

“CRT changes RSA’s plaintext.” → No. CRT reconstructs exactly the same result modulo n.

“CRT is a security padding scheme.” → No. CRT is primarily an efficiency optimization.

“Square-and-Multiply and CRT are competing methods.” → No. Square-and-Multiply can be used inside each CRT branch.

“OAEP makes RSA faster.” → No. OAEP is a security encoding and adds processing.

“Padding means adding random zeroes.” → No. OAEP is a structured randomized encoding using hashing, MGF1 and XOR.

“Textbook RSA is secure because RSA itself is mathematically hard.” → No. Secure encryption requires an appropriate encoding/padding construction.

Take-home

RSA = the trapdoor mathematics

Square-and-Multiply = efficient exponentiation

CRT = efficient RSA private-key computation

OAEP = randomized secure encoding for RSA encryption

RSA: From textbook mathematics to practical cryptography

Efficiency: Square-and-Multiply + CRT
Security: OAEP

“What problem are we solving, and exactly how does the next technique solve it?”

Thank you!

Follow us



isfcr.pesu



www.isfcr.pes.edu



ISFCR

PESU ISFCR

PESU Center for
Information Security, Forensics and Cyber Resilience

PES UNIVERSITY



PESU Center for
Information Security,
Forensics and
Cyber Resilience



Welcome to

PES University

Ring Road Campus, Bengaluru



PESU Center for
Information Security,
Forensics and
Cyber Resilience



Applied Cryptography

UE24CS342AA4

Dr. Shruti Jadon

LECTURE 21

A Note on Security



Emergency Exit



Assembly Point



Washroom



No Chatting



Phones on silent



No Sleeping

A Note on Security

- In this course, you will be exposed to information about security problems and vulnerabilities with computing systems and networks.
- To be clear, you are not to use this or any other similar information to test the security of, break into, compromise, or otherwise attack, any system or network without the express consent of the owner.
- In particular, you will comply with all my instructions when doing the labs.
  - My instructions are in consonance with applicable laws of India and PES University policies.
  - If in any doubt, please consult your professor!
- Any violation is at **YOUR RISK!**
  And may result in severe consequences.

Diffie-Hellman Key Exchange

DHKE

Public Key Cryptography

☞ Public key cryptography came about as a solution to two problems:

- ***Key Distribution***
  - With private key cryptosystems, a major problem is how keys are distributed to a set of message senders and recipients.
- ***Digital Signatures***
  - Could a method be found that would guarantee, to the satisfaction of all parties, that a digital message had been sent by a particular person?

History

- First public-key protocols named after Whitfield Diffie and Martin Hellman.
- In 1976, Whitfield Diffie and Martin Hellman, then at Stanford University, came up with a method that addressed both problems.
- Their method was radically different from all previous methods used in cryptography -- it is a *Public Key Cryptosytem*
  - It uses two different keys:
    - One key for encryption, and
    - A different key for decryption

History

- In 1976, Whitfield Diffie and Martin Hellman published a paper with the innocent-looking title “New Directions in Cryptography.”
- In that work they observed that there is often asymmetry in the world; in particular, there are certain actions that can be easily performed but not easily reversed.
- Padlocks can be locked without a key (i.e., easily), but then cannot be reopened.
- It is easy to shatter a glass vase but extremely difficult to put it back together again.
- Algorithmically (and more germane for our purposes), it is easy to multiply two large primes but difficult to recover those primes from their product.

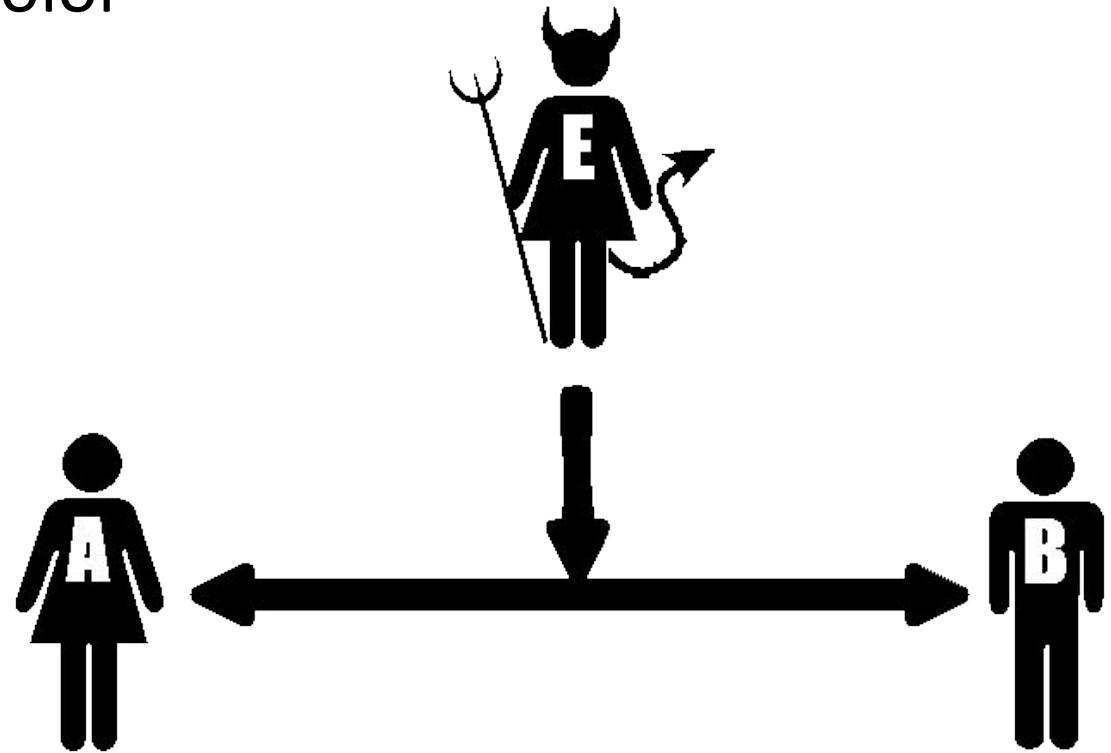
Diffie and Hellman realized that such phenomena could be used to derive interactive protocols for secure key exchange that allow two parties to share a secret key, via communication over a public channel, by having the parties perform operations that they can reverse but that an eavesdropper cannot.

Public Key Cryptosystems

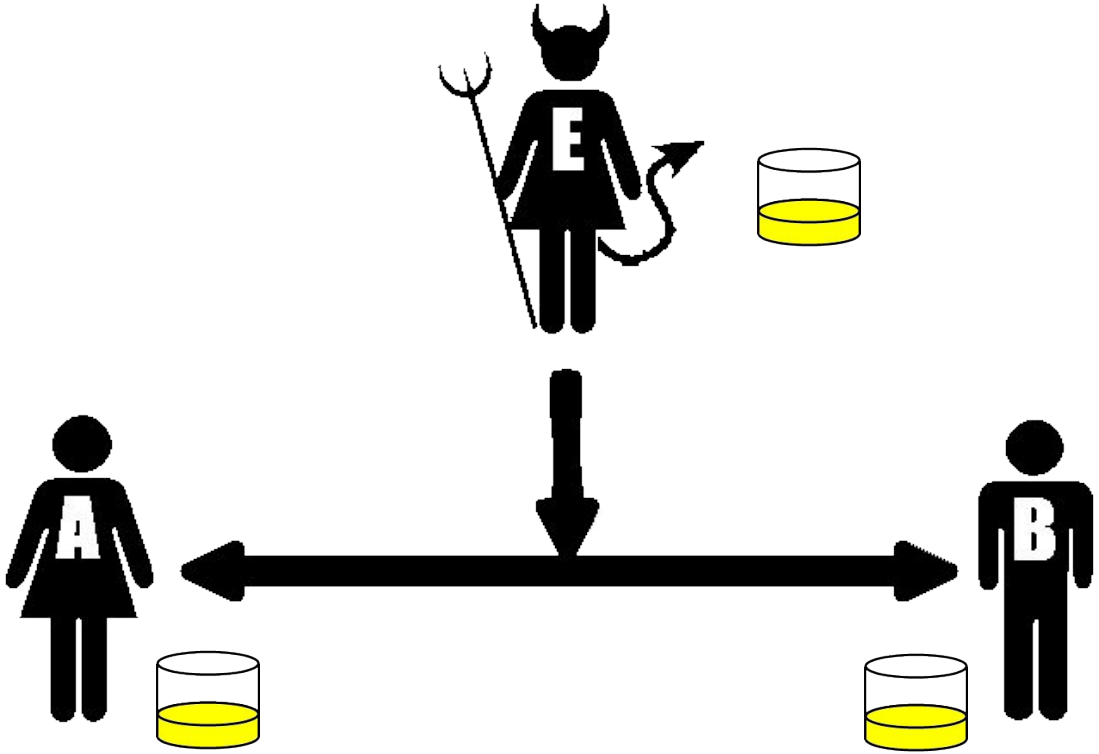
- Public Key cryptography relies on the following:
  - It is computationally infeasible to determine the decryption key, given only knowledge of the cryptographic algorithm and the encryption key (and vice versa)

- In some public key algorithms (including RSA):
  - Either of the two keys can be used for encryption, with the other being used for decryption.

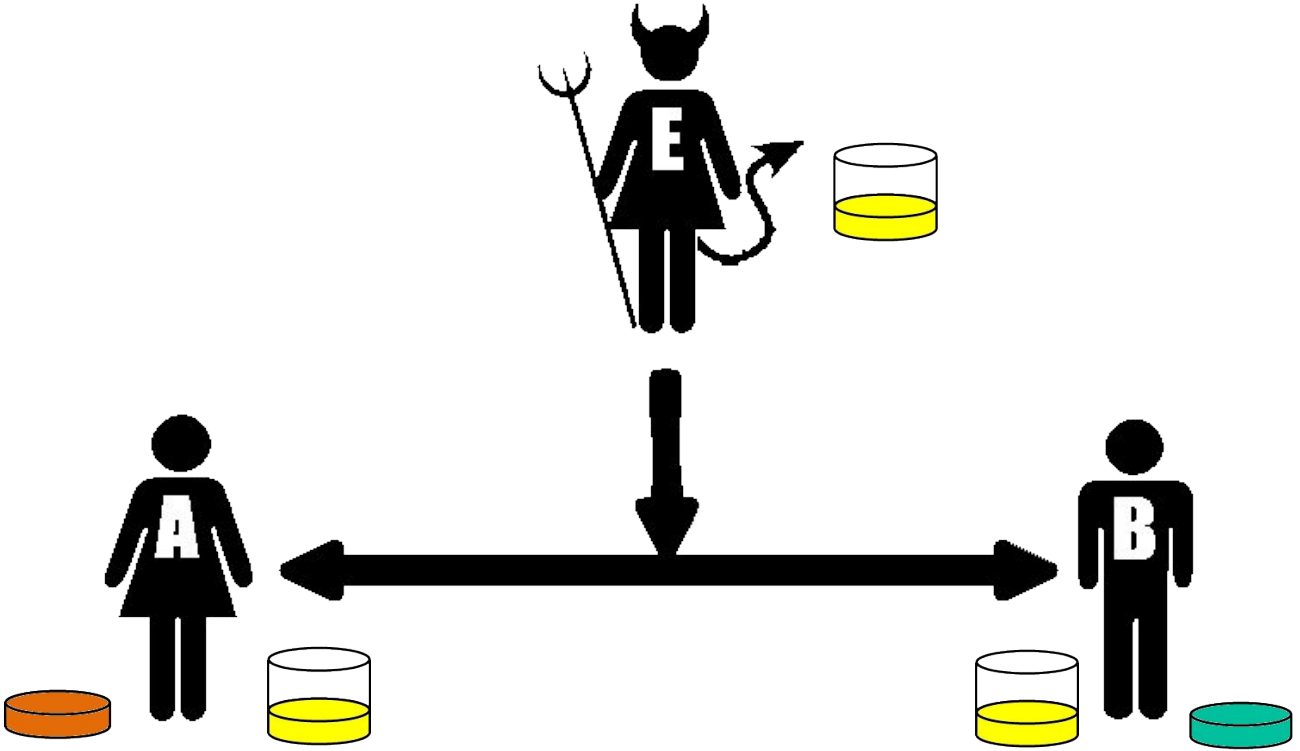
Alice & Bob with Eve listening wish to make a secret shared color



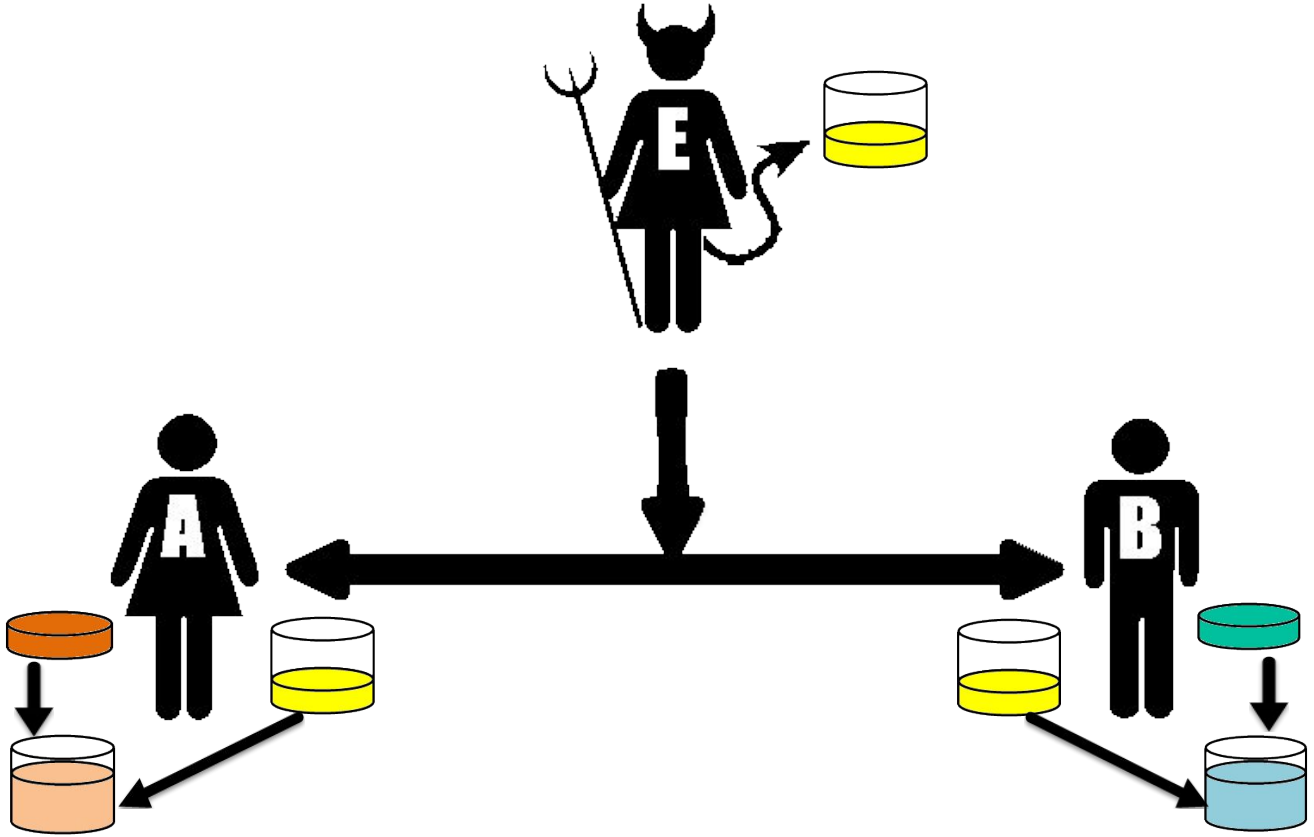
Step 1 - Both publicly agree to a shared color



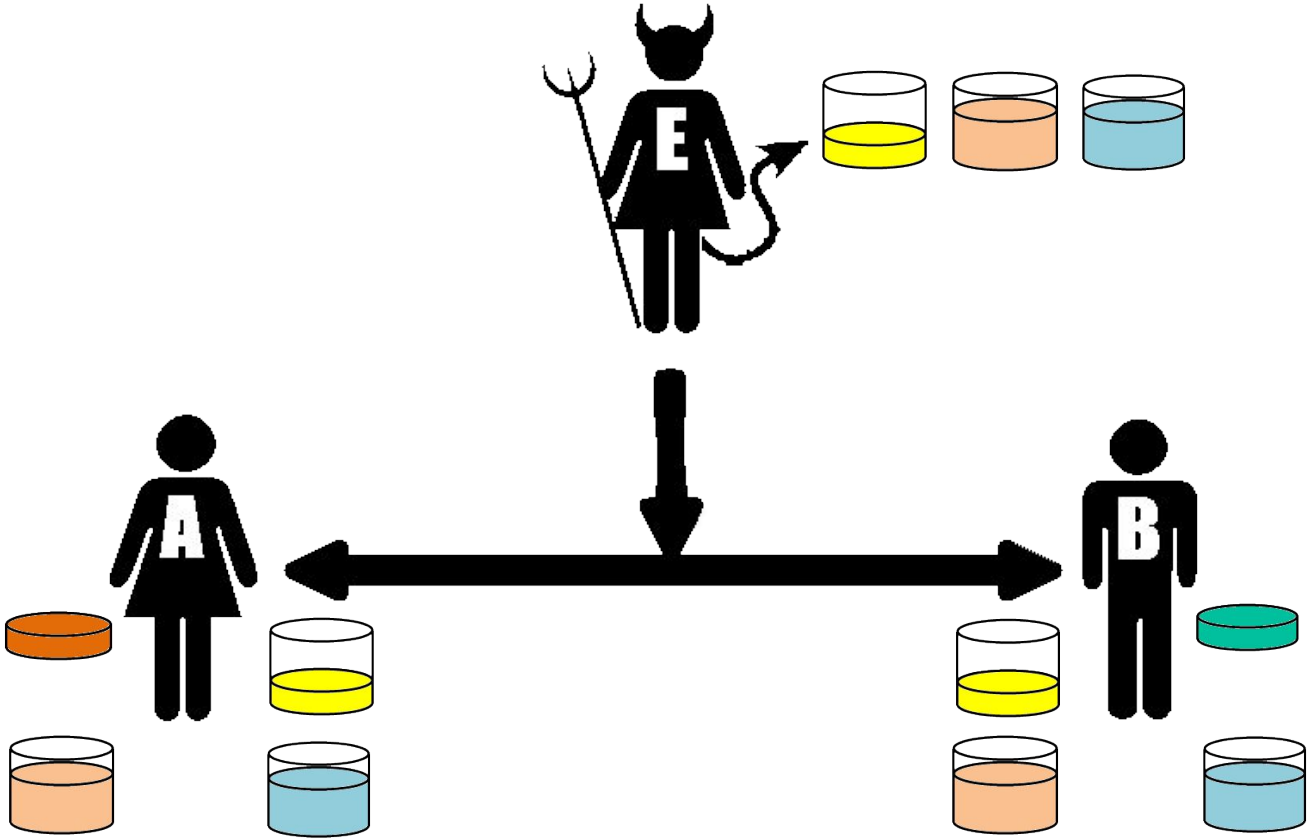
Step 2 - Each picks a secret color



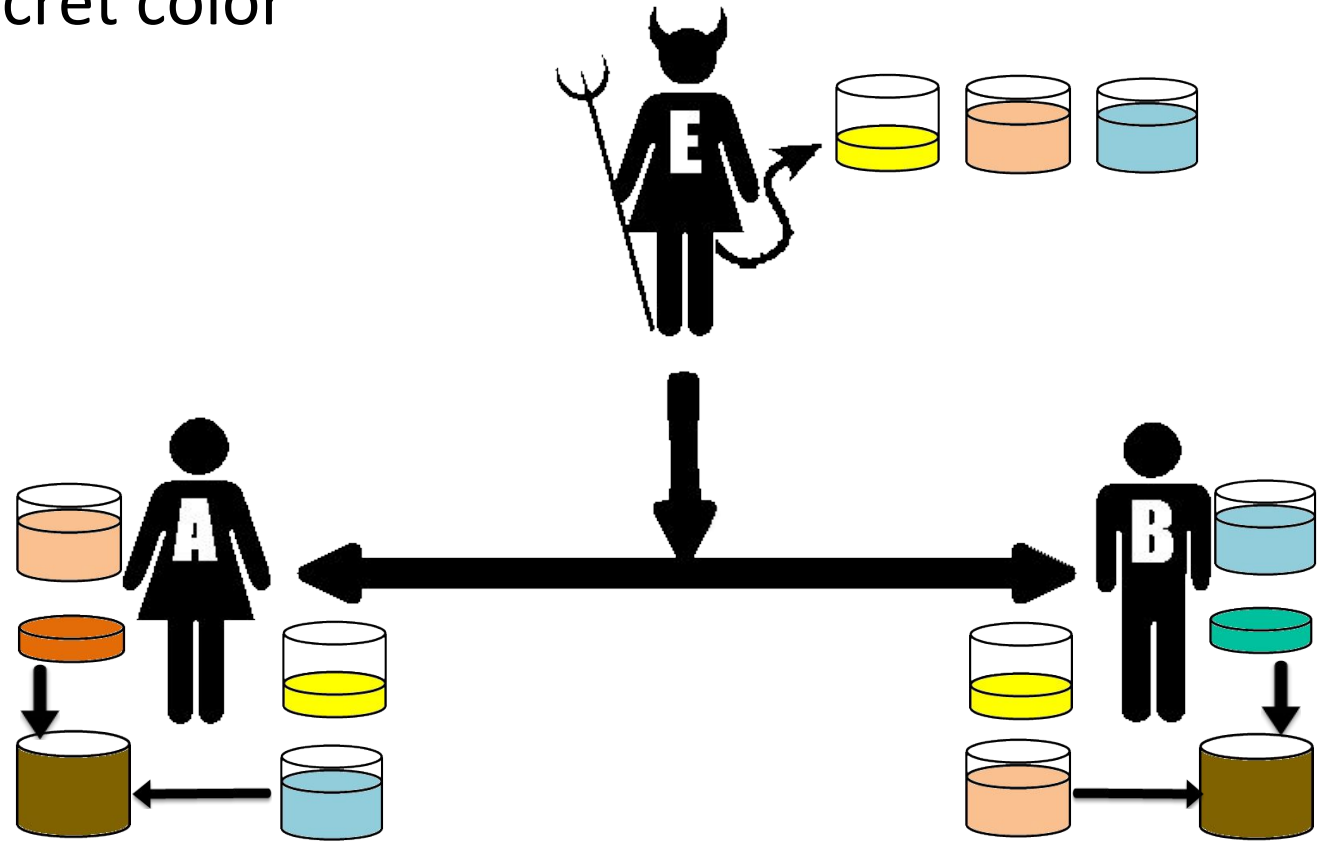
Step 3 - Each adds their secret color to the shared color



Step 4 - Each sends the other their new mixed color



Each combines the shared color from the other with their own secret color



Diffie – Hellman Key Exchange Algorithm steps:

Step-1: Select q (prime number) and α (α is primitive root of q)

Step-2: User A Key Generation: select X_A, $X_A < q$

Calculate **public key Y_A**, $Y_A = \alpha^{X_A} \bmod q$

YA Shared with user B

Step-3: User B Key Generation: select X_B, $X_B < q$

Calculate **public key Y_B**, $Y_B = \alpha^{X_B} \bmod q$

YB shared with user A

Step-4: Calculation of **secret key by user A**: $K = (Y_B)^{X_A} \bmod q$

Step-5: Calculation of **secret key by user B**: $K = (Y_A)^{X_B} \bmod q$

Working Principle

Alice	Bob
Public Keys available = P, G	Public Keys available = P, G
Private Key Selected = a	Private Key Selected = b
Key generated = $x = G^a mod P$	Key generated = $y = G^b mod P$
Exchange of generated keys takes place	
Key received = y	key received = x
Generated Secret Key = $k_a = y^a mod P$	Generated Secret Key = $k_b = x^b mod P$
Algebraically, it can be shown that $k_a = k_b$	



$x = 3$

Large prime numbers

$n = 11$

$g = 7$

$y = 5$



$A = g^x \bmod n$

$= 7^3 \bmod 11$

$= 343 \bmod 11$

$A = 2$

$B = g^y \bmod n$

$= 7^5 \bmod 11$

$= 16807 \bmod 11$

$B = 10$

Large prime numbers



$x = 3$

$n = 11$

$g = 7$

$y = 5$



$A = g^x \bmod n$

$= 7^3 \bmod 11$

$= 343 \bmod 11$

$B = 10$

$K = B^x \bmod n$

$= 10^3 \bmod 11$

$K = 10$

$B = g^y \bmod n$

$= 7^5 \bmod 11$

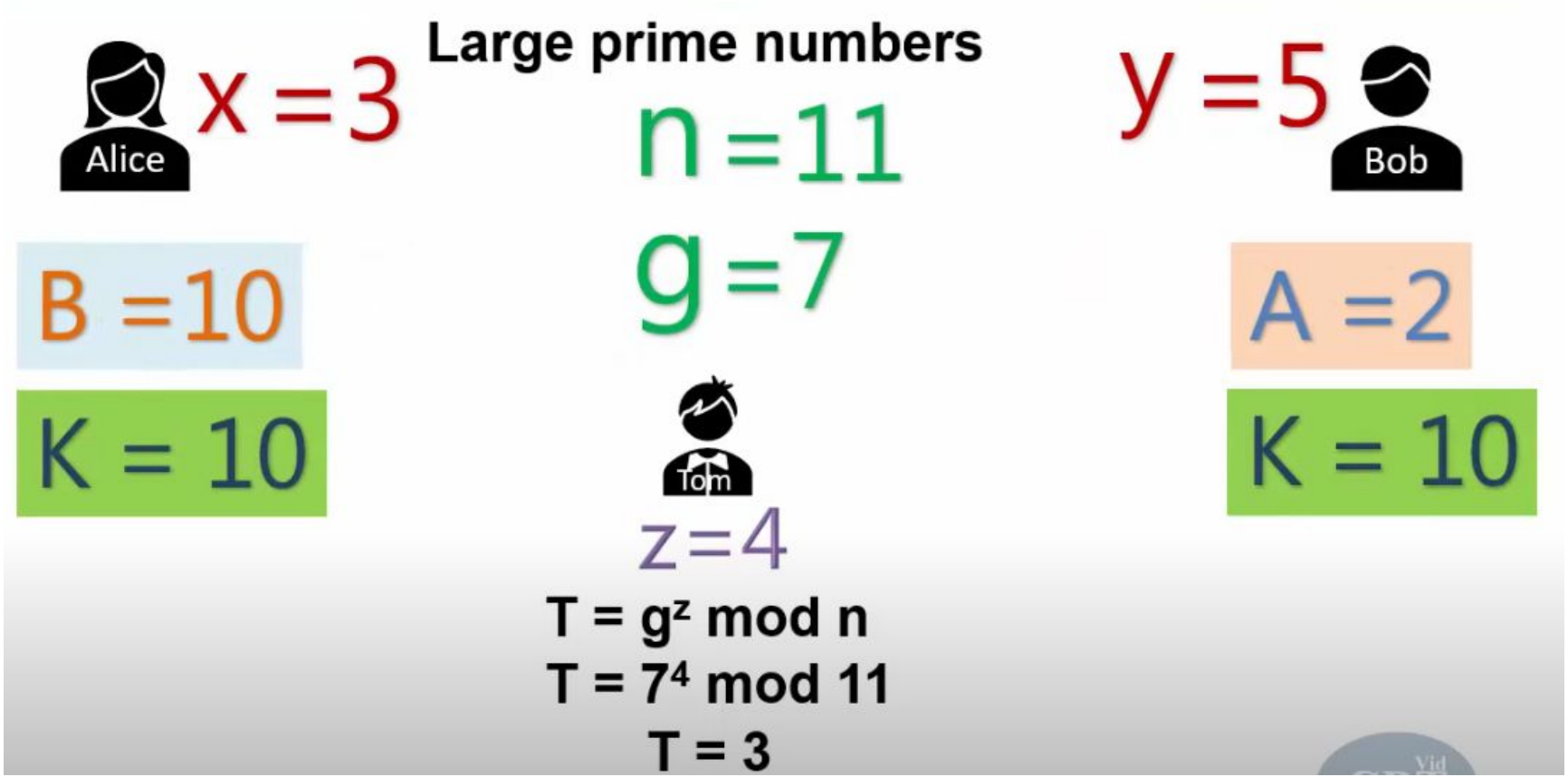
$= 16807 \bmod 11$

$A = 2$

$K = A^y \bmod n$

$= 2^5 \bmod 11$

$K = 10$


Large prime numbers
Alice
x = 3
n = 11
g = 7
y = 5
Bob
B = 10
A = 2
K = 10
K = 10
Tom
z = 4
T = g^z mod n
T = 7^4 mod 11
T = 3



$x = 3$

Large prime numbers

$n = 11$

$g = 7$

$y = 5$



$B = 10$

$K = 10$

$A = 2$

$K = 10$



$z = 4$

$K_A = B^z \bmod n$
$= 10^4 \bmod 11$

$K_A = 1$

$T = g^z \bmod n$
$T = 7^4 \bmod 11$
$T = 3$

$K_B = A^z \bmod n$
$= 2^4 \bmod 11$

$K_B = 5$

Example:

Step 1: Alice and Bob get public numbers P = 23, G = 9

Step 2: Alice selected a private key a = 4 and

Bob selected a private key b = 3

Step 3: Alice and Bob compute public values

Alice: x =(9^4 mod 23) = (6561 mod 23) = 6

Bob: y = (9^3 mod 23) = (729 mod 23) = 16

Step 4: Alice and Bob exchange public numbers

Step 5: Alice receives public key y =16 and

Bob receives public key x = 6

Step 6: Alice and Bob compute symmetric keys

Alice: ka = y^a mod p = 65536 mod 23 = 9

Bob: kb = x^b mod p = 216 mod 23 = 9

Step 7: 9 is the shared secret.

Exercise – 1: $q = 353, \alpha = 3, X_A = 97, X_B = 233.$

Determine ***Public Key*** and ***Shared key*** for both users using Diffie-Hellman key exchange algorithm.

Exercise - 1

Exercise – 1: $q = 353, \alpha = 3, X_A = 97, X_B = 233.$

Determine ***Public Key*** and ***Shared key*** for both users using Diffie-Hellman key exchange algorithm.

Solution:

1. Here $q = 353, \alpha = 3$
2. Calculate *Public key for user A*. $X_A = 97$,

   $Y_A = \alpha^{X_A} \bmod q = 3^{97} \bmod 353 = 40$ **$Y_A = 40$**
3. Calculate *Public key for user B*. $X_B = 233$,

   $Y_B = \alpha^{X_B} \bmod q = 3^{233} \bmod 353 = 248$ **$Y_B = 248$**
4. Calculation of *secret key by user A*,

   $K = (Y_B)^{X_A} \bmod q = 248^{97} \bmod 353 = 160$ **$K = 160$**
5. Calculation of *secret key by user B*,

   $K = (Y_A)^{X_B} \bmod q = 40^{233} \bmod 353 = 160$ **$K = 160$**

Exercise – 2: $q = 23$, $\alpha = 5$, $X_A = 6$, $X_B = 15$.

Determine ***Public Key*** and ***Shared key*** for both users using Diffie-Hellman key exchange algorithm.

Exercise - 2

Exercise – 2: $q = 23, \alpha = 5, X_A = 6, X_B = 15.$

Determine ***Public Key*** and ***Shared key*** for both users using Diffie-Hellman key exchange algorithm.

Solution:

1. Here $q = 23, \alpha = 5$
2. Calculate *Public key for user A*. $X_A = 6$,

   $Y_A = \alpha^{X_A} \bmod q = 5^6 \bmod 23 = 8$ **$Y_A = 8$**
3. Calculate *Public key for user B*. $X_B = 15$,

   $Y_B = \alpha^{X_B} \bmod q = 5^{15} \bmod 23 = 19$ **$Y_B = 19$**
4. Calculation of *secret key by user A*,

   $K = (Y_B)^{X_A} \bmod q = 19^6 \bmod 23 = 2$ **$K = 2$**
5. Calculation of *secret key by user B*,

   $K = (Y_A)^{X_B} \bmod q = 8^{15} \bmod 23 = 2$ **$K = 2$**

Problem with the Algorithm

- Can diffie-hellman solve our problem associated with key exchange? Unfortunately, not quite!
- This algorithm fall into “man in the middle attack”
- This work as follows:
- Step-1:
- Alice want to communicate to Bob so as usual they use diffie-hellman key exchange so Let n=11 & g=7. they find K1 and K2 .

- **Step-2:**
  - Alice does not realize that the attacker EVE is listening the conversation between her and Bob.
  - EVE simply pick up the value of n and g. (n=11,g=7)
  - Alice n=11,g=7 EVE n=11,g=7 Bob n=11,g=7
- **Step-3:**
  - Now let us assume that Alice, EVE & Bob select random number x & y as bellow:
  - Alice EVE Bob

    x=3 x=8,y=6 y=9

- **Step-4:**
  - Now all these person calculate the value of A and B.

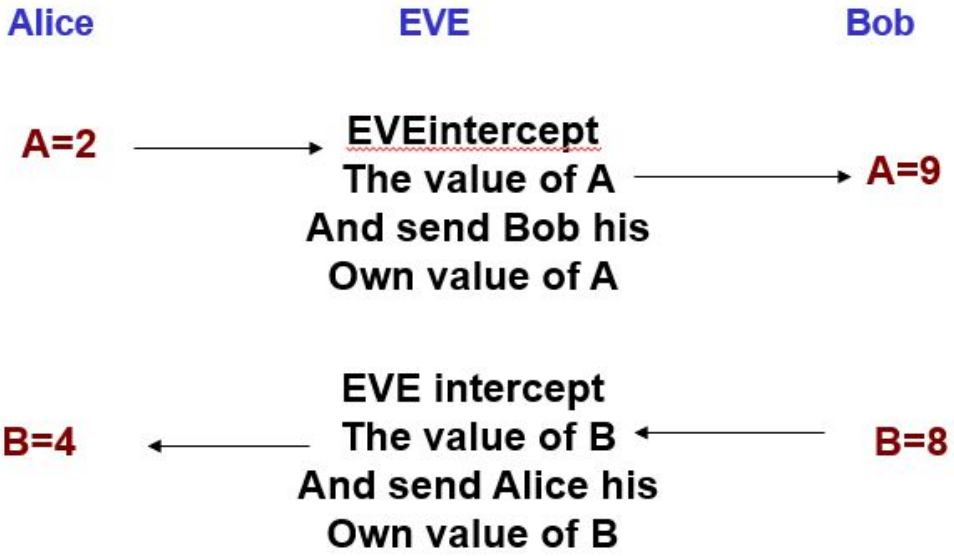
$A=g^x \bmod n$
$=7^3 \bmod 11 = 2$

$A=g^x \bmod n$
$=7^8 \bmod 11 = 9$

$B=g^y \bmod n$
$=7^6 \bmod 11 = 4$

$B=g^y \bmod n$
$=7^9 \bmod 11 = 8$

- **Step-5:**



- **Step-6:**
  - So at last value of A,B to users are:

A=2,B=4*	A=2,B=8	A=9*,B=8

* indicate value of A and B changed.

- **Step-7:**
  - Based on these values, all the three person now calculate their Keys.

Alice	EVE	Bob
$K1=B^x \bmod n$ $=4^3 \bmod 11$ $=9$	$K1=B^x \bmod n$ $=8^8 \bmod 11$ $=5$ $K2=A^y \bmod n$ $=2^6 \bmod 11$ $=9$	$K2=A^y \bmod n$ $=9^9 \bmod 11$ $=5$

Solution: ? (User authentication required)

Thank you!

Follow us



isfcr.pesu



www.isfcr.pes.edu



ISFCR

PESU ISFCR

PESU Center for
Information Security, Forensics and Cyber Resilience

PES UNIVERSITY



PESU Center for
Information Security,
Forensics and
Cyber Resilience



Welcome to

PES University

Ring Road Campus, Bengaluru



PESU Center for
Information Security,
Forensics and
Cyber Resilience



Applied Cryptography

UE24CS342AA4

Dr. Shruti Jadon

LECTURE

A Note on Security



Emergency Exit



Assembly Point



Washroom



No Chatting



Phones on silent



No Sleeping

A Note on Security

- In this course, you will be exposed to information about security problems and vulnerabilities with computing systems and networks.
- To be clear, you are not to use this or any other similar information to test the security of, break into, compromise, or otherwise attack, any system or network without the express consent of the owner.
- In particular, you will comply with all my instructions when doing the labs.
  - My instructions are in consonance with applicable laws of India and PES University policies.
  - If in any doubt, please consult your professor!
- Any violation is at **YOUR RISK!**
  And may result in severe consequences.

Introduction

- The security of many cryptographic schemes, particularly those related to Diffie-Hellman key exchange and Elgamal encryption, rely on the computational intractability of finding solutions to the Discrete Logarithm Problem (DLP).
- The Diffie-Hellman Key Exchange (DHKE), the first published asymmetric scheme, is based on the closely related Diffie-Hellman Problem (DHP)
- The DLP is fundamental to modern asymmetric cryptography and serves as one of the most important one-way functions.
- The DLP is inherently defined over a finite cyclic group G.

Definition

- A function f() is considered one-way if calculating the output y=f(x) is computationally easy, but inverting it to find $x=f^{-1}(y)$ is computationally infeasible.

Definition 8.3.1 Discrete Logarithm Problem (DLP) in $\mathbb{Z}_p^*$
Given is the finite cyclic group $\mathbb{Z}_p^*$ of order $p-1$ and a primitive element $\alpha \in \mathbb{Z}_p^*$ and another element $\beta \in \mathbb{Z}_p^*$. The DLP is the problem of determining the integer $1 \leq x \leq p-1$ such that:

$$\alpha^x \equiv \beta \mod p$$

- This integer x is termed the discrete logarithm of β to the base α

An easy example

Suppose we know:

$2^x \equiv 3 \pmod 5$

x	2^x mod 5
1	2
2	4
3	3
4	1

We find that:

x=3

because 2^3=8, and 8 mod 5=3.

The Discrete Logarithm Problem

Example 1: Solve $\log_2 9 \bmod 11$.

Solution:

Here p=11, g=2, X=9

$\log_g X \equiv n \pmod p$

$X \equiv g^n \pmod p$

$9 \equiv 2^n \pmod{11}$

Try 'n' = 1, 2, 3, ...

$9 \equiv 2^6 \pmod{11}$

Answer is 6.

DLP in Conventional Cryptography (Z_p^*)

- The DLP is most commonly studied within the multiplicative group of a prime field Z_p^* or a designated subgroup of it.
- In this context, the group operation is modular multiplication, and the problem requires determining the integer x that solves the equation:

$$\alpha^x \equiv \beta \pmod{p}$$

- Since computing this modular exponentiation is easy, but solving for the exponent (x) is hard (for sufficiently large parameters), this forms the necessary one-way function.

DLP in Conventional Cryptography (Z_p^*)

- The DHKE allows two parties (Alice and Bob) to derive a shared secret:

$$k_{AB} \equiv \alpha^{ab} \pmod{p}$$

solely by exchanging these public values over an insecure channel

$$A \equiv \alpha^{a} \text{ and } B \equiv \alpha^{b}$$

- Additionally, the Elgamal encryption scheme is a crucial extension of the DHKE, where the derived session key is used as a multiplicative mask to encrypt a message, and its security is also rooted in the DHP/DLP.

1. Alice privately chooses an integer a.
2. Bob privately chooses an integer b.
3. Alice calculates A sends A to Bob.
4. Bob calculates B and sends B to Alice.
5. Alice calculates B^a mod p.
6. Bob calculates A^b mod p.

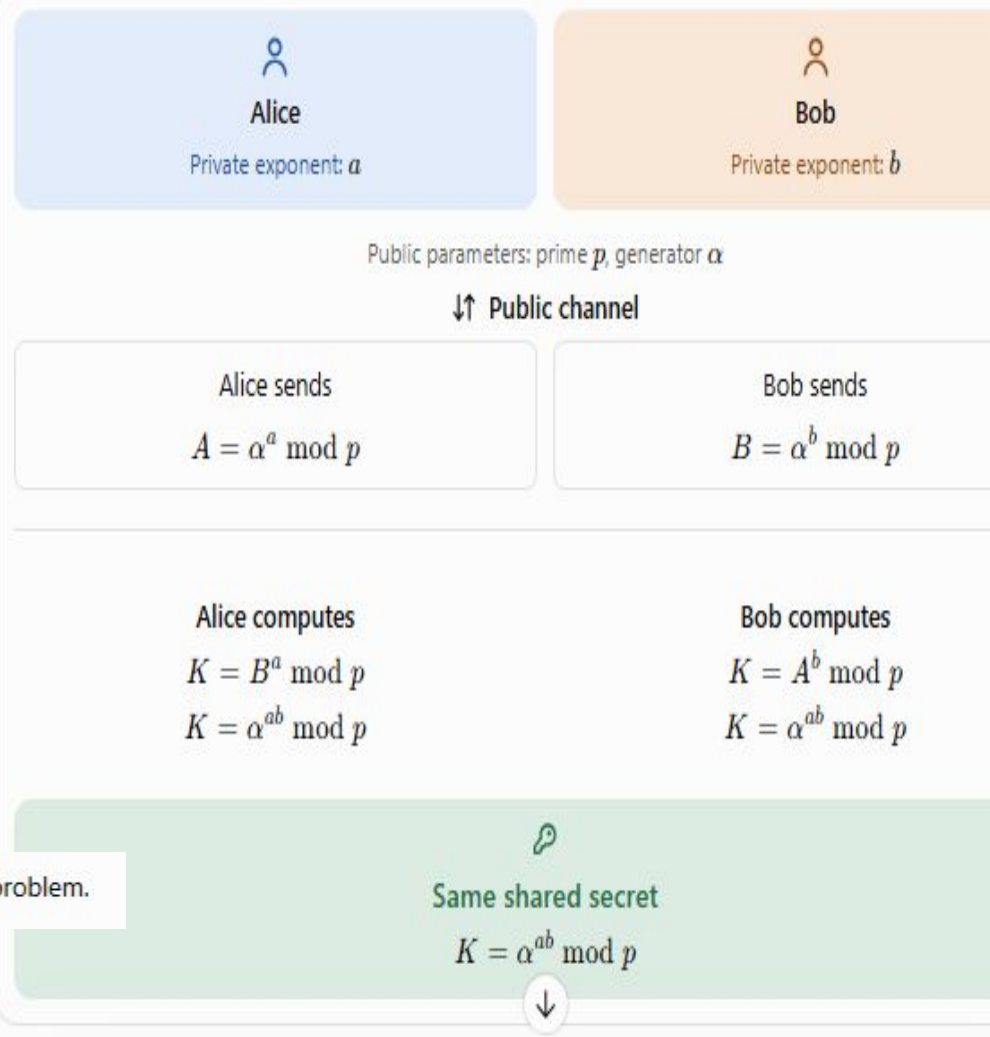
Both get the same answer because:

$$B^a = (\alpha^b)^a = \alpha^{ab}$$

and

$$A^b = (\alpha^a)^b = \alpha^{ab}$$

Eve sees p, α, A, B, but recovering the private exponents a and b is a DLP-type problem.



Generalization and Elliptic Curve DLP (ECDLP)

- The DLP is not restricted to prime fields; it can be generalized to any cyclic group where the DL problem is intractable.
- A major application of this generalization is in Elliptic Curve Cryptography (ECC), which is based on the Elliptic Curve Discrete Logarithm Problem (ECDLP).
- For the ECDLP, the cyclic group G is formed by the points on a specific elliptic curve E, and the group operation is defined as point addition
- The ECDLP challenge is to find the integer d (the private key) such that repeated point addition of a primitive base point P results in the public key point

$$T: P + P + \cdots + P = dP = T$$

- Point multiplication (dP) is efficiently computed using the Double-and-Add Algorithm, which is analogous to the square-and-multiply algorithm used for modular exponentiation
- A key advantage of ECC is that it requires considerably shorter operands (e.g., 160–256 bits) to achieve the same security level as traditional DL schemes in Z_p^* (which often require

Security and Attacks Against the DLP

- Algorithms for solving the DLP can be classified into two main categories:
  - Generic algorithms: work in any cyclic group
  - Non-generic algorithms: exploit the inherent algebraic structure of specific groups

Index-Calculus Method (Nongeneric)

- This is the most powerful attack against the DLP in Z_p^* and $GF(2^m)^*$
- It has a subexponential running time, meaning it is significantly faster than generic methods for these groups
- This attack forces the use of very large primes p (e.g., 1024 bits for 80-bit security)
- Critically, the index-calculus method is not applicable to elliptic curve groups, which is the main reason for ECC's shorter key sizes

Generic Attacks

- These methods, which include Pollard's Rho method and Shanks' Baby-Step Giant-Step method, have a running time proportional to the square root of the group order, O(|G|)
- Pollard's Rho is currently the best known attack for elliptic curve groups.
- To counter these square-root attacks, cyclic groups must have an order of at least 2^{160} to achieve 80-bit security, or 2^{256} for 128-bit security.

Pohlig-Hellman Algorithm

- This specific attack exploits small prime factors of the group order,

$$| Z_p^* | = p-1$$

- To prevent this, cryptographic systems must ensure that the largest prime factor of the group order is itself large (e.g., at least 160 bits long)
- This is why DHKE protocols often operate within a subgroup of Z_p^* that has a prime order q.

Thank you!

Follow us



isfcr.pesu



www.isfcr.pes.edu



ISFCR

PESU ISFCR

PESU Center for
Information Security, Forensics and Cyber Resilience

PES UNIVERSITY



PESU Center for
Information Security,
Forensics and
Cyber Resilience



Welcome to

PES University

Ring Road Campus, Bengaluru



PESU Center for
Information Security,
Forensics and
Cyber Resilience



Applied Cryptography

UE24CS342AA4

Dr. Shruti Jadon

LECTURE 23

A Note on Security



Emergency Exit



Assembly Point



Washroom



No Chatting



Phones on silent



No Sleeping

A Note on Security

- In this course, you will be exposed to information about security problems and vulnerabilities with computing systems and networks.
- To be clear, you are not to use this or any other similar information to test the security of, break into, compromise, or otherwise attack, any system or network without the express consent of the owner.
- In particular, you will comply with all my instructions when doing the labs.
  - My instructions are in consonance with applicable laws of India and PES University policies.
  - If in any doubt, please consult your professor!
- Any violation is at **YOUR RISK!**
  And may result in severe consequences.

Elliptic Curve Cryptography

ECC

Elliptic Curve Cryptography (ECC)

- The Elliptic Curve Cryptography (ECC) is modern family of public-key cryptosystems, which is based on the algebraic structures of the **elliptic curves over finite fields** and on the difficulty of the **Elliptic Curve Discrete Logarithm Problem (ECDLP).**
- The **ECC cryptography** is considered a natural modern **successor of the RSA** cryptosystem, because ECC uses **smaller keys** and signatures than RSA for the same level of security and provides very **fast key generation**, **fast key agreement** and **fast signatures**.

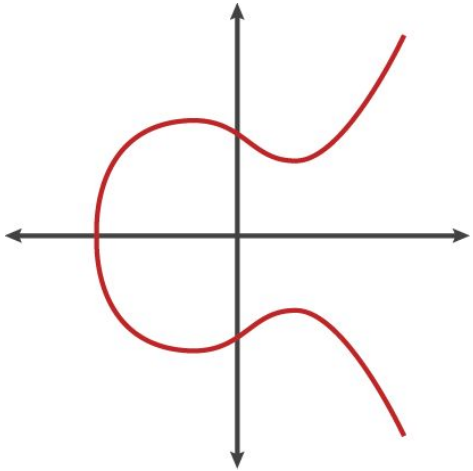
Outline

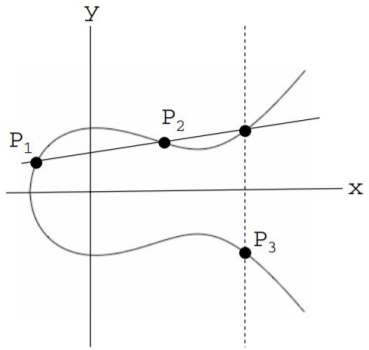
- Elliptic curves over reals
- Elliptic curves over Z_p
- ECDH and ECDSA

Introduction to ECC

- An elliptic curve is defined by a equation in two variable with coefficients
- Elliptic curves are not ellipse. Elliptic curves are described by cubic equation
- Elliptic curve cryptography makes use of elliptic curves, in which the variables and coefficients are all restricted to elements of finite fields

Elliptic Curves





- A set of points that satisfy a specific mathematical equation:

  $y^2 = x^3 + ax + b.$

Properties:

- It is symmetric about the x-axis.
- Any non-vertical line will intersect the curve in at most three places.
- If P_1 and P_2 are on E, we can define,

  $P_3 = P_1 + P_2$

Elliptic Curves

- Elliptic curve over real numbers are nothing byt set of points (x,y) which satisfy an elliptic curve equation y2=x3+ax+b, where x, y, a and b are real numbers
- We can create curves over other fields
-

Definition 9.1.1 Elliptic Curve

The elliptic curve *over* $\mathbb{Z}_p$, $p > 3$, *is the set of all pairs* $(x,y) \in \mathbb{Z}_p$ *which fulfill*

$$y^2 \equiv x^3 + a \cdot x + b \bmod p \qquad (9.1)$$

together with an imaginary point of infinity $\mathcal{O}$, *where*

$$a, b \in \mathbb{Z}_p$$

and the condition $4 \cdot a^3 + 27 \cdot b^2 \neq 0 \bmod p$.

Elliptic Curve Discrete Logarithm Problem

- Cryptosystems rely on the hardness of the Elliptic Curve Discrete Logarithm Problem (ECDLP)
- Cryptosystems are based on the idea that d is large and kept secret and attackers cannot compute it easily
- If d is known, an efficient method to compute the point multiplication dP is required to create a reasonable cryptosystem
- The method for efficient point multiplication on elliptic curves: Double-and-AddAlgorithm

Definition 9.2.1 Elliptic Curved Discrete Logarithm Problem (ECDLP)

Given is an elliptic curve E. We consider a primitive element P and another element T. The DL problem is finding the integer d, where $1 \leq d \leq \#E$, *such that:*

$$\underbrace{P+P+\cdots+P}_{d \text{ times}} = dP = T. \tag{9.2}$$

Double and Add Algorithm

Input: Elliptic curve E, an elliptic curve point P and a scalar d with bits d_i

Output: $T = d\,P$

Initialization:

$T = P$

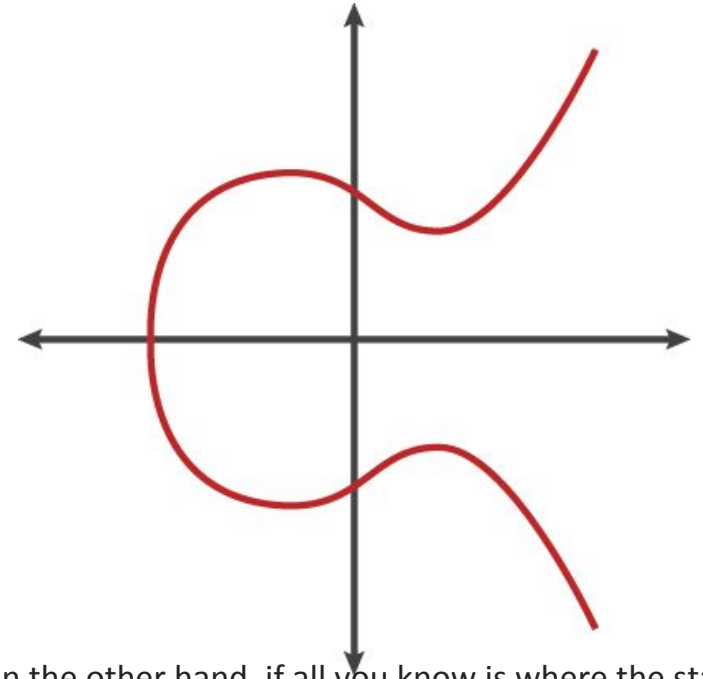
Algorithm:

```
1     FOR i = t − 1 DOWNTO 0
1.1       T = T + T mod n
          IF d_i = 1
1.2           T = T + P mod n
2     RETURN (T)
```

Double and Add Algorithm for Point Multiplication

Step		
#0	$P = \mathbf{1}_2 P$	inital setting, bit processed: $d_4 = 1$
#1a	$P + P = 2P = \mathbf{10}_2 P$	DOUBLE, bit processed: d_3
#1b	$2P + P = 3P = 10_2 P + 1_2 P = \mathbf{11}_2 P$	ADD, since $d_3 = 1$
#2a	$3P + 3P = 6P = 2(11_2 P) = \mathbf{110}_2 P$	DOUBLE, bit processed: d_2
#2b		no ADD, since $d_2 = 0$
#3a	$6P + 6P = 12P = 2(110_2 P) = \mathbf{1100}_2 P$	DOUBLE, bit processed: d_1
#3b	$12P + P = 13P = 1100_2 P + 1_2 P = \mathbf{1101}_2 P$	ADD, since $d_1 = 1$
#4a	$13P + 13P = 26P = 2(1101_2 P) = \mathbf{11010}_2 P$	DOUBLE, bit processed: d_0
#4b		no ADD, since $d_0 = 0$

ECC's Trapdoor

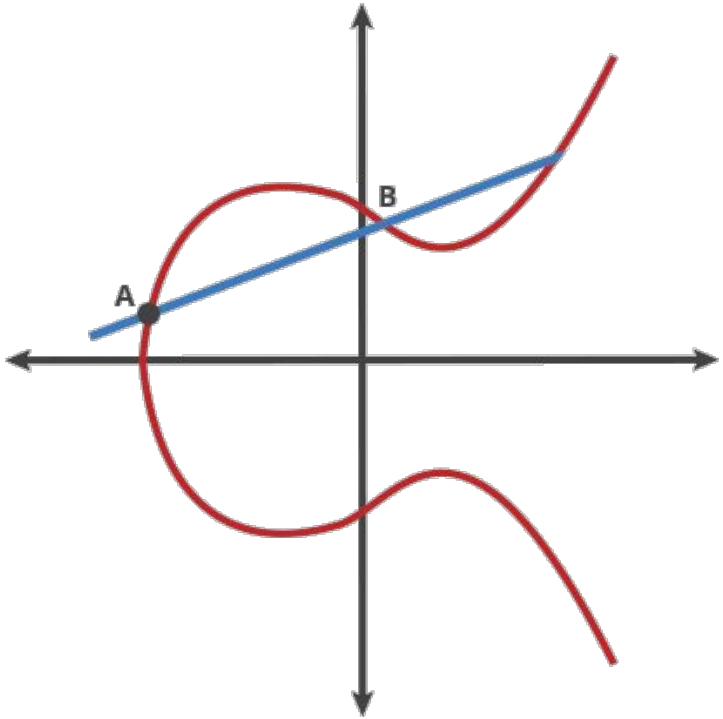


On the other hand, if all you know is where the starting point and ending point are, it's nearly impossible to find how many hops it took to get there.

- First, we start with an arbitrary point on the curve (say, A).
- Next, we use the dot function to find a new point.
- Finally, we keep repeating the dot function to hop around the curve until we finally end up at our last point (say, E).

- **Trapdoor?**
- If you know where the starting point (A) is and how many hops are required to get to the ending point (E), it's very easy to find the ending point.

ECC's Trapdoor



- Starting at **A**:
- **A dot B = -C** (Draw a line from A to B and it intersects at -C)
- Reflect across the X-axis from -C to C
- **A dot C = -D** (Draw a line from A to C and it intersects -D)
- Reflect across the X-axis from -D to D
- **A dot D = -E** (Draw a line from A to D and it intersects -E)
- Reflect across the X-axis from -E to E

- **Public Key:** Starting Point A, Ending Point E
- **Private Key:** Number of hops from A to E

What makes ECC hard to break?

- The elliptic curve discrete logarithm problem is:
  - **given k, P -> “easy” to compute Q**
  - **given Q, P -> “hard” to find k**
- The problem is computationally intractable for large values of k.
- To find the private key k, an attacker would need to calculate the discrete logarithm: k = log k mod p.
- known as the elliptic curve logarithm problem
  - **k must be large enough**

ECC Algorithms

- **E**lliptic **C**urve **D**iscrete **S**igning **A**lgorithm (ECDSA) for Digital Signature.
- **E**lliptic **C**urve **D**iffie-**H**ellman (ECDH) for key agreement.
- ECC **encryption** algorithms and hybrid encryption schemes like the ECIES.

ECDSA - Elliptic Curve Discrete Signing Algorithm

- It is a digital signature scheme which provides authenticity, integrity and non-repudiation
- ECDSA is protocol combining ECC+ hashing to create digital signatures
- Ecdsa relies on elliptic curve math like secp256k1, curve25519 and hash functions (like SHA-256 or SHA-3)
- ECDSA involves 2 main operation
  1) Signing
     a) The sender uses their private key to create a signature on a message.
     b) This signature depends on: The message's hash (like SHA-256), sender's private key, random number (nonce)

ECDSA - Elliptic Curve Discrete Signing Algorithm

1) Verification
   a) The receiver uses the sender's public key to verify that:
      i) The signature (r, s) is valid for the given message.
      ii) The message hasn't been tampered with.
   b) If verification succeeds → message is authentic.
   c) If not → message was altered or not signed by the true sender.

Real life use cases

- It is used in bitcoin, ethereum to sign transactions
- Used in TLS certificates
- In DigitalIDS/ Smart Cards for secure identity verification

ECDSA - Elliptic Curve Discrete Signing Algorithm

Weakness

- Each signature in ECDSA uses a unique random number (nonce) called k. If the same k is reused or poorly generated, the attacker can recover your private key instantly.
- ECDSA math is complex. Small coding mistakes can create side-channel leaks or incorrect signatures.
- Attackers can exploit timing, power consumption, or EM radiation during signing to infer private keys.

ECDH- Elliptic curve Diffie-Hellman

- Classic diffie-hellman Works in a multiplicative group of integers modulo a large prime number p.
- ECDH works in the additive group of points on a elliptic curve over a finite field
- Security relies on the Elliptic Curve Discrete Logarithm Problem (ECDLP).
- No sub-exponential algorithm is known for solving ECDLP the best known attacks grow exponentially.
- This means ECDH gives the same security with much smaller keys.

ECDH- Elliptic curve Diffie-Hellman

Real life use cases

- It is used in TLS handshakes to establish a shared encryption key between your browser and a website.
- Many VPN protocols (like OpenVPN, IKEv2/IPsec, and WireGuard) use ECDH/ECDHE to generate session keys.

Weakness

- Invalid curve attacks can happen if implementations don't check whether received public keys are valid points on the curve.
- ECDH provides only key agreement, not authentication. It's vulnerable to man-in-the-middle (MITM) attacks unless combined with digital signatures or certificates

Security of ECC

- To **protect** a 128-bit AES key it would take a:
  - RSA Key Size: 3072 bits
  - ECC Key Size: 256 bits
- How do we strengthen RSA?
  - Increase the key length
- **Impractical?**

NIST guidelines for public key sizes for AES

ECC KEY SIZE (Bits)	RSA KEY SIZE (Bits)	KEY SIZE RATIO	AES KEY SIZE (Bits)
163	1024	1 : 6	
256	3072	1 : 12	128
384	7680	1 : 20	192
512	15 360	1 : 30	256

Supplied by NIST to ANSI X9F1

Applications of ECC

- Many devices are small and have limited storage and computational power
- Where can we apply ECC?
  - Wireless communication devices
  - Smart cards
  - Web servers that need to handle many encryption sessions
- Any application where security is needed but lacks the power, storage and computational power that is necessary for our current cryptosystems.

Benefits of ECC

- Same benefits of the other cryptosystems: confidentiality, integrity, authentication and non-repudiation but…
- Shorter key lengths
  - Encryption, Decryption and Signature Verification speed up
  - Storage and bandwidth savings

Thank you!

Follow us



isfcr.pesu



www.isfcr.pes.edu



ISFCR

PESU ISFCR

PESU Center for
Information Security, Forensics and Cyber Resilience

PES UNIVERSITY



PESU Center for
Information Security,
Forensics and
Cyber Resilience



Welcome to

PES University

Ring Road Campus, Bengaluru



PESU Center for
Information Security,
Forensics and
Cyber Resilience



Applied Cryptography

UE24CS342AA4

Dr. Shruti Jadon

LECTURE 24

A Note on Security



Emergency Exit



Assembly Point



Washroom



No Chatting



Phones on silent



No Sleeping

A Note on Security

- In this course, you will be exposed to information about security problems and vulnerabilities with computing systems and networks.
- To be clear, you are not to use this or any other similar information to test the security of, break into, compromise, or otherwise attack, any system or network without the express consent of the owner.
- In particular, you will comply with all my instructions when doing the labs.
  - My instructions are in consonance with applicable laws of India and PES University policies.
  - If in any doubt, please consult your professor!
- Any violation is at **YOUR RISK!**
  And may result in severe consequences.

Scenario 1

- Suppose a software company, SoftAreUs, releases a soft- ware update for its product. Customers download the software update file U by some means, say from a public distribution site or from a peer-to-peer network. Before installing U on their machine, customers want to verify that U really is from SoftAreUs.
- How do you verify ?

Scenario 2

- As a second motivating example, suppose Bob receives an email claiming to be from his friend Alice. Bob wants to verify that the email really is from Alice.
- How does Bob do it?

Topics to be discussed in Today's Class

- Digital Signatures
  - Need for Digital Signatures
  - DS Process
  - Services offered by Digital Signatures.

Let's recollect - Public Key Infrastructure – PKI

- A cryptographic system that uses two keys.
- A **public key** known to everyone
- **A private key ,** has full control to the key owner.
- A unique element to the public key system is that the public and private keys are related in such a way that only the public key can be used to encrypt messages and only the corresponding private key can be used to decrypt them.
- It is virtually impossible to deduce the private key if you know the public key.

Digital Signatures

Digital Signatures - Introduction

- Digital signature is a mathematical scheme that is used to authenticate the sender of an electronic document
- A digital signature is nothing but an attachment to any piece of electronic information, which represents the content of the document and the identity of the owner of that document uniquely

Digital Signatures - Motivation

Electronic Records

1. Easy to make copies
2. Fast distribution
3. Copies are as good as original
4. Easily modifiable



Therefore, they lack integrity & authenticity

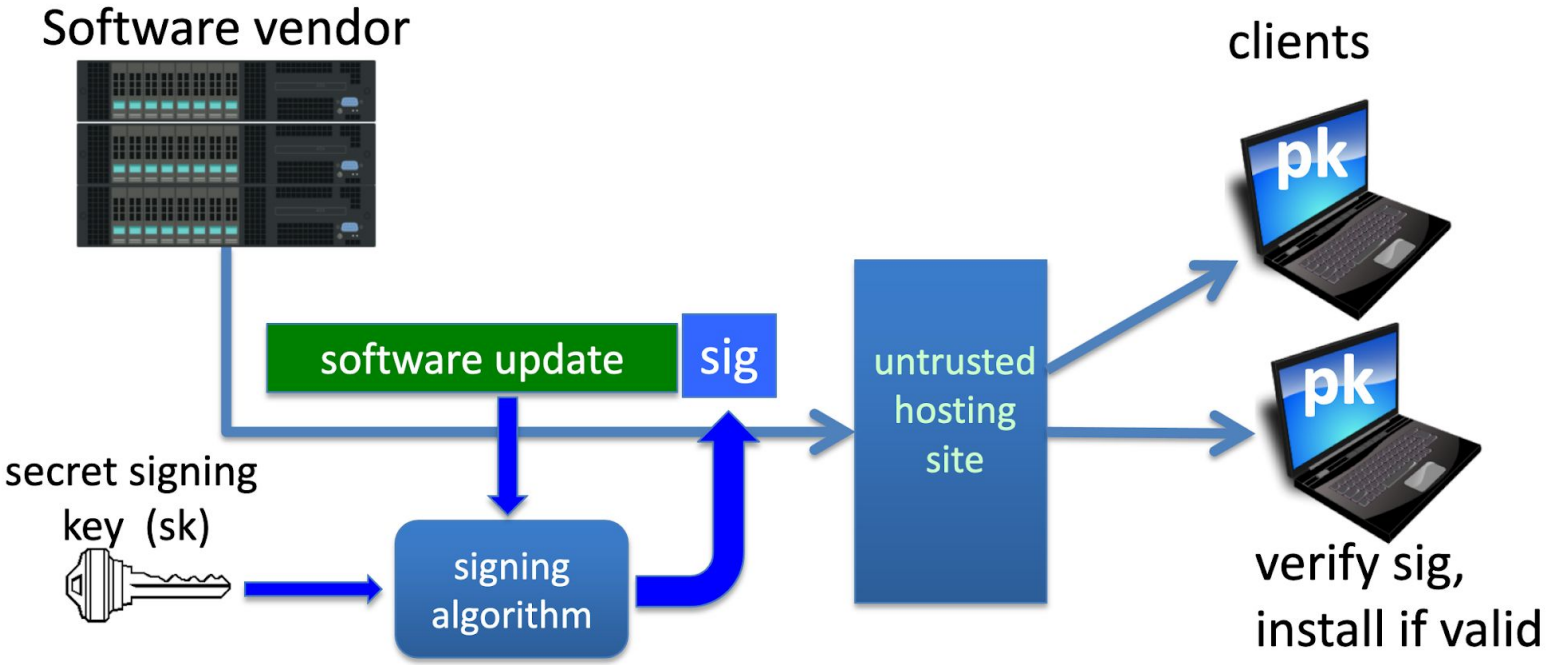
Digital Signatures - Motivation

- To provide Authenticity, Integrity and Non-repudiation to electronic documents.
- To use the Internet as the safe and secure medium for e-Commerce and e-Governance

Who uses digital signatures?

- HR teams
- Governments
- Cryptocurrencies
- Financial sectors

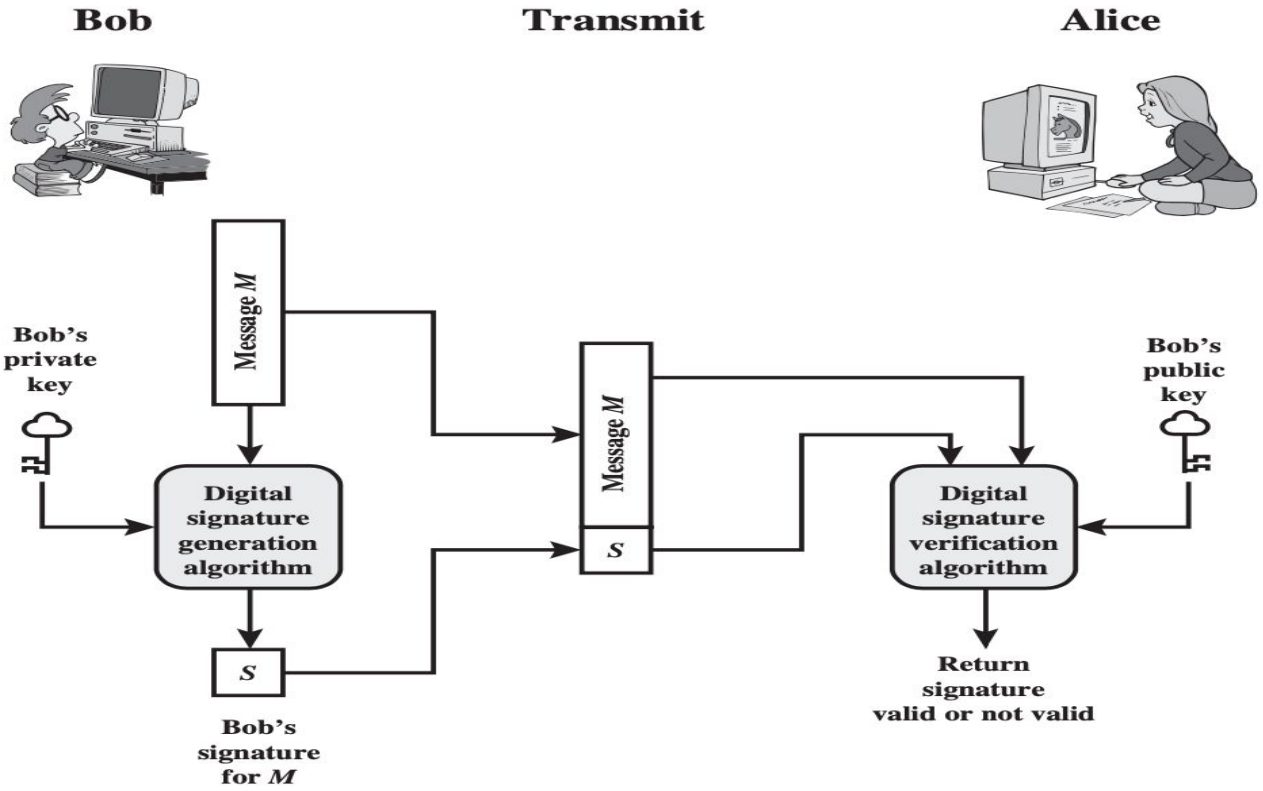
A more realistic example



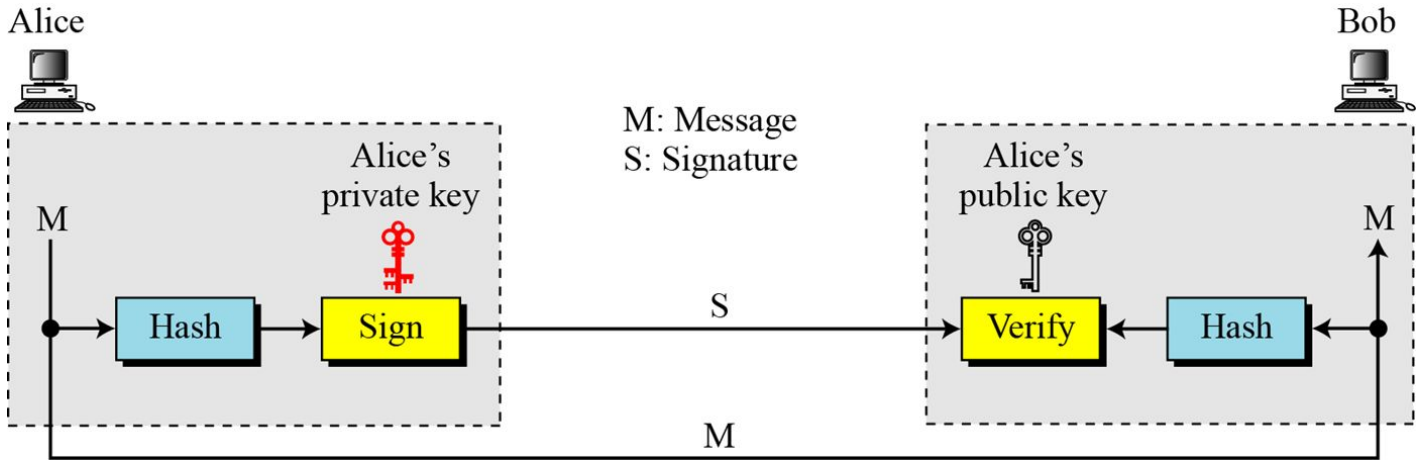
Digital Signature - Process Steps

- Key generation
- Signing
- Verification

Generic Model of Digital Signature Process



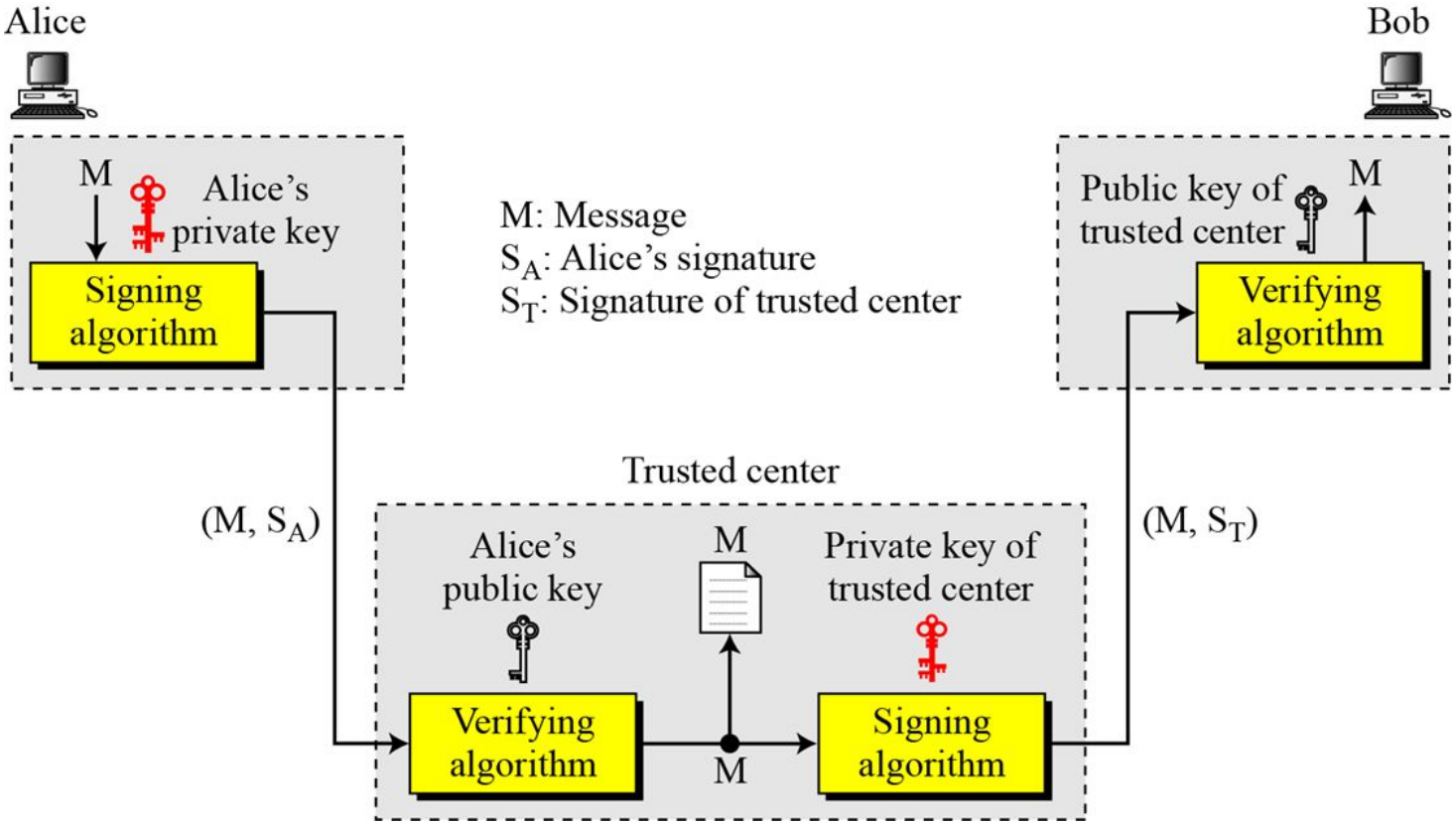
Signing the Digest



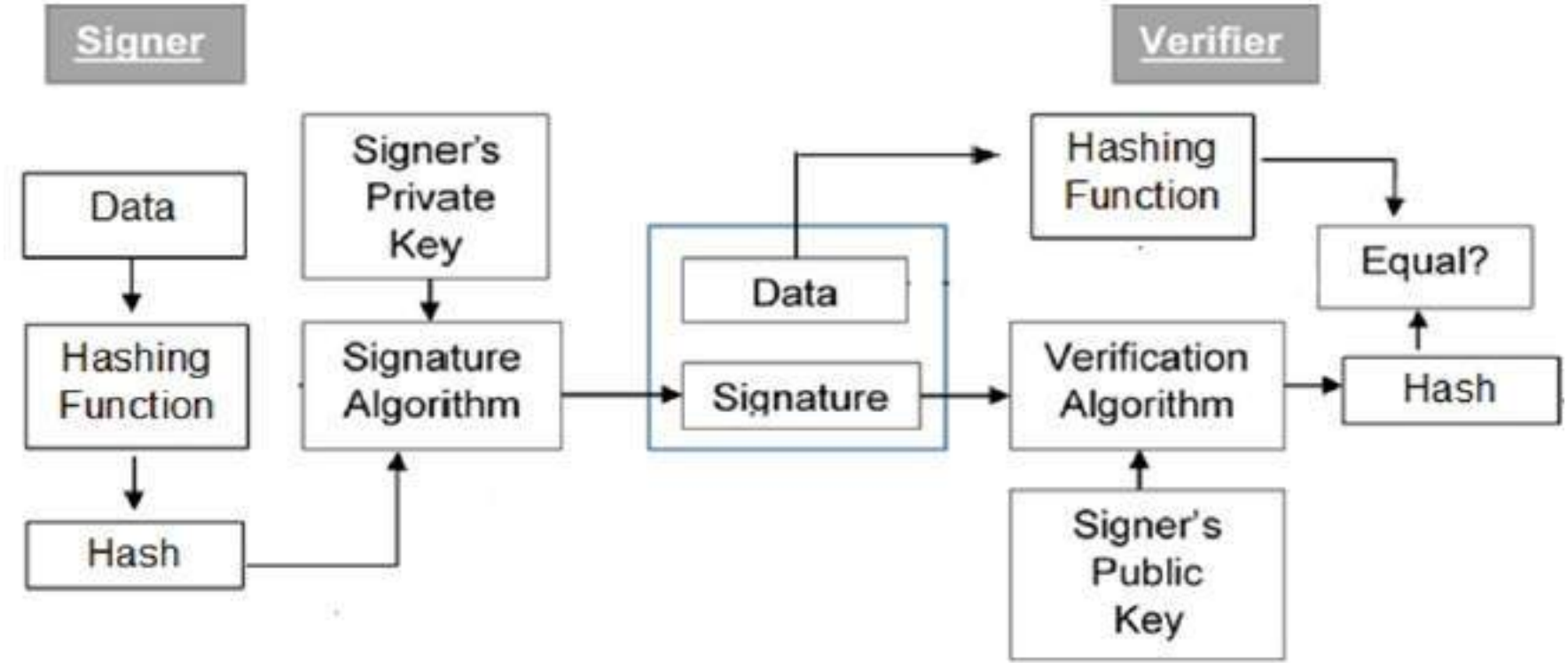
Services

- Message Authentication
- Message Integrity
- Nonrepudiation
- Confidentiality

Using a trusted center for nonrepudiation



Digital Signature – Block Diagram



Digital Signature – Block Diagram

☞ The block diagram of the digital signature process is as follows:

- Message or Document: This is the data or document that needs to be signed.
- Hashing: The document or message is converted into a fixed-length hash using a cryptographic hash function.
- Encryption: The hash is then encrypted using the sender's private key.
- Signature: The encrypted hash is the document's digital signature.
- Verification: The signature is then verified using the sender's public key to ensure the signature's validity and the document's authenticity.

Digital Signatures – Key concepts

This is a course on Cryptography

efcc61c1c03db8d8ea8569545c073c814a0ed755

My place of birth is at Bengaluru.

fe1188eecd44ee23e13c4b6655edc8cd5cdb6f25

I am an Infosec Engineer.

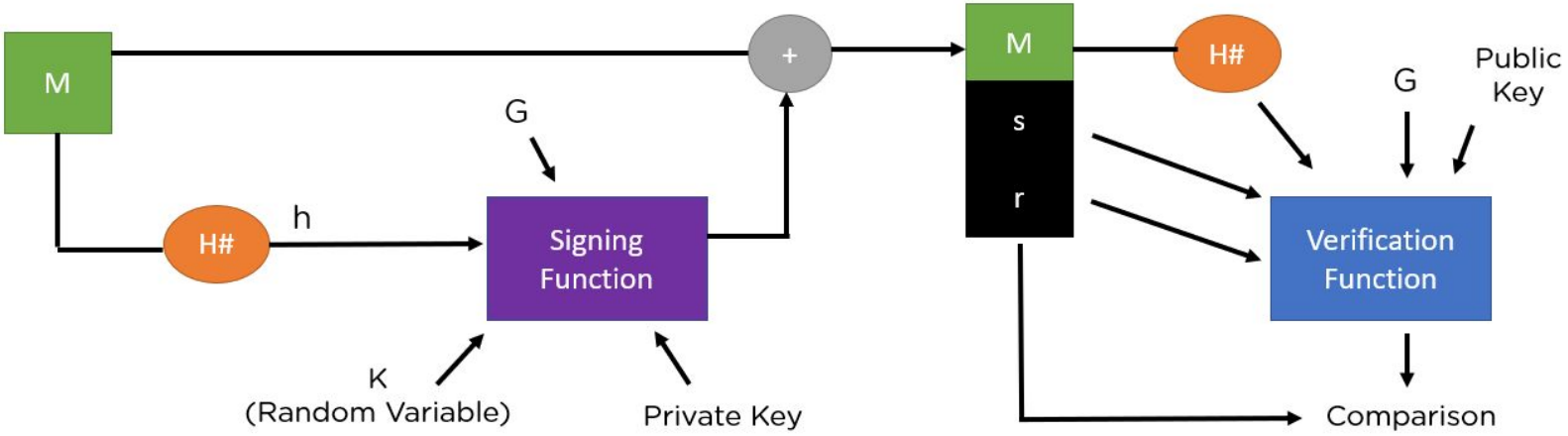
ea0ae29b3b2c20fc018aaca45c3746a057b893e7

- Digital Signatures are numbers
- They are not person dependent; they are document content dependent

Importance of Digital Signature

- **Ensures Authenticity:** Digital signatures ensure the authenticity of a message or transaction by proving that the message or signature was created using the private key associated with the digital signature.
- **Offers Non-repudiation:** A digital signature provides an entire record that a specific individual signed the document or transaction at a particular time. This feature prevents the individual from denying that they signed it.
- **Provides Security:** Digital signatures use encryption algorithms to protect the data from unauthorized access and tampering. The cryptographic techniques used by digital signatures also protect the data from being changed or manipulated during transmission.
- **Improves Efficiency:** Digital signatures can reduce the time and money spent on paperwork, printing, scanning, and mailing documents.
- **Enhances Compliance:** Digital signatures help organizations to meet regulatory and legal compliance requirements by providing an audit trail of signed documents.

What Is the DSA Algorithm?



Advantages of DSA



Highly robust



Better speed



Less storage



Patent Free

Cons of Using Digital Signature Algorithm

1. Limited Key Length: DSA has limited key lengths, which limit its use in specific applications.
2. Rigid Key Management: Key management is rigid and requires specific key lengths.
3. Limited Digital Certificate Support: DSA does not support certificates, which limits its use in specific applications.
4. Incremental signature algorithm: DSA is not an incremental signature algorithm, meaning it cannot be updated or changed once a signature is generated.
5. Relatively new: DSA is a relatively new algorithm and has not been extensively studied or vetted like some of the more established algorithms.
6. Impact on storage and transmission: DSA signatures can be larger than signatures created with other algorithms, which can impact storage and transmission efficiency.

RSA Signatures

- The **RSA** public-key cryptosystem provides a cryptographically secure **digital signature scheme** (sign + verify), based on the math of the **modular exponentiations** and discrete logarithms and the difficulty of the **integer factorization problem** (**IFP**).
- The **RSA sign / verify** process works as follows:
  - The RSA Sign algorithm computes a message **hash**, then **encrypts** the hash with the private key exponent to obtain the **signature**. The obtained signature is an **integer** number (the RSA encrypted message hash).
  - The **RSA verify** algorithm first computes the message **hash**, then **decrypts** the message **signature** with the **public key** exponent and compares the obtained **decrypted hash** with the **hash** of the signed message to ensure the signature is valid.

DSA VS RSA

- **DSA (Digital Signature Algorithm)**
  - DSA is a signature-only algorithm and requires a private key for signing and a public key for verifying.
  - DSA is a faster algorithm and is simpler to implement than RSA.
  - DSA is more secure than RSA as it provides message integrity and non-repudiation.
- **RSA (Rivest-Shamir-Adleman)**
  - RSA is a signature and encryption algorithm that can be used for both digital signatures and encryption.
  - RSA is a slower algorithm and is more challenging to implement than DSA.
  - RSA is less secure than DSA, which is vulnerable to chosen-ciphertext attacks and requires more computing power.

Other signature schemes

☞ **ECDSA** are derived from **ElGamal** signature scheme.

Digital Signatures – Applications

- Electronic Mail
- Data Storage
- Electronic fund transfer
- Software distribution
- Smart cards
- ISDN
- Time stamped signature
- Blind signatures

Digital Certificate

What is a digital certificate?

- A **digital certificate** is an electronic file that is tied to a cryptographic key pair and authenticates the identity of a website, individual, organization, user, device or server.
- This form of authentication is a type of cryptography that requires the use of public and private keys to validate users.
- A digital certificate contains the following identifiable information:
  - User's name
  - Company or department of user
  - IP (internet protocol) address or serial number of device
  - Copy of the public key from a certificate holder
  - Duration of time the certificate is valid for
  - Domain certificate is authorized to represent

Different types of digital certification

☞ **TLS/SSL certificates:**

- The TLS/SSL certificate is used to secure communications between a computer and the server, and it is hosted by the server.
- When a client computer seeks to access the server, the server will present the digital certificate to prove that it is authentic and the desired destination.

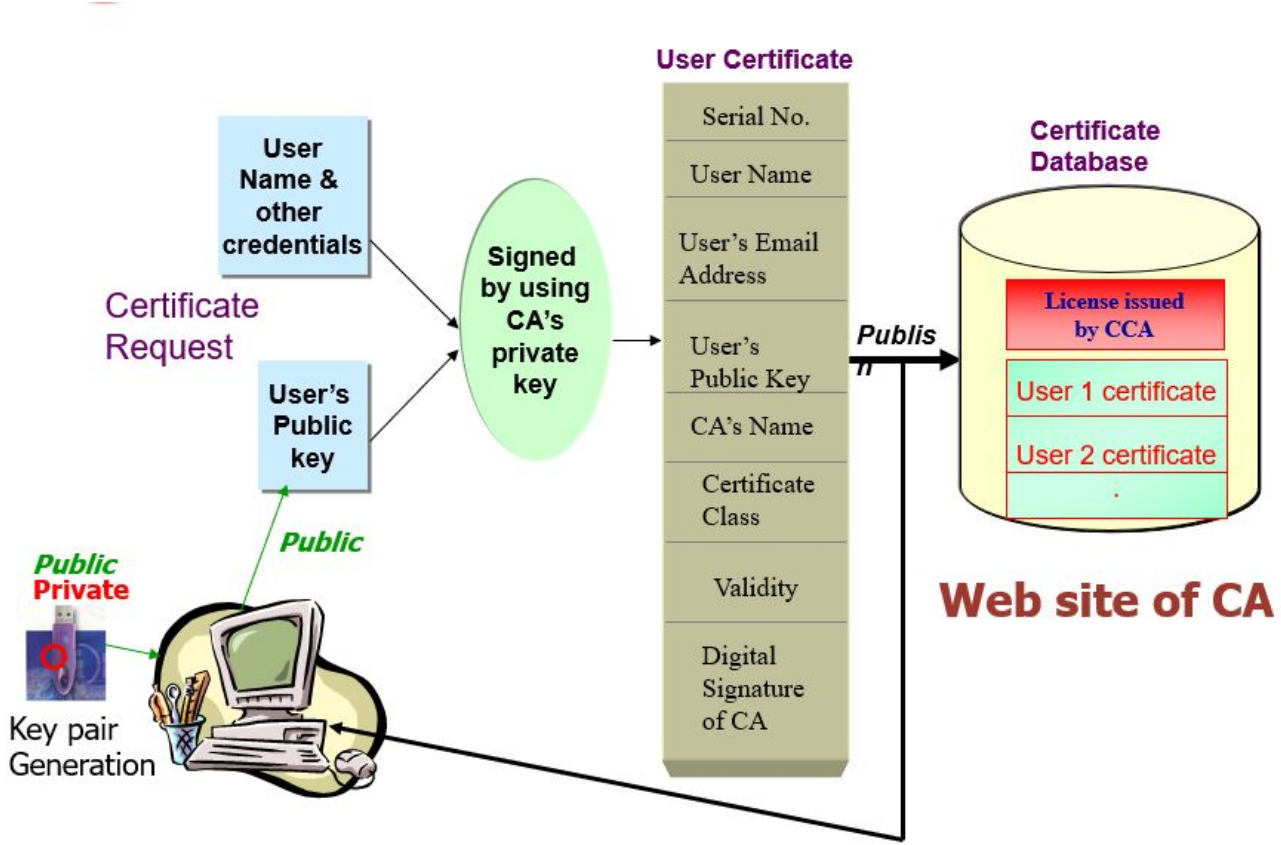
Different types of digital certification

- **Client certificates:**
  - This is a form of a digital ID that can identify one machine to another — a specific user to another user.
  - This can be used to allow a user to access a protected and secure database and also for email.
- **Code signing certificates:**
  - This type of digital certificate involves software or files.
  - The publisher or developer of software will sign it to validate its authenticity to users downloading it.

Who Can Issue a Digital Certificate?

- A Digital Certificate can be issued by the Certifying Authority also known as CA, which is a trusted third party for use by other parties.
- These organizations are trusted third parties whose primary role is to verify the identities of organizations, individuals, or devices and then issue digital certificates to confirm those identities.
- It means an individual who has been granted a license to issue a digital signature certificate under the law.

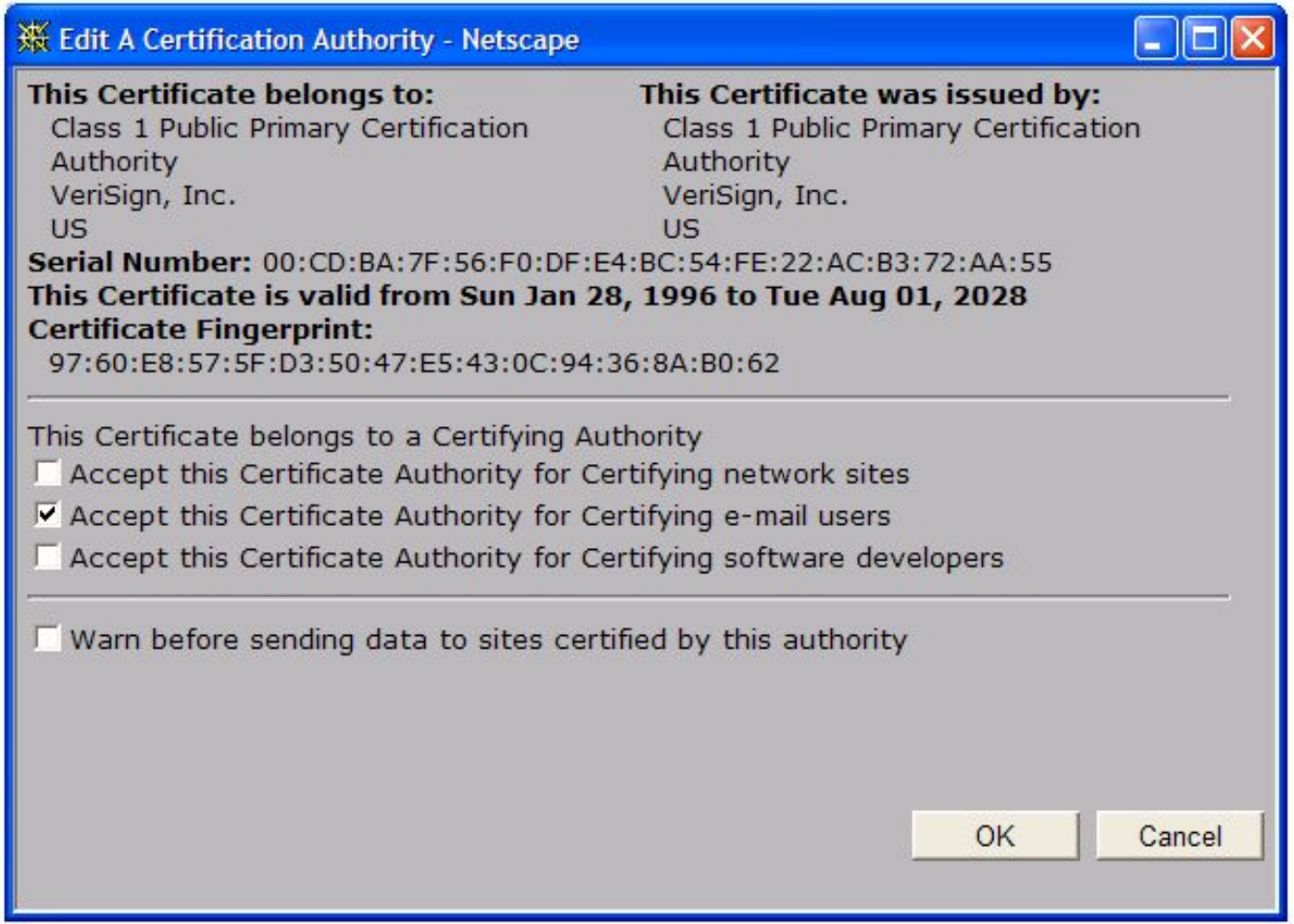
Digital certificate – Public key certification



How Do Digital Certificates Work?

1. An entity generates a pair of cryptographic keys: a private key and a public key.
2. The entity keeps the private key secret and submits the public key, along with identifying information, to a Certificate Authority.
3. The CA verifies the entity's identity through various means, which may include document checks, phone calls, or in-person meetings, depending on the type of certificate.
4. Once verified, the CA creates a digital certificate containing the public key and identity information.
5. The CA signs the certificate with its own private key, thereby vouching for the certificate's authenticity.
6. The signed certificate is issued to the entity and can be freely distributed.
7. When someone wants to communicate securely with the certificate holder, they can use the public key in the certificate to encrypt messages or verify digital signatures.

Sample NETSCAPE digital signature look like is as below.



Challenges and Considerations

- Certificate Management
- Private Key Security
- CA Trustworthiness
- Revocation Effectiveness

Challenge	Main question	Risk if poorly handled
Certificate Management	Is the certificate correctly issued, tracked and maintained?	Wrong or expired certificates may be trusted
Private Key Security	Is the signing key protected?	Attackers may create forged signatures
CA Trustworthiness	Can we trust the authority that issued the certificate?	Fraudulent identity-to-key bindings
Revocation Effectiveness	Can we identify certificates that should no longer be trusted?	Compromised keys may continue to be accepted

Thank you!

Follow us



isfcr.pesu



www.isfcr.pes.edu



ISFCR

PESU ISFCR

PESU Center for
Information Security, Forensics and Cyber Resilience

PES UNIVERSITY



PESU Center for
Information Security,
Forensics and
Cyber Resilience



Welcome to

PES University

Ring Road Campus, Bengaluru



PESU Center for
Information Security,
Forensics and
Cyber Resilience



Applied Cryptography

UE24CS342AA4

Dr. Shruti Jadon

LECTURE 25

A Note on Security



Emergency Exit



Assembly Point



Washroom



No Chatting



Phones on silent



No Sleeping

A Note on Security

- In this course, you will be exposed to information about security problems and vulnerabilities with computing systems and networks.
- To be clear, you are not to use this or any other similar information to test the security of, break into, compromise, or otherwise attack, any system or network without the express consent of the owner.
- In particular, you will comply with all my instructions when doing the labs.
  - My instructions are in consonance with applicable laws of India and PES University policies.
  - If in any doubt, please consult your professor!
- Any violation is at **YOUR RISK!**
  And may result in severe consequences.

Collision resistant hash function

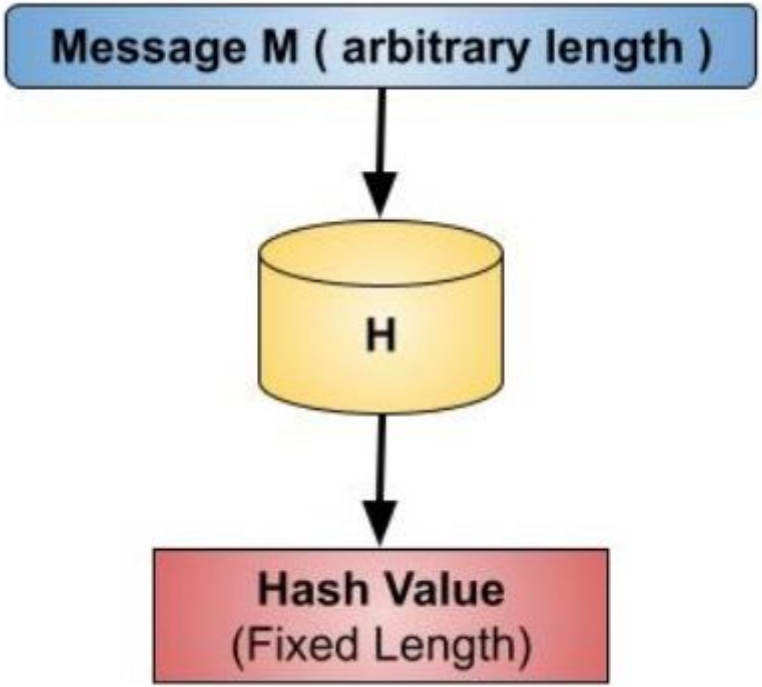
Hashing in Public Key Cryptography?

☞ A hash function is a serious mathematical process that holds a critical role in public key cryptography:

- Securely store passwords in a database,
- Ensure data integrity (in a lot of different applications) by indicating when data has been altered,
- Make secure authentication possible, and
- Organize content and files in a way that increases efficiency.

Hash functions

- A hash function is any function that can be used to map data of arbitrary size to fixed-size values.
- The values returned by a hash function are called hash values, hash codes, digests, or simply hashes.
- Hash functions are mathematical functions that transform or "map" a given data set into a bit string of fixed size, also known as the "hash value."
- Hash functions play a crucial role in various security applications, including password storage (hash values instead of passwords), digital signatures, and data integrity checks.


Message M (arbitrary length)
H
Hash Value
(Fixed Length)

Properties of a Strong Hash Algorithm

- **Determinism** — A hash algorithm should be deterministic, meaning that it always gives you an output of identical size regardless of the size of the input you started with.
- **Pre-Image Resistance** -it's infeasible to reverse a hash value to recover the original input plaintext message. Hence, the concept of hashes being irreversible, one-way functions.
- **Collision Resistance** — If two unique samples of input data result in identical outputs, it's known as a collision. This is bad news and means that the algorithm you're using to hash the data is broken and, therefore, insecure.

- **Avalanche Effect** — What this means is that any change made to an input, no matter how small, will result in a massive change in the output.
- **Hash Speed** — Hash algorithms should operate at a reasonable speed. In many situations, hashing algorithms should compute hash values quickly; this is considered an ideal property of a cryptographic hash function.

Iterated Cryptographic Hash Functions

- In an iterated cryptographic hash function scheme, a function with an input of fixed size is created and is used a required number of times.

- This fixed-size input function is known as the compression function. This function compresses an n-bit string to an m-bit string, preferably n>m.

Types

- Merkle-Damgard scheme
  - Message Digest
  - Secure Hash Algorithm
- Hash function based on block cipher
  - Rabin scheme
  - Davis meyer scheme
  - Matyas meyer oseas scheme
  - Miyaguchi-preneel scheme

Merkle-Damgard Scheme in Cryptography

- The Merkle-Damgard Scheme is an iterated hash function. It is used in the compression of messages to create message digests.
- MD scheme(discovered by Ralph Merkle) is used to build collision-resistant cryptographic hash functions from collision-resistant one-way compression functions.
- It is used in algorithms like SHA-1, SHA-256, etc.

Step 1

☞ The message length and padding are appended to create a larger message. This message should be evenly divisible in blocks of n bits. Here n is the size of the block to be processed by the compression algorithm.



Step2

☞ The message is now divided into ‘t’ blocks. Each block has a size of n bits. The blocks are named M1, M2, M3, M4, … Mt. Since there are t blocks, there will be t iterations, and each iteration would produce a digest. And the digests created at t iterations are called H1, H2, H3, … Ht.

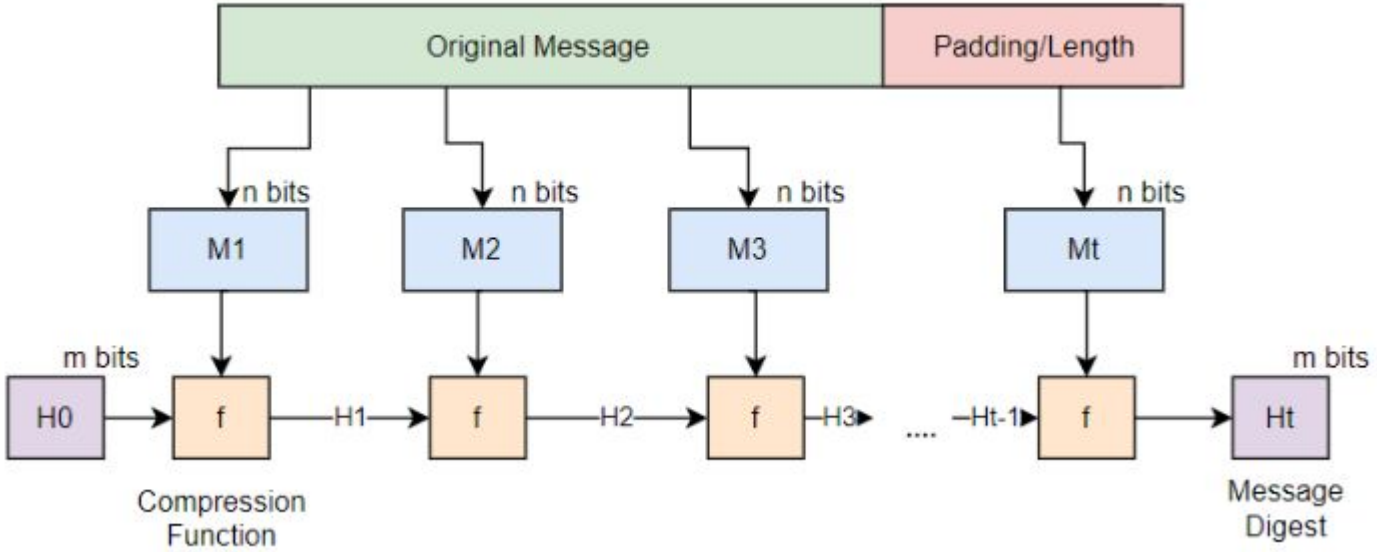


Step3

- There are t iterations of the compression function. At ith iteration, the compression takes in input the parameters Hi-t and Mi to create the digest Hi. Hi-1 is the digest created by the compression function in the previous iteration. For the first iteration, H0 is provided as the initialization digest value.

$$f(Hi\text{-}1, Mi) = Hi$$

- Thus in the first iteration, the compression function operates on the block of message M1 and the initialized digest (constant value) H0. This gives the digest H1, which along with M2, is then used to compute H2 and so on.



Step 4:

The digest Ht that is obtained in the end is treated as the digest for the original message M.

Other Hash Function Uses

- ☞ pseudorandom function (PRF)
  - Generate session keys, nonces
  - Produce key from password
  - Derive keys from master key cooperatively
- ☞ pseudorandom number generator (PRNG)
  - Vernam Cipher/OTP
  - S/Key, proof of “what you have” via messages

More Hash Function Uses

- ☞ to create a one-way password file
  - store hash of password not actual password
  - e.g., Unix, Windows NT, etc.
  - salt to deter precomputation attacks
  - Rainbow tables
- ☞ for intrusion detection and virus detection
  - keep & check hash of files on system
  - e.g., Tripwire

Lamport One-time Passwords

☞ Password safety in distributed system

server compromise does not compromise P

interception of authentication exchange does not compromise password either

☞ Alice picks Password PA

Hashes password N times, HN(PA)

Server stores (Alice, N, HN(PA))

Attacker can’t get PA from HN(PA)

Hash Function Requirements

Requirement	Description
Variable input size	H can be applied to a block of data of any size.
Fixed output size	H produces a fixed-length output.
Efficiency	H(x) is relatively easy to compute for any given x, making both hardware and software implementations practical.
Preimage resistant (one-way property)	For any given hash value h, it is computationally infeasible to find y such that H(y) = h.
Second preimage resistant (weak collision resistant)	For any given block x, it is computationally infeasible to find y ! x with H(y) = H(x).
Collision resistant (strong collision resistant)	It is computationally infeasible to find any pair (x, y) such that H(x) = H(y).
Pseudorandomness	Output of H meets standard tests for pseudorandomness

Attacks on Hash Functions

- have brute-force attacks and cryptanalysis
- a preimage or second preimage attack

  find y s.t. H(y) equals a given hash value
- collision resistance

  find two messages x & y with same hash so H(x) = H(y)
- hence value $2^{m/2}$ determines strength of hash code against brute-force attacks
- 128-bits inadequate, 160-bits suspect

MD5

History

- Initial checking of integrity – checksums, then CRC
- These are only good at detecting lost information due to hardware or transmission errors
- The checksum has no real protection of data integrity
- Easily circumvented or reverse-engineered

Potential Attack

- A wants to obtain privileges from B
- A generates two messages with the same hash values
- A presents an innocent message to B for his digital signature
- A applies the signature to the other malicious message with the same hash

One way

- The solution lay in one-way hashing algorithms
- These should keep two messages from colliding (having the same hash)
- They should also be sufficiently difficult to reverse-engineer

MD5

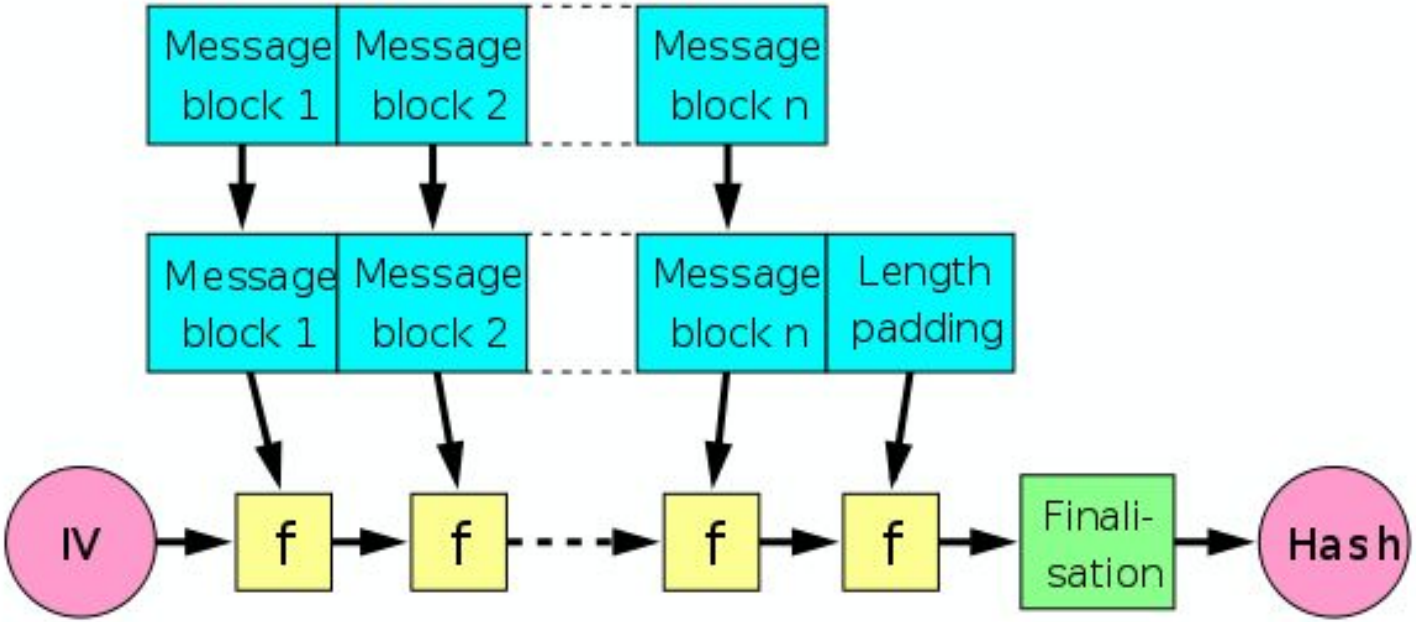
- The **MD5 message-digest algorithm** is a widely used hash function producing a 128-bit hash value.
- Has many vulnerabilities.
- It can still be used as a checksum to verify data integrity, but only against unintentional corruption.
- One basic requirement of any cryptographic hash function is that it should be computationally infeasible to find two distinct messages that hash to the same value. MD5 fails this requirement

History

- MD5 algorithm was developed by Professor Ronald L. Rivest in 1991. According to RFC 1321, “MD5 message-digest algorithm takes as input a message of arbitrary length and produces as output a 128-bit "fingerprint" or "message digest" of the input
- The MD5 algorithm is intended for digital signature applications, where a large file must be "compressed" in a secure manner before being encrypted with a private (secret) key under a public-key cryptosystem such as RSA.”

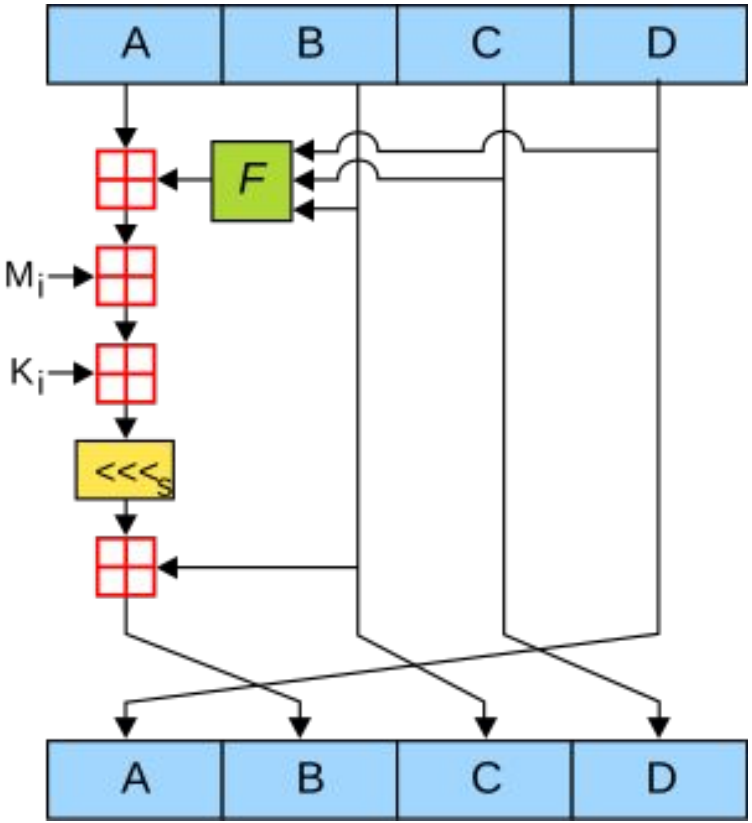
MD5 uses the Merkle–Damgård construction

- MD5 processes a variable-length message into a fixed-length output of 128 bits.
- The input message is broken up into chunks of 512-bit blocks (sixteen 32-bit words);
- the message is padded so that its length is divisible by 512.


Message block 1
Message block 2
Message block n
Message block 1
Message block 2
Message block n
Length padding
IV
f
f
f
f
Finali-sation
Hash

Working

- The main MD5 algorithm operates on a 128-bit state, divided into four 32-bit words, denoted *A*, *B*, *C*, and *D*. These are initialized to certain fixed constants.
- The main algorithm then uses each 512-bit message block in turn to modify the state.
- The processing of a message block consists of four similar stages, termed *rounds*; each round is composed of 16 similar operations based on a non-linear function *F*, modular addition, and left rotation.


A
B
C
D
F
M_i
K_i
<<<s
A
B
C
D

Implementation

- Step1 Append padding bits

  The input message is "padded" (extended) so that its length (in bits) equals to 448 mod 512. Padding is always performed, even if the length of the message is already 448 mod 512.

  Padding is performed as follows: a single "1" bit is appended to the message, and then "0" bits are appended so that the length in bits of the padded message becomes congruent to 448 mod 512. At least one bit and at most 512 bits are appended.

☞ Step2. Append length

A 64-bit representation of the length of the message is appended to the result of step1. If the length of the message is greater than 2^64, only the low-order 64 bits will be used.

The resulting message (after padding with bits and with b) has a length that is an exact multiple of 512 bits. The input message will have a length that is an exact multiple of 16 (32-bit) words.

☞ Step3. Initialize MD buffer

A four-word buffer (A, B, C, D) is used to compute the message digest. Each of A, B, C, D is a 32-bit register. These registers are initialized to the following values in hexadecimal, low-order bytes first):

word A: 01 23 45 67
word B: 89 ab cd ef
word C: fe dc ba 98
word D: 76 54 32 10

☞ Step4. Process message in 16-word blocks

Four functions will be defined such that each function takes an input of three 32-bit words and produces a 32-bit word output.

F (X, Y, Z) = XY or not (X) Z

G (X, Y, Z) = XZ or Y not (Z)

H (X, Y, Z) = X xor Y xor Z

I (X, Y, Z) = Y xor (X or not (Z))

Round	Function	Formula	Steps
1	F	$(B \wedge C) \vee (\neg B \wedge D)$	Steps 1 to 16
2	G	$(B \wedge D) \vee (C \wedge \neg D)$	Steps 17 to 32
3	H	$B \oplus C \oplus D$	Steps 33 to 48
4	I	$C \oplus (B \vee \neg D)$	Steps 49 to 64

MD5 vs. MD4

- A fourth round has been added.
- Each step has a unique additive constant.
- The function g in round 2 was changed from (XY v XZ v YZ) to (XZ v Y not(Z)).
- Each step adds in the result of the previous step.
- The order in which input words are accessed in rounds 2 and 3 is changed.
- The shift amounts in each round have been optimized. The shifts in different rounds are distinct.

MD5 Flaws

- ☞ Rainbow tables for passwords
- ☞ COLLISIONS!

MD5 Collisions

- 2004 Wang et. al delivered an algorithm that could produce collisions in a few hours on an IBM p690 cluster
- Algorithm was improved by Lenstra et. al in 2005 to a few hours on a single laptop
- A digest algorithm does not provide integrity if collisions are so simple to produce
- **SHA or Whirlpool should be considered until a replacement for MD5 can be found**

Libraries

☞ Below is a list of cryptography libraries that support MD5:

- Botan
- Bouncy Castle
- cryptlib
- Crypto++
- Libgcrypt
- Nettle
- OpenSSL
- wolfSSL

Thank you!

Follow us



isfcr.pesu



www.isfcr.pes.edu



ISFCR

PESU ISFCR

PESU Center for
Information Security, Forensics and Cyber Resilience

PES UNIVERSITY



PESU Center for
Information Security,
Forensics and
Cyber Resilience



Welcome to

PES University

Ring Road Campus, Bengaluru



PESU Center for
Information Security,
Forensics and
Cyber Resilience



Applied Cryptography

UE24CS342AA4

Dr. Shruti Jadon

LECTURE 26

A Note on Security



Emergency Exit



Assembly Point



Washroom



No Chatting



Phones on silent



No Sleeping

A Note on Security

- In this course, you will be exposed to information about security problems and vulnerabilities with computing systems and networks.
- To be clear, you are not to use this or any other similar information to test the security of, break into, compromise, or otherwise attack, any system or network without the express consent of the owner.
- In particular, you will comply with all my instructions when doing the labs.
  - My instructions are in consonance with applicable laws of India and PES University policies.
  - If in any doubt, please consult your professor!
- Any violation is at **YOUR RISK!**
  And may result in severe consequences.

SHA

Secure hash algorithm

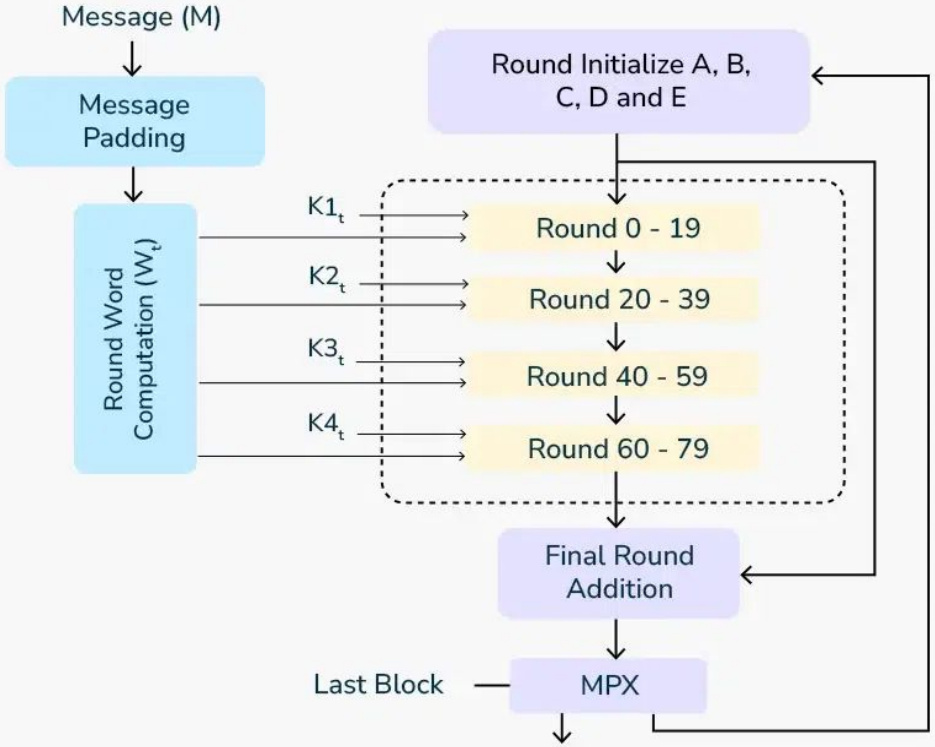
History

- SHA originally designed by NIST & NSA in 1993
- was revised in 1995 as SHA-1
- US standard for use with DSA signature scheme
- standard is FIPS 180-1 1995, also Internet RFC3174
- nb. the algorithm is SHA, the standard is SHS
- based on design of MD5 with key differences
- produces 160-bit hash values
- recent 2005 results on security of SHA-1 have raised concerns on its use in future applications

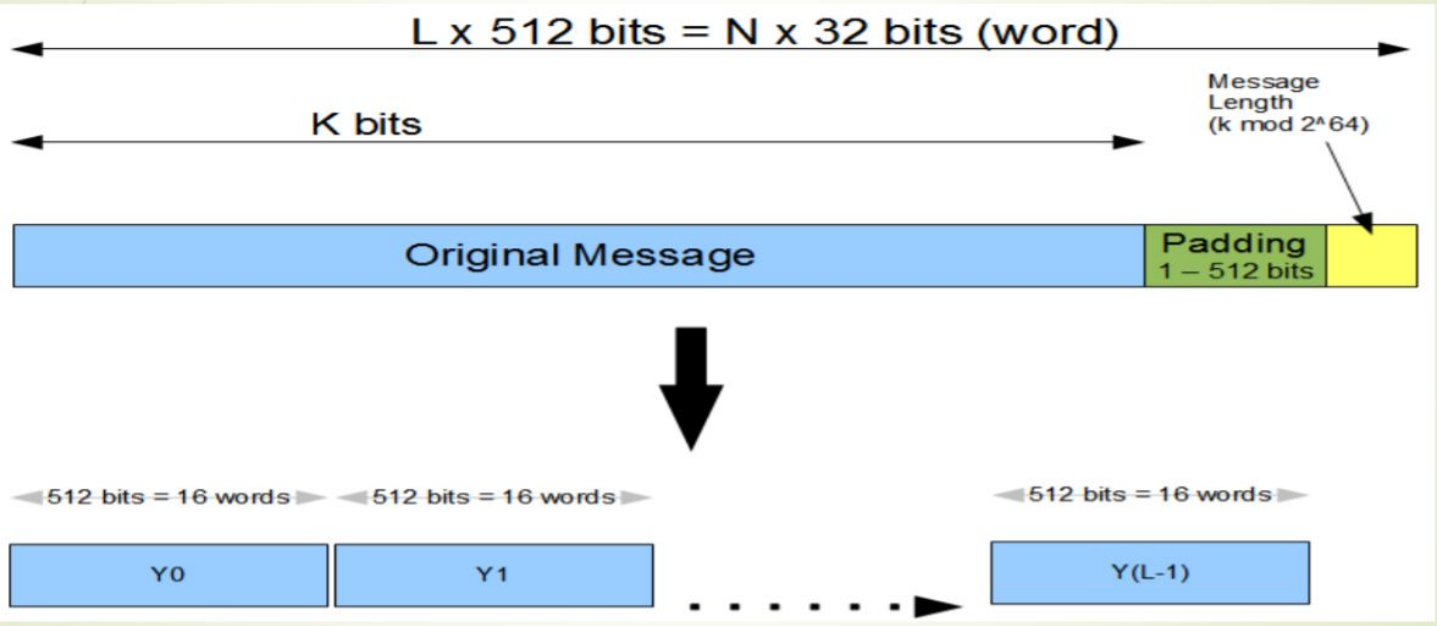
SHA-1 Hash

- SHA-1 or Secure Hash Algorithm 1 is a cryptographic algorithm that takes an input and produces a 160-bit (20-byte) hash value.
- This hash value is known as a message digest. This message digest is usually then rendered as a hexadecimal number which is 40 digits long.
- It is a U.S. Federal Information Processing Standard and was designed by the United States National Security Agency. SHA-1 is been considered insecure since 2005.
- Major tech giants browsers like Microsoft, Google, Apple, and Mozilla have stopped accepting SHA-1 SSL certificates by 2017.

SHA-1 Algorithm Block Diagram



Divide the input into 512-bit blocks



Initialize chaining variables

Chaining Variables	Hex values
A	01 23 45 67
B	89 AB CD EF
C	FE DC BA 98
D	76 54 32 10
E	C3 D2 E1 F0

Process Blocks

Steps involved

- Copy chaining variables A-E into variables a-e.
- Divide current 512-bit block into 16 subblocks of 32-bits.
- SHA has 4 rounds, each consisting of 20 steps.
- Each round takes 3 inputs-
  - 512-bit block,
  - The register abcde
  - A constant K[t] (where t= 0 to 79)
- SHA has a total of 80 iterations (4 rounds X 20 - iterations).

1 Iteration of SHA

- Each iteration consists of following operations:-

  abcde = (e +Process P + S5 (a) + W[t] + K[t]), a, S30(b) , c , d

- Where, abcde = The register made up of 5 variables a, b, c, d, e.
- Process P = The logic operation.
- St = Circular-left shift of 32-bit sub-block by t bits.
- W[t[= A 32-bit derived from the current 32-bit sub-block.
- K[t] = One of the five additive constants.

Process P in each SHA round

Round	Process P
1	(b AND c) OR ((NOT b) AND (d))
2	b XOR c XOR d
3	(b AND c) OR (b AND d) OR (c AND d)
4	b XOR c XOR d

Working Procedure

1. **Message (M)**:
    1. The original input message that needs to be hashed.
2. **Message Padding**:
    1. The initial step where the message is padded to ensure its length is congruent to 448 modulo 512. This step prepares the message for processing in 512-bit blocks.
3. **Round Word Computation (WtW_tWt)**:
    1. After padding, the message is divided into blocks of 512 bits, and each block is further divided into 16 words of 32 bits. These words are then expanded into 80 32-bit words, which are used in the subsequent rounds.

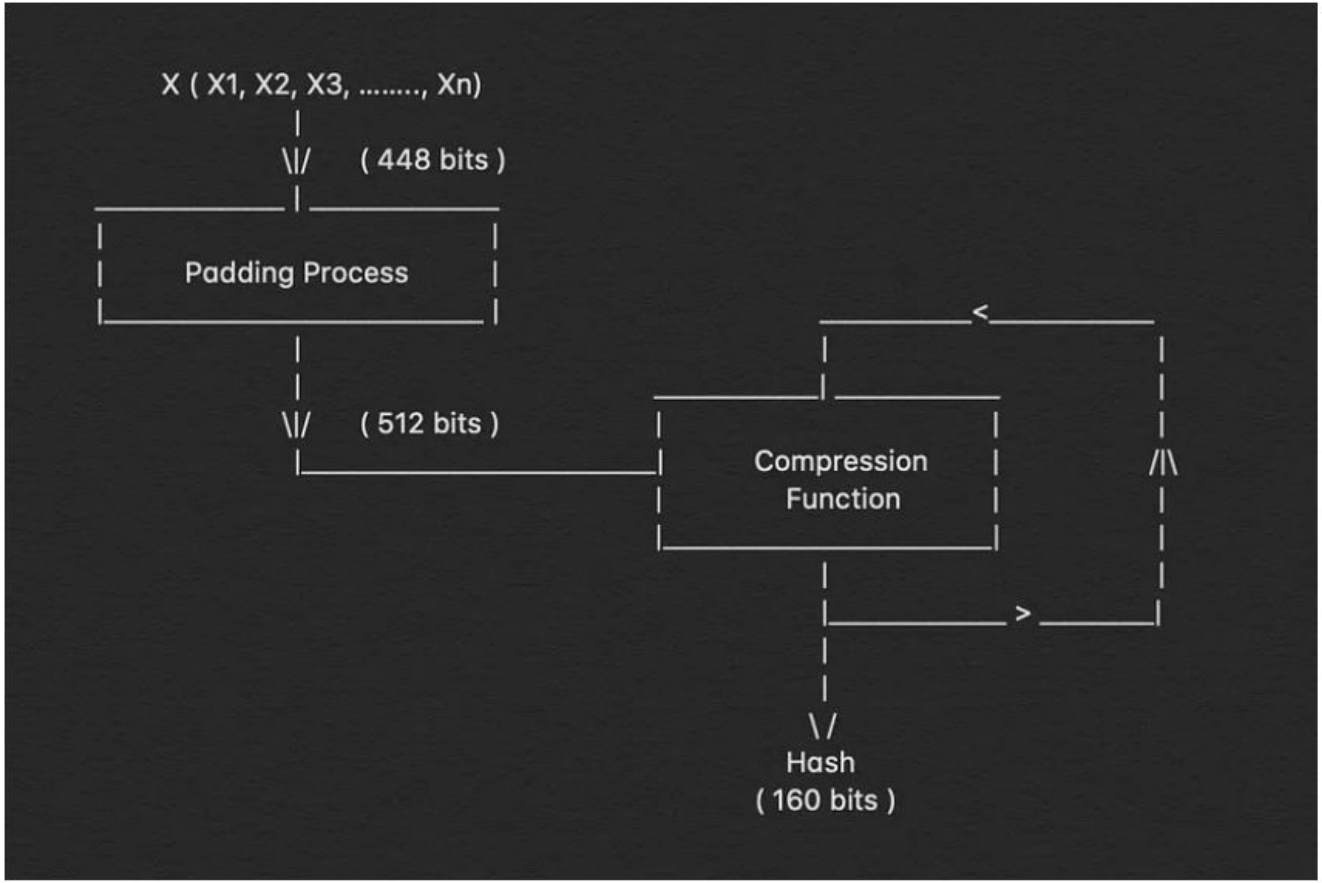
4. **Round Initialize (A, B, C, D, and E)**:
    4. Initialization of five working variables (A, B, C, D, and E) with specific constant values. These variables are used to compute the hash value iteratively.
5. **Round Constants (KtK_tKt)**:
    4. SHA-1 uses four constant values (K1K_1K1, K2K_2K2, K3K_3K3, K4K_4K4), each applied in a specific range of rounds:
        5. K1K_1K1 for rounds 0-19
        6. K2K_2K2 for rounds 20-39
        7. K3K_3K3 for rounds 40-59
        8. K4K_4K4 for rounds 60-79

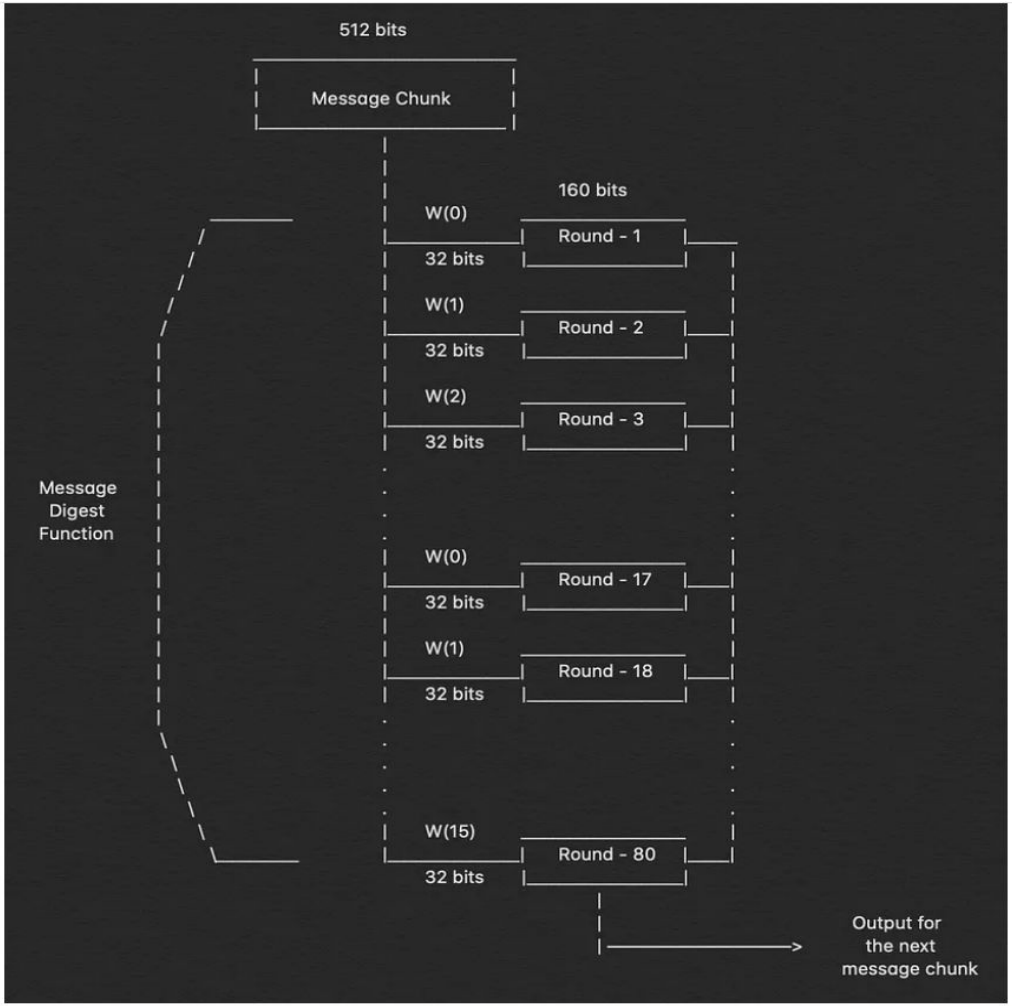
6. **Rounds (0-79)**:
    6. The main computation loop of SHA-1, divided into four stages (each corresponding to one of the constants K1K_1K1 to K4K_4K4). In each round, a combination of logical functions and operations is performed on the working variables (A, B, C, D, and E) using the words generated in the previous step.
7. **Final Round Addition**:
    6. After all 80 rounds, the resulting values of A, B, C, D, and E are added to the original hash values to produce the final hash.
8. **MPX (Multiplexing)**:
    6. Combines the results from the final round addition to form the final message digest.

SHA-1

- Digest sizes: 160 bits
- Block sizes: 512 bits
- Certification: FIPS PUB 180-4, CRYPTREC (Monitored)
- Rounds: 80
- Structure: Merkle–Damgård construction
- First published: 1993 (SHA-0), 1995 (SHA-1)


Arbitrary Message
SHA-1 Hash
Hash
(160 bits)


X (X1, X2, X3, ..., Xn)
(448 bits)
Padding Process
(512 bits)
Compression
Function
Hash
(160 bits)


512 bits
Message Chunk
160 bits
W(0)
Round - 1
32 bits
W(1)
Round - 2
32 bits
W(2)
Round - 3
32 bits
Message Digest Function
W(0)
Round - 17
32 bits
W(1)
Round - 18
32 bits
W(15)
Round - 80
32 bits
Output for the next message chunk

Versions And parameters	SHA-1	SHA-224	SHA-256	SHA-384	SHA-512
Message Digest size	160	224	256	384	512
Message size	$<2^{64}$	$<2^{64}$	$<2^{64}$	$<2^{128}$	$<2^{128}$
Block size	512	512	512	1024	1024
Word size	32	32	32	64	64
Number of Steps	80	64	64	80	80

Cryptographic Hashes

Merkle–Damgård Construction

Cryptographic Hash Functions

- A **cryptographic hash function** converts an input of arbitrary length into a fixed-length output called a **hash value** or **digest**.
- The same input always produces the same hash.
- A small change in the input should produce a significantly different hash.
- A secure hash function should make it computationally difficult to find **collisions**.
- Hash functions are widely used for **integrity, authentication, digital signatures, and password-related applications**.

Example

Message:

"Hello"

↓ Hash Function

Hash:

2cf24dba5fb0a30e…

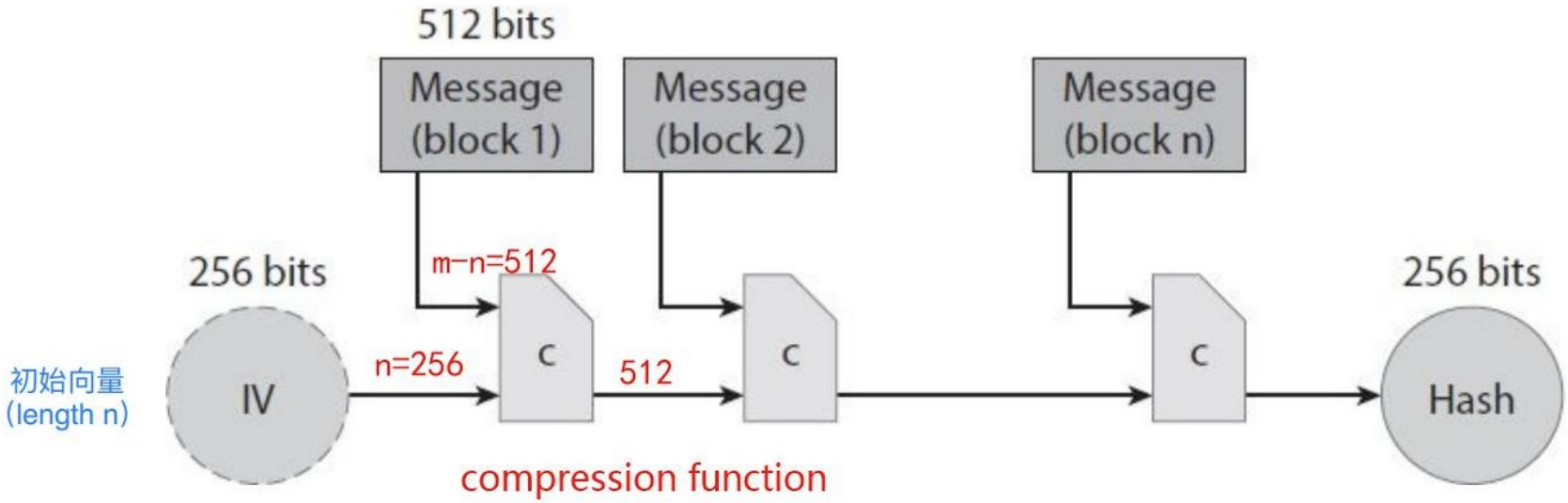
Change one character:

"hello"

↓

different hash

Example



SHA-256 hash function (simplified). SHA-256 uses the Merkle-Damgård transform to turn a fixed-length collision-resistant compression function into a hash function that accepts arbitrary-length inputs. The input is padded, so that its length is a multiple of 512 bits. IV stands for initialization vector.

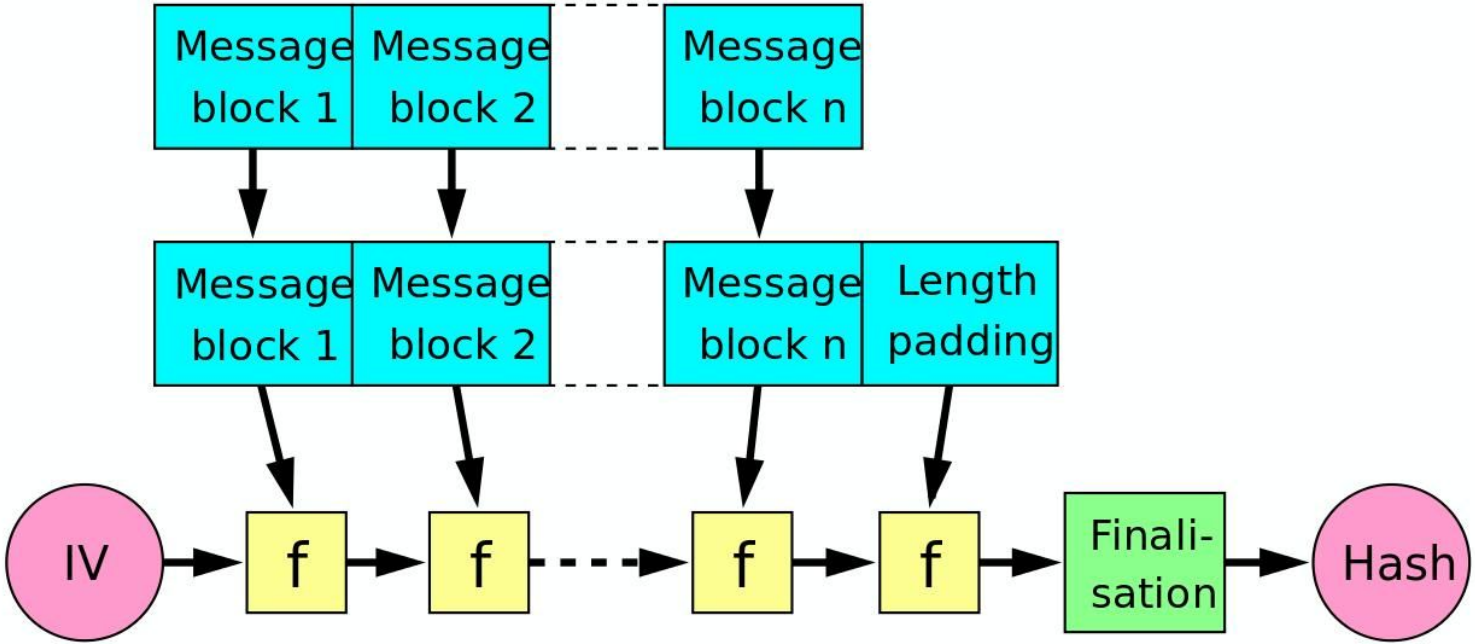
Why Merkle–Damgård Construction?

- Hash functions need to process messages of **arbitrary length**.
- A compression function normally works on **fixed-size inputs**.
- Merkle–Damgård provides a method to build a hash function for arbitrary-length messages.
- The message is divided into **fixed-size blocks**.
- Each block is processed sequentially using a **compression function**.
- The output from one stage becomes the input state for the next stage.

Compression Function

- A **compression function** takes a fixed-size internal state and a fixed-size message block as input.
- It produces a new fixed-size internal state.
- It is called "compression" because the output state is smaller than the combined input.
- The compression function is repeatedly applied to each message block.
- Its **collision resistance** is fundamental to the security of the resulting Merkle–Damgård hash.

Compression Function



Merkle–Damgård Construction

- The message is first **padded** to an appropriate block length.
- The padded message is divided into blocks:

  M_1, M_2, ..., M□.
- A fixed **Initialization Vector (IV)** provides the initial state.
- Each message block is processed by the compression function.
- The output of one round becomes the input state of the next round.
- The final internal state becomes the **hash value**.

Main Formula

$$H_0 = IV$$

$$H_i = f(H_{i-1}, M_i)$$

$$Hash(M) = H_n$$

The Merkle-Damgard Scheme in Cryptography turns a fixed-input collision-resistant compression function f into a hash function that accepts messages of arbitrary length.

- **M**: The original input message.
- **Pad(M)**: The padded message, split into equal-sized blocks $M_1, M_2, \ldots, M_t$ of length b.
- **IV or H_0**: The fixed Initial Value (a starting chaining variable of length n).
- **f**: The compression function that takes a b-bit message block and an n-bit previous chaining value H_{i-1}, outputting a new n-bit chaining value H_i

$$f : \{0,1\}^b \times \{0,1\}^n \rightarrow \{0,1\}^n$$

Padding and Length Strengthening

- The compression function processes **fixed-size blocks**.
- The message may not naturally have the required length.
- Padding makes the message length a suitable multiple of the block size.
- Merkle–Damgård strengthening also encodes the **original message length**.
- This prevents certain ambiguous encodings and is important to the security argument.

Original Message

↓

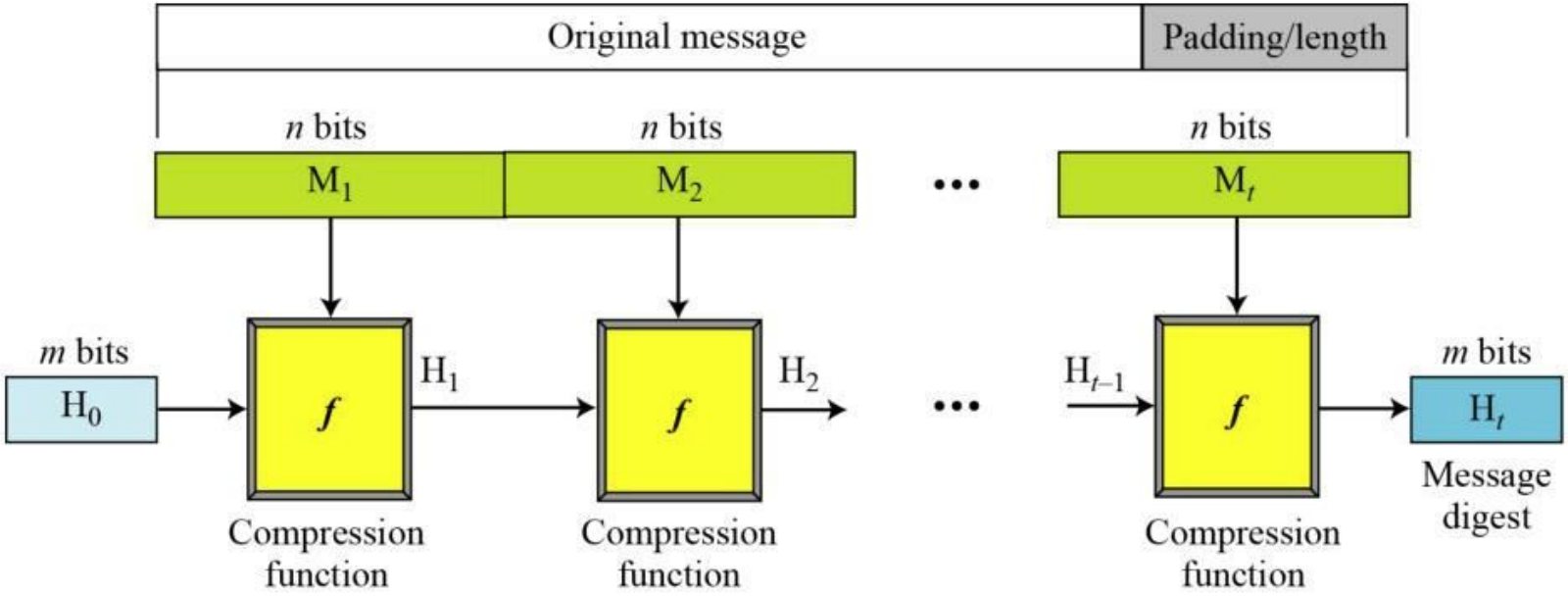
Message + Padding + Length

↓

Fixed-Size Blocks

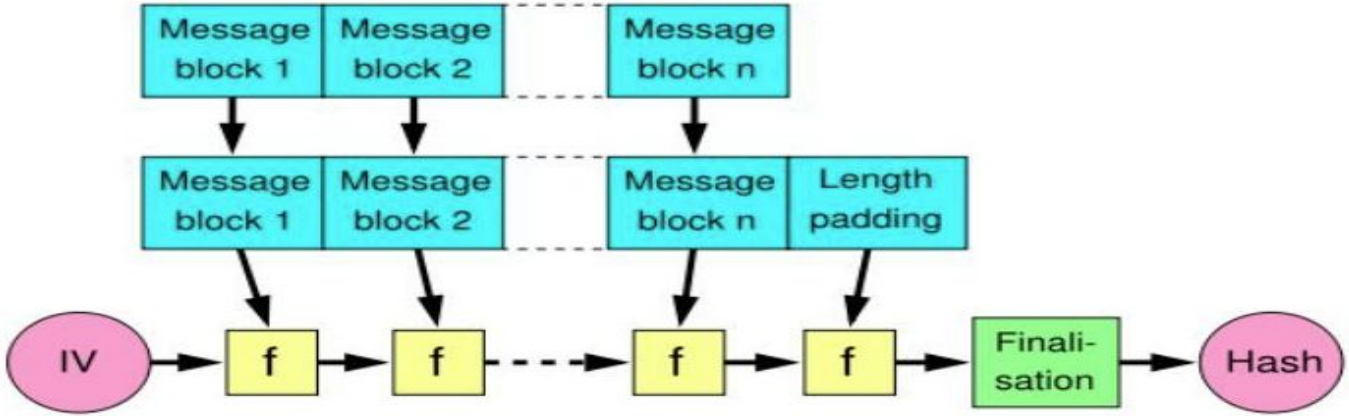
Padding and Length Strengthening

Merkle-Damgard Scheme



Working Example

- Message is divided into fixed-size blocks and padded
- Uses a compression function f, which takes a chaining variable (of size of hash output) and a message block, and outputs the next chaining variable
- Final chaining variable is the hash value



$$M=m_1 m_2 \ldots m_n;\ C_0=IV,\ C_{i+1}=f(C_i,m_i);\ H(M)=C_n$$

Security of Merkle–Damgård

- Security depends heavily on the **compression function**.
- A properly padded Merkle–Damgård construction preserves **collision resistance** when the underlying compression function is collision resistant.
- Finding two different messages with the same final hash should therefore be computationally difficult.
- The **IV and padding rules** are important parts of the construction.
- Proper length strengthening is required for the standard security argument.

Examples, Advantages & Limitation

Examples

Merkle–Damgård-style constructions have been used in several well-known hash families, including:

- **MD5**
- **SHA-1**
- **SHA-2 family**

However, **MD5 and SHA-1 are no longer considered secure against collision attacks**. The construction itself should therefore be distinguished from the security status of particular algorithms

Advantages

- Handles **arbitrary-length messages**.
- Simple iterative structure.
- Built from a fixed-size compression function.
- Provides a clear security reduction to the compression function.

Examples, Advantages & Limitation

Important Limitation

- Classic Merkle–Damgård designs can be susceptible to **length-extension attacks** in certain constructions.
- This is one reason constructions such as **HMAC** are preferred when a hash function is used for message authentication.

Final Takeaway

Merkle–Damgård turns a fixed-size compression function into a hash function capable of processing messages of arbitrary length.

Q&A

Q1. If a message is larger than the input size supported by a compression function, how can Merkle–Damgård process it?
Answer: It **divides the message into fixed-size blocks** and processes them one by one. The output of each block becomes the input state for the next block

Q2. What happens if we change just one bit in the first message block?
Answer: The change affects the **chaining state**, which then affects all subsequent blocks. Therefore, the final hash should change significantly due to the **avalanche effect**.

Q3. Why is padding and length strengthening needed in Merkle–Damgård?
Answer: Padding makes the message fit the required block size, while **length strengthening records the original message length**. This helps avoid ambiguity and is important for the security construction.

Q&A

Q1. If a message is larger than the input size supported by a compression function, how can Merkle–Damgård process it?
Answer: It **divides the message into fixed-size blocks** and processes them one by one. The output of each block becomes the input state for the next block

Q2. What happens if we change just one bit in the first message block?
Answer: The change affects the **chaining state**, which then affects all subsequent blocks. Therefore, the final hash should change significantly due to the **avalanche effect**.

Q3. Why is padding and length strengthening needed in Merkle–Damgård?
Answer: Padding makes the message fit the required block size, while **length strengthening records the original message length**. This helps avoid ambiguity and is important for the security construction.

Thank you!

Follow us



isfcr.pesu



www.isfcr.pes.edu



ISFCR

PESU ISFCR

PESU Center for
Information Security, Forensics and Cyber Resilience

PES UNIVERSITY